

- footer.component.ts : src/app/components/footer/ footer.component.ts

```
import { Component } from '@angular/core';
import { TranslateService } from '@ngx-translate/core';

@Component({
  selector: 'app-footer',
  templateUrl: './footer.component.html',
  styleUrls: ['./footer.component.scss']
})
export class FooterComponent {
  currentYear = new Date().getFullYear();

  constructor(private translate: TranslateService) { }
}
```

- footer.component.html : src/app/components/footer/ footer.component.html

```
<footer class="footer bg-dark text-light py-5">
  <div class="container">
    <div class="row">
      <!-- Section principale du footer -->
      <div class="col-lg-4 col-md-6 mb-4 mb-lg-0">
        <div class="d-flex align-items-center mb-3">
          <span class="h5 mb-0 text-white">{{ 'app.title' | translate
}}</span>
        </div>
        <p class="text-muted mb-3">
          {{ 'app.subtitle' | translate }}
        </p>
        <p class="text-muted small">
          {{ 'app.description' | translate }}
        </p>
      </div>

      <!-- Section Procédures -->
      <div class="col-lg-2 col-md-6 col-sm-6 mb-4 mb-lg-0">
        <h6 class="text-white mb-3">{{ 'navigation.procedures' |
translate }}</h6>
        <ul class="list-unstyled">
          <li><a routerLink="/procedures/police-justice" class="text-
muted footer-link">{{
              'categories.police_justice.name' | translate
}}</a></li>
          <li><a routerLink="/procedures/family" class="text-muted
footer-link">{{ 'categories.family.name' |
              translate }}</a></li>
```

```

        <li><a routerLink="/procedures/transport" class="text-muted
footer-link">{{
            'categories.transport.name' | translate }}</a></li>
        <li><a routerLink="/procedures/education" class="text-muted
footer-link">{{
            'categories.education.name' | translate }}</a></li>
        <li><a routerLink="/procedures" class="text-muted footer-
link">
            <i class="fas fa-arrow-right me-1"></i>{{
'buttons.view_all' | translate }}
            </a></li>
        </ul>
    </div>

    <!-- Section Services Populaires -->
    <div class="col-lg-3 col-md-6 col-sm-6 mb-4 mb-lg-0">
        <h6 class="text-white mb-3">Popular Services</h6>
        <ul class="list-unstyled">
            <li><a routerLink="/service-detail/passport" class="text-
muted footer-link">{{
                'services.passport.name' | translate }}</a></li>
            <li><a routerLink="/service-detail/birth-certificate-copy"
class="text-muted footer-link">{{
                'services.birth_certificate_copy.name' | translate
            }}</a></li>
            <li><a routerLink="/service-detail/national-id" class="text-
muted footer-link">{{
                'services.national_id.name' | translate }}</a></li>
            <li><a routerLink="/search-files" class="text-muted footer-
link">
                <i class="fas fa-search me-1"></i>{{
'navigation.search_files' | translate }}
                </a></li>
        </ul>
    </div>

    <!-- Section Support et Contact -->
    <div class="col-lg-3 col-md-6">
        <h6 class="text-white mb-3">{{ 'help.contact_support' | translate
    }}</h6>
        <ul class="list-unstyled mb-3">
            <li><a href="#" class="text-muted footer-link">{{ 'help.faq'
| translate }}</a></li>
            <li><a href="#" class="text-muted footer-link">{{
'help.user_guide' | translate }}</a></li>
            <li><a href="#" class="text-muted footer-link">Contact
Us</a></li>
            <li><a href="#" class="text-muted footer-link">Privacy
Policy</a></li>

```

```

        <li><a href="#" class="text-muted footer-link">Terms of
Service</a></li>
    </ul>

    <!-- Informations de contact -->
    <div class="contact-info">
        <div class="contact-item mb-2">
            <i class="fas fa-phone text-primary me-2"></i>
            <small class="text-muted">+237(0) 233 47 50 51 / +237(6)
70 60 42 15</small>
        </div>
        <div class="contact-item mb-2">
            <i class="fas fa-envelope text-primary me-2"></i>
            <small class="text-muted">info@mkgov.cm</small>
        </div>
        <div class="contact-item">
            <i class="fas fa-clock text-primary me-2"></i>
            <small class="text-muted">24/7 Online Services</small>
        </div>
    </div>
</div>

<!-- Section Administrations Fournisseuses -->
<div class="separator-line my-4"></div>
<div class="row">
    <div class="col-12">
        <h6 class="text-white mb-3 text-center">Government Agencies &
Service Providers</h6>
        <div class="row text-center">
            <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
                <a href="https://www.dgsn.cm" target="_blank"
class="agency-link">
                    <small class="text-muted">DGSN</small>
                </a>
            </div>
            <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
                <a href="https://www.minjustice.gov.cm" target="_blank"
class="agency-link">
                    <small class="text-muted">Ministry of Justice</small>
                </a>
            </div>
            <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
                <a href="https://www.mintransport.gov.cm" target="_blank"
class="agency-link">
                    <small class="text-muted">Ministry of
Transport</small>
                </a>
            </div>
        </div>
    </div>

```

```
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.minesup.gov.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Ministry of Higher
Education</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.cfce.cm" target="_blank"
class="agency-link">
                <small class="text-muted">CFCE</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.minfopra.gov.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Ministry of Public
Service</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.mindcaf.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Ministry of State
Property</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.minrex.gov.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Ministry of External
Relations</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.minsante.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Ministry of Public
Health</small>
            </a>
        </div>
        <div class="col-lg-2 col-md-3 col-sm-4 col-6 mb-3">
            <a href="https://www.minat.gov.cm" target="_blank"
class="agency-link">
                <small class="text-muted">Prefecture Services</small>
            </a>
        </div>
    </div>
</div>
```

```

</div>

<!-- Ligne de séparation -->
<hr class="my-4 border-secondary">

<!-- Section inférieure -->
<div class="row align-items-center">
    <div class="col-lg-6 col-md-12 mb-3 mb-lg-0">
        <p class="text-muted mb-0 small">
            © {{ currentYear }} MK Gov - Digital Government Portal. All
rights reserved.
        </p>
        <p class="text-muted mb-0 small">
            Republic of Cameroon - Ministry of Public Service and
Administrative Reform
        </p>
    </div>

    <div class="col-lg-6 col-md-12 text-lg-end">
        <!-- Réseaux sociaux -->
        <div class="social-links">
            <a href="#" class="text-muted me-2" title="Facebook" aria-
label="Facebook">
                <i class="fab fa-facebook-f"></i>
            </a>
            <a href="#" class="text-muted me-2" title="Twitter" aria-
label="Twitter">
                <i class="fab fa-twitter"></i>
            </a>
            <a href="#" class="text-muted me-2" title="LinkedIn" aria-
label="LinkedIn">
                <i class="fab fa-linkedin-in"></i>
            </a>
            <a href="#" class="text-muted me-2" title="YouTube" aria-
label="YouTube">
                <i class="fab fa-youtube"></i>
            </a>
            <a href="#" class="text-muted" title="Instagram" aria-
label="Instagram">
                <i class="fab fa-instagram"></i>
            </a>
        </div>

        <!-- Badges de certification -->
        <div class="certification-badges mt-3">
            <span class="badge bg-success me-2">
                <i class="fas fa-shield-alt me-1"></i>Secure
            </span>
            <span class="badge bg-primary me-2">

```



```

    transition: all 0.3s ease;
    display: block;
    padding: 0.25rem 0;
    border-radius: 4px;

    &:hover {
        color: var(--primary-color) !important;
        background-color: rgba(26, 115, 232, 0.1);
        padding-left: 0.5rem;
        transform: translateX(5px);
    }

    i {
        color: var(--primary-color);
        font-size: 0.9rem;
    }
}

// Agency links - Fixed color visibility
.agency-link {
    color: #adb5bd !important;
    text-decoration: none;
    transition: all 0.3s ease;
    display: block;
    padding: 0.5rem;
    border-radius: 8px;
    border: 1px solid transparent;

    &:hover {
        color: var(--primary-color) !important;
        background-color: rgba(26, 115, 232, 0.1);
        border-color: var(--primary-color);
        transform: translateY(-2px);
        box-shadow: 0 4px 8px rgba(26, 115, 232, 0.2);
    }

    small {
        color: #adb5bd !important;
        font-weight: 500;
        font-size: 0.8rem;
    }
}

// Contact info styling - Fixed color visibility
.contact-info {
    .contact-item {
        display: flex;
        align-items: center;
        margin-bottom: 0.5rem;
    }
}

```

```

    i {
        font-size: 0.9rem;
        width: 16px;
        text-align: center;
        color: var(--primary-color) !important;
    }

    small {
        color: #adb5bd !important;
        font-size: 0.85rem;
    }
}

// Copyright and legal text - Fixed color visibility
.row {
    p {
        color: #adb5bd !important;

        &.text-muted {
            color: #adb5bd !important;
        }
    }
}

// Legal notice - Fixed color visibility
.legal-notice {
    background-color: rgba(255, 255, 255, 0.02) !important;
    border: 1px solid rgba(255, 255, 255, 0.1) !important;
    border-radius: 8px;

    small {
        color: #adb5bd !important;
        line-height: 1.5;

        i.fa-info-circle {
            font-size: 1rem;
            color: var(--primary-color) !important;
        }

        strong {
            color: #ffffff !important;
        }
    }
}

// Social links styling
.social-links {

```



```

display: flex;
justify-content: flex-end;
gap: 1rem;

@media (max-width: 768px) {
  justify-content: center;
  margin-bottom: 1rem;
}

a {
  width: 40px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 50%;
  background-color: rgba(255, 255, 255, 0.1);
  transition: all 0.3s ease;
  color: #adb5bd !important;

  &:hover {
    color: white !important;
    background-color: var(--primary-color);
    transform: translateY(-3px);
    box-shadow: 0 4px 12px rgba(26, 115, 232, 0.3);
  }

  i {
    font-size: 1.1rem;
    color: inherit !important;
  }
}

}

// Separator line
.separator-line {
  height: 2px;
  background: linear-gradient(90deg,
    transparent 0%,
    var(--primary-color) 20%,
    var(--secondary-color) 50%,
    var(--primary-color) 80%,
    transparent 100%);
  border: none;
  margin: 2rem 0;
}

// Certification badges
.certification-badges {

```

```

display: flex;
justify-content: flex-end;
gap: 0.5rem;
flex-wrap: wrap;

@media (max-width: 768px) {
  justify-content: center;
}

.badge {
  font-size: 0.75rem;
  padding: 0.5rem 0.75rem;
  border-radius: 15px;

  i {
    font-size: 0.7rem;
    color: inherit !important;
  }
}

}

// Headings in footer
h6 {
  font-weight: 600;
  margin-bottom: 1rem;
  color: white !important;
  font-size: 1rem;
  letter-spacing: 0.5px;
  text-transform: uppercase;

  &::after {
    content: '';
    display: block;
    width: 30px;
    height: 2px;
    background-color: var(--primary-color);
    margin-top: 0.5rem;
  }
}

// Lists styling
ul.list-unstyled {
  li {
    margin-bottom: 0.5rem;

    &.last-child {
      margin-bottom: 0;
    }
  }
}

```

```
}

// Text color overrides for all muted text elements
.text-muted {
  color: #adb5bd !important;
}

// Responsive design
@media (max-width: 768px) {
  .agency-link {
    margin-bottom: 0.5rem;
  }

  .text-lg-end {
    text-align: center !important;
  }

  .social-links {
    margin-bottom: 1rem;
  }

  .certification-badges {
    margin-top: 1rem;
  }
}

@media (max-width: 576px) {
  padding: 2rem 0 !important;

  h6 {
    font-size: 0.9rem;
    margin-bottom: 0.75rem;
  }

  .agency-link small {
    font-size: 0.75rem;
    color: #adb5bd !important;
  }

  .contact-info .contact-item {
    margin-bottom: 0.25rem;

    i {
      font-size: 0.8rem;
      color: var(--primary-color) !important;
    }

    small {
      font-size: 0.8rem;
    }
  }
}
```

```

        color: #adb5bd !important;
    }
}
}
}

```

- header.component.ts : src/app/components/header/ header.component.ts

```

import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { TranslateService } from '@ngx-translate/core';
import { AuthService } from '../../services/auth.service';
import { TranslationService } from '../../services/translation.service';
import { LoginModalComponent } from '../../modals/login-modal/login-modal.component';
import { User } from '../../models/user.model';

@Component({
  selector: 'app-header',
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.scss']
})
export class HeaderComponent implements OnInit {
  currentUser: User | null = null;
  isMenuCollapsed = true;

  constructor(
    private router: Router,
    private modalService: NgbModal,
    private authService: AuthService,
    private translationService: TranslationService,
    private translate: TranslateService
  ) { }

  ngOnInit(): void {
    this.authService.currentUser$.subscribe(user => {
      this.currentUser = user;
    });
  }

  openLoginModal(): void {
    const modalRef = this.modalService.open(LoginModalComponent, {
      centered: true,
      backdrop: 'static'
    });
  }
}

```

```

logout(): void {
    this.authService.logout();
    this.router.navigate(['/home']);
}

navigateTo(route: string): void {
    this.router.navigate([route]);
    this.isMenuCollapsed = true;
}
}

```

- header.component.html : src/app/components/header/ header.component.html

```

<nav class="navbar navbar-expand-lg navbar-dark bg-primary fixed-top">
  <div class="container">
    <!-- Brand -->
    <a class="navbar-brand d-flex align-items-center" routerLink="/home">

      <span class="brand-text">{{ 'app.title' | translate }}</span>

    </a>

    <!-- Mobile menu button -->
    <button class="navbar-toggler" type="button" (click)="isMenuCollapsed =
!isMenuCollapsed"
      [attr.aria-expanded]="!isMenuCollapsed">
      <span class="navbar-toggler-icon"></span>
    </button>

    <!-- Navigation menu -->
    <div class="collapse navbar-collapse" [ngClass]="{'show':
!isMenuCollapsed}">
      <ul class="navbar-nav me-auto">
        <li class="nav-item">
          <a class="nav-link" routerLink="/home"
routerLinkActive="active" (click)="navigateTo('/home')">
            {{ 'navigation.home' | translate }}
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" routerLink="/procedures"
routerLinkActive="active"
            (click)="navigateTo('/procedures')">
            {{ 'navigation.procedures' | translate }}
          </a>
        </li>
        <li class="nav-item">
          <a class="nav-link" routerLink="/search-files"
routerLinkActive="active"

```

```

                (click)="navigateTo('/search-files')">
                {{ 'navigation.search_files' | translate }}
            </a>
        </li>
    </ul>

    <!-- Right side menu -->
    <ul class="navbar-nav ms-auto align-items-center">
        <!-- Language Selector -->
        <li class="nav-item">
            <app-language-selector></app-language-selector>
        </li>

        <!-- Authentication menu -->
        <li class="nav-item" *ngIf="!currentUser">
            <a class="nav-link" routerLink="/registration"
routerLinkActive="active"
                (click)="navigateTo('/registration')">
                {{ 'navigation.registration' | translate }}
            </a>
        </li>
        <li class="nav-item" *ngIf="!currentUser">
            <button class="btn btn-outline-light btn-sm ms-2"
(click)="openLoginModal()">
                {{ 'navigation.login' | translate }}
            </button>
        </li>

        <!-- User menu -->
        <li class="nav-item dropdown" *ngIf="currentUser" ngbDropdown>
            <a class="nav-link dropdown-toggle" id="userDropdown"
ngbDropdownToggle>
                <i class="fas fa-user me-1"></i>
                {{ currentUser.firstName || currentUser.email }}
            </a>
            <div class="dropdown-menu" ngbDropdownMenu>
                <a class="dropdown-item" href="#"
(click)="$event.preventDefault()">
                    <i class="fas fa-user me-2"></i>
                    {{ 'navigation.my_account' | translate }}
                </a>
                <div class="dropdown-divider"></div>
                <a class="dropdown-item" href="#" (click)="logout();
$event.preventDefault()">
                    <i class="fas fa-sign-out-alt me-2"></i>
                    {{ 'navigation.logout' | translate }}
                </a>
            </div>
        </li>
    </ul>

```

```

        </ul>
      </div>
    </div>
  </nav>

```

- header.component.scss : src/app/components/header/ header.component.scss

```

.navbar {
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  z-index: 1030;

  .navbar-brand {
    font-weight: 700;
    font-size: 1.5rem;

    .brand-text {
      background: linear-gradient(45deg, #fff, #e3f2fd);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      background-clip: text;
    }
  }
}

.nav-link {
  font-weight: 500;
  transition: all 0.3s ease;
  border-radius: 6px;
  margin: 0 2px;

  &:hover {
    background-color: rgba(255, 255, 255, 0.1);
    transform: translateY(-1px);
  }

  &.active {
    background-color: rgba(255, 255, 255, 0.2);
  }
}

.btn-outline-light {
  border-radius: 20px;
  font-weight: 500;
  transition: all 0.3s ease;

  &:hover {
    transform: translateY(-1px);
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
  }
}

```

```

    }
}

@media (max-width: 991px) {
    .navbar-nav {
        text-align: center;
        margin-top: 1rem;

        .nav-item {
            margin: 0.25rem 0;
        }
    }
}

```

- language-selector.component.ts :src/app/components/language-selector/language-selector.component.ts

```

import { Component, OnInit } from '@angular/core';
import { TranslationService } from '../../services/translation.service';

@Component({
  selector: 'app-language-selector',
  templateUrl: './language-selector.component.html',
  styleUrls: ['./language-selector.component.scss']
})
export class LanguageSelectorComponent implements OnInit {
  availableLanguages = this.translationService.getAvailableLanguages();
  currentLanguage = 'en';

  constructor(private translationService: TranslationService) { }

  ngOnInit(): void {
    this.translationService.currentLanguage$.subscribe(language => {
      this.currentLanguage = language;
    });
  }

  changeLanguage(languageCode: string): void {
    this.translationService.setLanguage(languageCode);
  }

  getCurrentLanguage() {
    return this.availableLanguages.find(lang => lang.code ===
this.currentLanguage);
  }
}

```



- language-selector.component.html :src/app/components/language-selector/language-selector.component.html

```
<div class="language-selector" ngbDropdown>
  <button class="btn btn-outline-light btn-sm dropdown-toggle"
id="languageDropdown" ngbDropdownToggle>
    <span class="flag-icon me-1">{{ getCurrentLanguage()?.flag }}</span>
    {{ getCurrentLanguage()?.name }}
  </button>
  <div class="dropdown-menu" ngbDropdownMenu>
    <a class="dropdown-item" *ngFor="let language of availableLanguages"
href="#"
      (click)="changeLanguage(language.code); $event.preventDefault()"
      [class.active]="language.code === currentLanguage">
      <span class="flag-icon me-2">{{ language.flag }}</span>
      {{ language.name }}
    </a>
  </div>
</div>
```

- language-selector.component.scss :src/app/components/language-selector/language-selector.component.scss

```
.language-selector {
  .dropdown-item {
    transition: all 0.3s ease;

    &:hover {
      background-color: var(--primary-color);
      color: white;
    }

    &.active {
      background-color: var(--primary-color);
      color: white;
    }
  }

  .flag-icon {
    font-size: 1.1em;
  }
}
```

- forgot-password-modal.component.ts :src/app/components/modals/forgot-password-modal/ forgot-password-modal.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { NgbActiveModal } from '@ng-bootstrap/ng-bootstrap';
```

```

import { TranslateService } from '@ngx-translate/core';
import { AuthService } from '../../services/auth.service';

@Component({
  selector: 'app-forgot-password-modal',
  templateUrl: './forgot-password-modal.component.html',
  styleUrls: ['./forgot-password-modal.component.scss']
})
export class ForgotPasswordModalComponent implements OnInit {
  forgotPasswordForm: FormGroup;
  isLoading = false;
  isCodeSent = false;
  successMessage = '';
  errorMessage = '';

  constructor(
    private fb: FormBuilder,
    private activeModal: NgbActiveModal,
    private authService: AuthService,
    private translate: TranslateService
  ) {
    this.forgotPasswordForm = this.fb.group({
      identifier: ['', [Validators.required]]
    });
  }

  ngOnInit(): void { }

  onSubmit(): void {
    if (this.forgotPasswordForm.valid) {
      this.isLoading = true;
      this.errorMessage = '';

      const identifier = this.forgotPasswordForm.get('identifier')?.value;

      this.authService.sendPasswordResetCode(identifier).subscribe({
        next: (response) => {
          this.isLoading = false;
          if (response.success) {
            this.isCodeSent = true;
            this.successMessage = response.message;
          } else {
            this.errorMessage = response.message;
          }
        },
        error: (error) => {
          this.isLoading = false;
          this.errorMessage = 'Failed to send reset code. Please try again.';
        }
      });
    }
  }
}

```

```

    });
  } else {
    this.forgotPasswordForm.get('identifier')?.markAsTouched();
  }
}

close(): void {
  this.activeModal.dismiss();
}
}

```

- forgot-password-modal.component.html :src/app/components/modals/forgot-password-modal/ forgot-password-modal.component.html:

```

<div class="modal-header">
  <h4 class="modal-title">
    <i class="fas fa-key me-2"></i>
    Reset Password
  </h4>
  <button type="button" class="btn-close" (click)="close()"></button>
</div>

<div class="modal-body">
  <div class="text-center mb-4" *ngIf="!isCodeSent">
    <div class="reset-icon mb-3">
      <i class="fas fa-lock fa-3x text-primary"></i>
    </div>
    <h5>Forgot your password?</h5>
    <p class="text-muted">Enter your phone number or email to receive a
verification code.</p>
  </div>

  <div class="text-center mb-4" *ngIf="isCodeSent">
    <div class="success-icon mb-3">
      <i class="fas fa-check-circle fa-3x text-success"></i>
    </div>
    <h5>Code Sent!</h5>
    <p class="text-muted">{{ successMessage }}</p>
  </div>

  <form [formGroup]="forgotPasswordForm" (ngSubmit)="onSubmit()"
*ngIf="!isCodeSent">
    <!-- Error Alert -->
    <div class="alert alert-danger" *ngIf="errorMessage" role="alert">
      <i class="fas fa-exclamation-triangle me-2"></i>
      {{ errorMessage }}
    </div>

```

```

    <!-- Identifier Field -->
    <div class="mb-3">
        <label class="form-label">Phone Number or Email</label>
        <div class="input-group">
            <span class="input-group-text">
                <i class="fas fa-user"></i>
            </span>
            <input type="text" class="form-control"
formControlName="identifier"
                placeholder="Enter your phone number or email"
                [class.is-
invalid]="forgotPasswordForm.get('identifier')?.invalid
                &&
forgotPasswordForm.get('identifier')?.touched">
            </div>
            <div class="invalid-feedback"
                *ngIf="forgotPasswordForm.get('identifier')?.invalid &&
forgotPasswordForm.get('identifier')?.touched">
                <small
*ngIf="forgotPasswordForm.get('identifier')?.errors?.['required']">
                    This field is required
                </small>
            </div>
        </div>

    <!-- Submit Button -->
        <button type="submit" class="btn btn-primary w-100"
[disabled]="isLoading">
            <span *ngIf="isLoading" class="spinner-border spinner-border-sm me-
2"></span>
            Send Reset Code
        </button>
    </form>

    <!-- Steps -->
    <div class="steps-container mt-4" *ngIf="!isCodeSent">
        <h6 class="mb-3">How it works:</h6>
        <div class="step-item">
            <div class="step-number">1</div>
            <div class="step-content">
                <strong>Submit your information</strong>
                <small class="text-muted d-block">Enter your registered phone
number or email</small>
            </div>
        </div>
        <div class="step-item">
            <div class="step-number">2</div>
            <div class="step-content">
                <strong>Receive verification code</strong>
            </div>
        </div>
    </div>

```

```

        <small class="text-muted d-block">A verification code will be
sent to verify your identity</small>
      </div>
    </div>
    <div class="step-item">
      <div class="step-number">3</div>
      <div class="step-content">
        <strong>Create new password</strong>
        <small class="text-muted d-block">Set up your new password and
confirm it</small>
      </div>
    </div>
  </div>
</div>

<div class="modal-footer" *ngIf="isCodeSent">
  <button type="button" class="btn btn-primary" (click)="close()">
    Close
  </button>
</div>

```

- forgot-password-modal.component.scss :src/app/components/modals/forgot-password-modal/ forgot-password-modal.component.scss:

```

.reset-icon,
.success-icon {
  opacity: 0.8;
}

.steps-container {
  .step-item {
    display: flex;
    align-items: flex-start;
    margin-bottom: 1rem;

    .step-number {
      width: 24px;
      height: 24px;
      background-color: var(--primary-color);
      color: white;
      border-radius: 50%;
      display: flex;
      align-items: center;
      justify-content: center;
      font-size: 0.8rem;
      font-weight: bold;
      margin-right: 0.75rem;
      flex-shrink: 0;
    }
  }
}

```

```

    }

    .step-content {
        flex: 1;
    }
}
}

```

- login-modal.component.ts :src/app/components/modals/login-modal/login-modal.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { NgbActiveModal } from '@ng-bootstrap/ng-bootstrap';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { Router } from '@angular/router';
import { TranslateService } from '@ngx-translate/core';
import { AuthService } from '../../../services/auth.service';
import { ForgotPasswordModalComponent } from '../forgot-password-modal/forgot-password-modal.component';

@Component({
  selector: 'app-login-modal',
  templateUrl: './login-modal.component.html',
  styleUrls: ['./login-modal.component.scss']
})
export class LoginComponent implements OnInit {
  loginForm: FormGroup;
  isLoading = false;
  errorMessage = '';
  showPassword = false;

  constructor(
    private fb: FormBuilder,
    private activeModal: NgbActiveModal,
    private modalService: NgbModal,
    private router: Router,
    private authService: AuthService,
    private translate: TranslateService
  ) {
    this.loginForm = this.fb.group({
      identifier: ['', [Validators.required]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  ngOnInit(): void { }

```

```

onSubmit(): void {
  if (this.loginForm.valid) {
    this.isLoading = true;
    this.errorMessage = '';

    const loginData = this.loginForm.value;

    this.authService.login(loginData).subscribe({
      next: (response) => {
        this.isLoading = false;
        if (response.success) {
          this.activeModal.close('login_success');
          // The guard will handle navigation to the intended route
        } else {
          this.errorMessage = response.message;
        }
      },
      error: (error) => {
        this.isLoading = false;
        this.errorMessage = 'Login failed. Please try again.';
      }
    });
  } else {
    this.markFormGroupTouched();
  }
}

private markFormGroupTouched(): void {
  Object.keys(this.loginForm.controls).forEach(key => {
    const control = this.loginForm.get(key);
    if (control) {
      control.markAsTouched();
    }
  });
}

openForgotPasswordModal(): void {
  this.activeModal.dismiss();
  const modalRef = this.modalService.open(ForgotPasswordModalComponent, {
    centered: true
  });
}

navigateToRegistration(): void {
  this.activeModal.dismiss();
  this.router.navigate(['/registration']);
}

close(): void {

```

```

    this.activeModal.dismiss();
  }

  togglePasswordVisibility(): void {
    this.showPassword = !this.showPassword;
  }
}

```

- login-modal.component.html :src/app/components/modals/login-modal/login-modal.component.html:

```

<div class="modal-header border-0">
  <div class="w-100 text-center">

    <h4 class="modal-title">{{ 'forms.login.title' | translate }}</h4>
  </div>
  <button type="button" class="btn-close" (click)="close()"></button>
</div>

<div class="modal-body px-4 pb-4">
  <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
    <!-- Error Alert -->
    <div class="alert alert-danger" *ngIf="errorMessage" role="alert">
      <i class="fas fa-exclamation-triangle me-2"></i>
      {{ errorMessage }}
    </div>

    <!-- Phone/Email Field -->
    <div class="mb-3">
      <label class="form-label">{{ 'forms.login.phone_email' | translate
    }}</label>
      <div class="input-group">
        <span class="input-group-text">
          <i class="fas fa-user"></i>
        </span>
        <input type="text" class="form-control"
formControlName="identifier"
          [placeholder]="'forms.login.phone_email' | translate"
          [class.is-invalid]="loginForm.get('identifier')?.invalid
&& loginForm.get('identifier')?.touched">
        </div>
        <div class="invalid-feedback"
          *ngIf="loginForm.get('identifier')?.invalid &&
loginForm.get('identifier')?.touched">
          <small
*ngIf="loginForm.get('identifier')?.errors?.['required']">
            This field is required
          </small>
        </div>
      </div>
    </div>
  </form>

```



```

        </div>
    </div>

    <!-- Password Field -->
    <div class="mb-3">
        <label class="form-label">{{ 'forms.login.password' | translate
    }}</label>
        <div class="input-group">
            <span class="input-group-text">
                <i class="fas fa-lock"></i>
            </span>
            <input [type]="showPassword ? 'text' : 'password'"
class="form-control" formControlName="password"
                [placeholder]="'forms.login.password' | translate"
                [class.is-invalid]="loginForm.get('password')?.invalid &&
loginForm.get('password')?.touched">
                <button type="button" class="btn btn-outline-secondary"
(click)="togglePasswordVisibility()">
                    <i class="fas" [class.fa-eye]="!showPassword" [class.fa-
eye-slash]="showPassword"></i>
                </button>
            </div>
            <div class="invalid-feedback"
                *ngIf="loginForm.get('password')?.invalid &&
loginForm.get('password')?.touched">
                <small
                    *ngIf="loginForm.get('password')?.errors?.['required']">
                        Password is required
                    </small>
                <small
                    *ngIf="loginForm.get('password')?.errors?.['minlength']">
                        Password must be at least 6 characters
                    </small>
                </div>
        </div>

    <!-- Forgot Password Link -->
    <div class="text-center mb-3">
        <a href="#" class="text-primary text-decoration-none"
            (click)="openForgotPasswordModal(); $event.preventDefault()">
            {{ 'forms.login.forgot_password' | translate }}
        </a>
    </div>

    <!-- Login Button -->
    <button type="submit" class="btn btn-primary w-100 mb-3"
[disabled]="isLoading">
        <span *ngIf="isLoading" class="spinner-border spinner-border-sm
me-2"></span>

```

```

        {{ 'forms.login.login_button' | translate }}
    </button>

    <!-- Terms Text -->
    <p class="text-center text-muted small mb-3">
        {{ 'forms.login.terms_text' | translate }}
    </p>

    <!-- Registration Link -->
    <div class="text-center">
        <span class="text-muted">{{ 'forms.login.no_account' | translate
    }}</span>
        <a href="#" class="text-primary text-decoration-none ms-1"
            (click)="navigateToRegistration(); $event.preventDefault()">
            {{ 'forms.login.create_account' | translate }}
        </a>
    </div>
</form>
</div>

```

- login-modal.component.scss :src/app/components/modals/login-modal/login-modal.component.scss:

```

.modal-header {
    .modal-title {
        font-weight: 600;
        color: var(--primary-color);
    }
}

.modal-body {
    .input-group-text {
        background-color: #f8f9fa;
        border-right: none;
        color: #6c757d;
    }

    .form-control {
        border-left: none;

        &:focus {
            box-shadow: 0 0 0 0.2rem rgba(26, 115, 232, 0.25);
        }
    }

    .btn-primary {
        border-radius: 25px;
        font-weight: 500;
    }
}

```

```
padding: 12px;
}
}
```

- service-modal.component.ts :src/app/components/modals/service-modal/service-modal.component.ts:

```
import { Component, Input, OnInit } from '@angular/core';
import { NgbActiveModal } from '@ng-bootstrap/ng-bootstrap';
import { Router } from '@angular/router';
import { TranslateService } from '@ngx-translate/core';
import { Service, ApplicantType } from '../../../models/service.model';
import { AuthService } from '../../../services/auth.service';
import { ApplicationService } from '../../../services/application.service';

@Component({
  selector: 'app-service-modal',
  templateUrl: './service-modal.component.html',
  styleUrls: ['./service-modal.component.scss']
})
export class ServiceModalComponent implements OnInit {
  @Input() service!: Service;

  selectedApplicantType: ApplicantType = ApplicantType.SELF;
  applicantTypes = ApplicantType;
  isLoading = false;

  constructor(
    private activeModal: NgbActiveModal,
    private router: Router,
    private authService: AuthService,
    private applicationService: ApplicationService,
    private translate: TranslateService
  ) { }

  ngOnInit(): void { }

  onApply(): void {
    const currentUser = this.authService.getCurrentUser();
    if (!currentUser) {
      this.activeModal.dismiss();
      return;
    }

    this.isLoading = true;

    this.applicationService.createApplication(
      this.service.id,
```

```

        this.service.nameKey,
        this.selectedApplicantType,
        currentUser.id!
    ).subscribe({
        next: (application) => {
            this.isLoading = false;
            this.activeModal.close('application_created');

            // Navigate to the appropriate service page
            if (this.service.id === 'passport') {
                this.router.navigate(['/passport-application'], {
                    queryParams: { applicationId: application.id }
                });
            } else if (this.service.id === 'birth-certificate-copy') {
                this.router.navigate(['/birth-certificate'], {
                    queryParams: { applicationId: application.id }
                });
            }
        },
        error: (error) => {
            this.isLoading = false;
            console.error('Error creating application:', error);
        }
    });
}

close(): void {
    this.activeModal.dismiss();
}

formatCurrency(amount: number, currency: string): string {
    return new Intl.NumberFormat('en-US', {
        style: 'currency',
        currency: currency === 'XAF' ? 'XAF' : currency,
        minimumFractionDigits: 0
    }).format(amount);
}
}

```

- service-modal.component.html :src/app/components/modals/service-modal/service-modal.component.html:

```

<div class="modal-header">
    <h4 class="modal-title">{{ service.nameKey | translate }}</h4>
    <button type="button" class="btn-close" (click)="close()"></button>
</div>

<div class="modal-body">

```

```

    <!-- Service Description -->
    <div class="service-description mb-4">
        <p class="text-muted">{{ service.detailedDescriptionKey | translate
    }}</p>
    </div>

    <!-- Service Details -->
    <div class="row mb-4">
        <div class="col-md-4">
            <div class="service-detail-item">
                <i class="fas fa-clock text-primary me-2"></i>
                <strong>{{ 'forms.service_application.processing_time' |
    translate }}:</strong>
                <br>
                <span class="text-muted">{{ service.processingTime }}</span>
            </div>
        </div>
        <div class="col-md-4">
            <div class="service-detail-item">
                <i class="fas fa-money-bill-wave text-success me-2"></i>
                <strong>{{ 'forms.service_application.cost' | translate
    }}:</strong>
                <br>
                <span class="text-muted">{{ formatCurrency(service.cost,
    service.currency) }}</span>
            </div>
        </div>
        <div class="col-md-4">
            <div class="service-detail-item">
                <i class="fas fa-building text-info me-2"></i>
                <strong>{{ 'forms.service_application.provided_by' | translate
    }}:</strong>
                <br>
                <span class="text-muted">{{ service.providedBy }}</span>
            </div>
        </div>
    </div>

    <!-- Required Documents -->
    <div class="mb-4" *ngIf="service.requirements &&
    service.requirements.length > 0">
        <h6 class="mb-3">{{ 'forms.service_application.required_documents' |
    translate }}</h6>
        <ul class="list-group">
            <li class="list-group-item d-flex align-items-center" *ngFor="let
    requirement of service.requirements">
                <i class="fas fa-file-alt text-primary me-3"></i>
                <div>
                    <strong>{{ requirement.nameKey | translate }}</strong>

```

```

        <br>
        <small class="text-muted">{{ requirement.descriptionKey |
translate }}</small>
    </div>
    <span class="badge bg-danger ms-auto"
*ngIf="requirement.isMandatory">Required</span>
    </li>
</ul>
</div>

<!-- Applicant Type Selection -->
<div class="mb-4" *ngIf="service.canApplyFor && service.canApplyFor.length
> 1">
    <h6 class="mb-3">{{ 'forms.service_application.apply_for' | translate
}}</h6>
    <div class="btn-group w-100" role="group">
        <input type="radio" class="btn-check" id="self"
[(ngModel)]="selectedApplicantType"
[value]="applicantTypes.SELF">
        <label class="btn btn-outline-primary" for="self">
            {{ 'forms.service_application.self' | translate }}
        </label>

        <input type="radio" class="btn-check" id="child"
[(ngModel)]="selectedApplicantType"
[value]="applicantTypes.CHILD">
*ngIf="service.canApplyFor.includes(applicantTypes.CHILD)">
        <label class="btn btn-outline-primary" for="child"
*ngIf="service.canApplyFor.includes(applicantTypes.CHILD)">
            {{ 'forms.service_application.child' | translate }}
        </label>

        <input type="radio" class="btn-check" id="other"
[(ngModel)]="selectedApplicantType"
[value]="applicantTypes.OTHER">
*ngIf="service.canApplyFor.includes(applicantTypes.OTHER)">
        <label class="btn btn-outline-primary" for="other"
*ngIf="service.canApplyFor.includes(applicantTypes.OTHER)">
            {{ 'forms.service_application.other' | translate }}
        </label>
    </div>
</div>

<!-- Warning Note -->
<div class="alert alert-warning" *ngIf="service.isImplemented">
    <i class="fas fa-info-circle me-2"></i>
    This service is fully implemented with real Cameroonian administrative
procedures.
</div>

```

```

    <div class="alert alert-info" *ngIf="!service.isImplemented">
      <i class="fas fa-construction me-2"></i>
      {{ 'messages.service_not_available' | translate }}
    </div>
  </div>

  <div class="modal-footer">
    <button type="button" class="btn btn-secondary" (click)="close()">
      {{ 'buttons.cancel' | translate }}
    </button>
    <button type="button" class="btn btn-primary"
[disabled]="!service.isImplemented || isLoading" (click)="onApply()">
      <span *ngIf="isLoading" class="spinner-border spinner-border-sm me-
2"></span>
      {{ 'forms.service_application.apply_button' | translate }}
    </button>
  </div>

```

- service-modal.component.scss :src/app/components/modals/service-modal/service-modal.component.scss:

```

.service-description {
  background-color: #f8f9fa;
  padding: 1rem;
  border-radius: 8px;
  border-left: 4px solid var(--primary-color);
}

.service-detail-item {
  text-align: center;
  padding: 1rem;
  border-radius: 8px;
  background-color: #f8f9fa;
  height: 100%;
}

.list-group-item {
  border: 1px solid #dee2e6;
  border-radius: 8px !important;
  margin-bottom: 0.5rem;
}

.btn-group {
  .btn-check:checked+.btn-outline-primary {
    background-color: var(--primary-color);
    border-color: var(--primary-color);
  }
}

```

```
}
```

- auth-guard.ts :src/app/guards/auth-guard.ts:

```
import { Injectable } from '@angular/core';
import { CanActivate, Router, ActivatedRouteSnapshot, RouterStateSnapshot }
from '@angular/router';
import { Observable } from 'rxjs';
import { AuthService } from '../services/auth.service';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { LoginModalComponent } from '../components/modals/login-modal/login-
modal.component';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {

  constructor(
    private authService: AuthService,
    private router: Router,
    private modalService: NgbModal
  ) { }

  canActivate(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): Observable<boolean> | Promise<boolean> | boolean {

    if (this.authService.isAuthenticated()) {
      return true;
    }

    // If not authenticated, open login modal
    const modalRef = this.modalService.open(LoginModalComponent, {
      centered: true,
      backdrop: 'static',
      keyboard: false
    });

    modalRef.result.then((result) => {
      if (result === 'login_success') {
        // Navigation will be handled by the modal component
        return true;
      } else {
        this.router.navigate(['/home']);
        return false;
      }
    });
  }
}
```



```

    }).catch(() => {
      this.router.navigate(['/home']);
      return false;
    });

    return false;
  }
}

```

- http.interceptor.ts :src/app/interceptors/http.interceptor.ts:

```

import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent,
HttpErrorResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, finalize } from 'rxjs/operators';
import { AuthService } from '../services/auth.service';
import { Router } from '@angular/router';

@Injectable()
export class HttpErrorInterceptor implements HttpInterceptor {
  private activeRequests = 0;
  private loadingElement?: HTMLElement;

  constructor(
    private authService: AuthService,
    private router: Router
  ) { }

  intercept(req: HttpRequest<any>, next: HttpHandler):
Observable<HttpEvent<any>> {
    this.showLoading();

    // Add auth token if available
    const currentUser = this.authService.getCurrentUser();
    if (currentUser && currentUser.isAuthenticated) {
      req = req.clone({
        headers: {
          'Authorization': `Bearer ${currentUser.id}`,
          'Content-Type': 'application/json'
        }
      });
    }

    return next.handle(req).pipe(
      catchError((error: HttpErrorResponse) => {
        let errorMessage = 'An error occurred';

```

```

        if (error.error instanceof ErrorEvent) {
            // Client-side error
            errorMessage = `Error: ${error.error.message}`;
        } else {
            // Server-side error
            switch (error.status) {
                case 401:
                    errorMessage = 'Unauthorized access';
                    this.authService.logout();
                    this.router.navigate(['/home']);
                    break;
                case 403:
                    errorMessage = 'Access forbidden';
                    break;
                case 404:
                    errorMessage = 'Resource not found';
                    break;
                case 500:
                    errorMessage = 'Internal server error';
                    break;
                default:
                    errorMessage = `Error Code:
${error.status}\nMessage: ${error.message}`;
            }
        }

        console.error('HTTP Error:', errorMessage);
        return throwError(() => new Error(errorMessage));
    }},
    finalize(() => {
        this.hideLoading();
    })
);
}

private showLoading(): void {
    this.activeRequests++;
    if (this.activeRequests === 1) {
        this.loadingElement = document.createElement('div');
        this.loadingElement.className = 'loading-overlay';
        this.loadingElement.innerHTML = `
<div class="loading-spinner">
  <div class="spinner-border text-primary" role="status">
    <span class="visually-hidden">Loading...</span>
  </div>
</div>
`;
        document.body.appendChild(this.loadingElement);
    }
}

```

```

    }

    private hideLoading(): void {
        this.activeRequests--;
        if (this.activeRequests === 0 && this.loadingElement) {
            document.body.removeChild(this.loadingElement);
            this.loadingElement = undefined;
        }
    }
}

```

- application.model.ts :src/app/models/application.model.ts:

```

import { ApplicantType } from './service.model';

export interface Application {
    id: string;
    userId: string;
    serviceId: string;
    serviceName: string;
    applicantType: ApplicantType;
    applicantDetails?: ApplicantDetails;
    status: ApplicationStatus;
    submissionDate: Date;
    expectedCompletionDate?: Date;
    completionDate?: Date;
    documents: ApplicationDocument[];
    timeline: ApplicationTimeline[];
    payments: Payment[];
    trackingNumber: string;
    notes?: string;
}

export interface ApplicantDetails {
    firstName: string;
    lastName: string;
    relationship?: string; // for 'other' type
    dateOfBirth?: Date;
    placeOfBirth?: string;
}

export interface ApplicationDocument {
    id: string;
    name: string;
    type: DocumentType;
    fileName: string;
    fileUrl: string;
    uploadDate: Date;
}

```

```
    isVerified: boolean;
    verificationDate?: Date;
    verificationNotes?: string;
}

export interface ApplicationTimeline {
    id: string;
    step: string;
    status: TimelineStatus;
    date: Date;
    description: string;
    isCompleted: boolean;
    estimatedDuration?: string;
}

export interface Payment {
    id: string;
    amount: number;
    currency: string;
    description: string;
    method: PaymentMethod;
    status: PaymentStatus;
    date: Date;
    referenceNumber: string;
    isSimulated: boolean;
}

export enum ApplicationStatus {
    DRAFT = 'draft',
    SUBMITTED = 'submitted',
    UNDER_REVIEW = 'under_review',
    PAYMENT_PENDING = 'payment_pending',
    PROCESSING = 'processing',
    READY_FOR_PICKUP = 'ready_for_pickup',
    COMPLETED = 'completed',
    REJECTED = 'rejected',
    CANCELLED = 'cancelled'
}

export enum TimelineStatus {
    PENDING = 'pending',
    IN_PROGRESS = 'in_progress',
    COMPLETED = 'completed',
    SKIPPED = 'skipped'
}

export enum PaymentMethod {
    MTN_MONEY = 'mtn_money',
    ORANGE_MONEY = 'orange_money',
```

```

    CREDIT_CARD = 'credit_card',
    BANK_TRANSFER = 'bank_transfer',
    CASH = 'cash'
}

export enum PaymentStatus {
    PENDING = 'pending',
    PROCESSING = 'processing',
    COMPLETED = 'completed',
    FAILED = 'failed',
    REFUNDED = 'refunded'
}

export enum DocumentType {
    IDENTITY = 'identity',
    BIRTH_CERTIFICATE = 'birth_certificate',
    MARRIAGE_CERTIFICATE = 'marriage_certificate',
    PHOTO = 'photo',
    RESIDENCE_PROOF = 'residence_proof',
    FISCAL_STAMP = 'fiscal_stamp',
    FORM = 'form',
    AUTHORIZATION = 'authorization'
}

```

- service.model.ts :src/app/models/service.model.ts:

```

export interface ServiceCategory {
    id: string;
    nameKey: string;
    descriptionKey: string;
    icon: string;
    color: string;
    services: Service[];
}

export interface Service {
    id: string;
    categoryId: string;
    nameKey: string;
    descriptionKey: string;
    detailedDescriptionKey: string;
    requirements: ServiceRequirement[];
    processingTime: string;
    cost: number;
    currency: string;
    providedBy: string;
    legalTexts?: LegalText[];
    isImplemented: boolean;
}

```

```

    canApplyFor: ApplicantType[];
  }

export interface ServiceRequirement {
  id: string;
  nameKey: string;
  descriptionKey: string;
  isMandatory: boolean;
  documentType: DocumentType;
}

export interface LegalText {
  id: string;
  title: string;
  content: string;
  pdfUrl?: string;
}

export enum DocumentType {
  IDENTITY = 'identity',
  BIRTH_CERTIFICATE = 'birth_certificate',
  MARRIAGE_CERTIFICATE = 'marriage_certificate',
  PHOTO = 'photo',
  RESIDENCE_PROOF = 'residence_proof',
  FISCAL_STAMP = 'fiscal_stamp',
  FORM = 'form',
  AUTHORIZATION = 'authorization'
}

export enum ApplicantType {
  SELF = 'self',
  CHILD = 'child',
  OTHER = 'other'
}

```

- user.model.ts :src/app/models/user.model.ts:

```

export interface User {
  id?: string;
  cniNumber?: string;
  consularCardNumber?: string;
  phoneNumber: string;
  email: string;
  password?: string;
  firstName?: string;
  lastName?: string;
  dateOfBirth?: Date;
  placeOfBirth?: string;
}

```

```

    address?: string;
    isDiaspora: boolean;
    isAuthenticated?: boolean;
    registrationDate?: Date;
    lastLoginDate?: Date;
  }

  export interface RegistrationData {
    accountType: 'local' | 'diaspora';
    cniNumber?: string;
    consularCardNumber?: string;
    phoneNumber: string;
    email: string;
    password: string;
    confirmPassword: string;
    acceptedTerms: boolean;
  }

  export interface LoginData {
    identifier: string; // phone or email
    password: string;
  }

```

- birth-certificate.component.ts :src/app/pages/birth-certificate/birth-certificate.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { TranslateService } from '@ngx-translate/core';
import { ApplicationService } from '../../services/application.service';
import { AuthService } from '../../services/auth.service';
import { Application, ApplicationStatus, PaymentMethod, PaymentStatus } from '../../models/application.model';
import { ApplicantType } from '../../models/service.model';

@Component({
  selector: 'app-birth-certificate',
  templateUrl: './birth-certificate.component.html',
  styleUrls: ['./birth-certificate.component.scss']
})
export class BirthCertificateComponent implements OnInit {
  application: Application | null = null;
  certificateForm: FormGroup;
  currentStep = 1;
  totalSteps = 4;
  isLoading = false;
  isSubmitting = false;

```

```

errorMessage = '';
successMessage = '';

// Payment properties
selectedPaymentMethod: PaymentMethod = PaymentMethod.MTN_MONEY;
paymentMethods = [
  { value: PaymentMethod.MTN_MONEY, label: 'MTN Mobile Money', icon: 'fas fa-mobile-alt' },
  { value: PaymentMethod.ORANGE_MONEY, label: 'Orange Money', icon: 'fas fa-mobile-alt' },
  { value: PaymentMethod.CASH, label: 'Pay at Registry', icon: 'fas fa-money-bill-wave' }
];

constructor(
  private fb: FormBuilder,
  private route: ActivatedRoute,
  private router: Router,
  private applicationService: ApplicationService,
  private authService: AuthService,
  private translate: TranslateService
) {
  this.certificateForm = this.fb.group({
    // Request Information
    requestType: ['', Validators.required],
    requestFor: ['', Validators.required],

    // Person's Information (whose certificate is being requested)
    personFirstName: ['', Validators.required],
    personLastName: ['', Validators.required],
    personMiddleName: [''],
    personDateOfBirth: ['', Validators.required],
    personPlaceOfBirth: ['', Validators.required],
    personGender: ['', Validators.required],

    // Parents Information
    fatherFirstName: ['', Validators.required],
    fatherLastName: ['', Validators.required],
    motherFirstName: ['', Validators.required],
    motherLastName: ['', Validators.required],
    motherMaidenName: [''],

    // Requester Information (if different from person)
    requesterFirstName: [''],
    requesterLastName: [''],
    requesterPhoneNumber: [''],
    requesterEmail: [''],
    requesterAddress: [''],
    requesterIdNumber: [''],
  });
}

```



```

        relationshipToPerson: [''],
        reasonForRequest: [''],

        // Collection Information
        collectionMethod: ['pickup', Validators.required],
        collectionLocation: ['', Validators.required],
        urgentRequest: [false]
    });
}

ngOnInit(): void {
    this.route.queryParams.subscribe(params => {
        if (params['applicationId']) {
            this.loadApplication(params['applicationId']);
        } else {
            this.createNewApplication();
        }
    });

    this.prefillUserData();
    this.onRequestTypeChange();
}

private loadApplication(applicationId: string): void {
    this.applicationService.getApplicationById(applicationId).subscribe({
        next: (application) => {
            if (application) {
                this.application = application;
                this.determineCurrentStep();
            }
        },
        error: (error) => {
            console.error('Error loading application:', error);
            this.errorMessage = 'Failed to load application details.';
        }
    });
}

private createNewApplication(): void {
    const currentUser = this.authService.getCurrentUser();
    if (currentUser) {
        this.applicationService.createApplication(
            'birth-certificate-copy',
            'Birth Certificate Copy',
            ApplicantType.SELF,
            currentUser.id!
        ).subscribe({
            next: (application) => {
                this.application = application;
            }
        });
    }
}

```

```

    },
    error: (error) => {
        console.error('Error creating application:', error);
        this.errorMessage = 'Failed to create application.';
    }
});
}
}

private prefillUserData(): void {
    const currentUser = this.authService.getCurrentUser();
    if (currentUser) {
        this.certificateForm.patchValue({
            requesterPhoneNumber: currentUser.phoneNumber,
            requesterEmail: currentUser.email,
            requesterFirstName: currentUser.firstName || '',
            requesterLastName: currentUser.lastName || ''
        });
    }
}

private determineCurrentStep(): void {
    if (!this.application) return;

    switch (this.application.status) {
        case ApplicationStatus.DRAFT:
            this.currentStep = 1;
            break;
        case ApplicationStatus.SUBMITTED:
            this.currentStep = 3;
            break;
        case ApplicationStatus.PAYMENT_PENDING:
            this.currentStep = 3;
            break;
        case ApplicationStatus.PROCESSING:
            this.currentStep = 4;
            break;
        default:
            this.currentStep = 1;
    }
}

onRequestTypeChange(): void {
    const requestType = this.certificateForm.get('requestType')?.value;
    const currentUser = this.authService.getCurrentUser();

    if (requestType === 'self' && currentUser) {
        // Pre-fill person's information with user's data
        this.certificateForm.patchValue({

```

```

        personFirstName: currentUser.firstName || '',
        personLastName: currentUser.lastName || '',
        requesterFirstName: currentUser.firstName || '',
        requesterLastName: currentUser.lastName || '',
        requesterPhoneNumber: currentUser.phoneNumber,
        requesterEmail: currentUser.email,
        relationshipToPerson: 'Self',
        reasonForRequest: 'Personal use'
    });

    // Clear validators for requester info when requesting for self
    this.clearRequesterValidators();
} else {
    // Set validators for requester info when requesting for someone else
    this.setRequesterValidators();
}
}

private clearRequesterValidators(): void {
    const requesterFields = ['requesterFirstName', 'requesterLastName',
        'requesterPhoneNumber',
        'requesterEmail', 'requesterAddress', 'requesterIdNumber',
        'relationshipToPerson', 'reasonForRequest'];

    requesterFields.forEach(field => {
        this.certificateForm.get(field)?.clearValidators();
        this.certificateForm.get(field)?.updateValueAndValidity();
    });
}

private setRequesterValidators(): void {
    const requesterFields = ['requesterFirstName', 'requesterLastName',
        'requesterPhoneNumber',
        'requesterEmail', 'requesterAddress', 'requesterIdNumber',
        'relationshipToPerson', 'reasonForRequest'];

    requesterFields.forEach(field => {
        this.certificateForm.get(field)?.setValidators([Validators.required]);
        this.certificateForm.get(field)?.updateValueAndValidity();
    });
}

nextStep(): void {
    if (this.currentStep < this.totalSteps) {
        if (this.validateCurrentStep()) {
            this.currentStep++;
        }
    }
}
}

```

```

previousStep(): void {
    if (this.currentStep > 1) {
        this.currentStep--;
    }
}

private validateCurrentStep(): boolean {
    switch (this.currentStep) {
        case 1: // Person Information
            const personFields = ['personFirstName', 'personLastName',
'personDateOfBirth',
            'personPlaceOfBirth', 'personGender', 'fatherFirstName',
            'fatherLastName', 'motherFirstName', 'motherLastName'];
            return this.validateFields(personFields);
        case 2: // Requester Information
            if (this.certificateForm.get('requestType')?.value === 'other') {
                const requesterFields = ['requesterFirstName', 'requesterLastName',
'requesterPhoneNumber',
                'requesterEmail', 'requesterAddress', 'requesterIdNumber',
                'relationshipToPerson', 'reasonForRequest'];
                return this.validateFields(requesterFields);
            }
            return true;
        default:
            return true;
    }
}

private validateFields(fieldNames: string[]): boolean {
    let isValid = true;
    fieldNames.forEach(fieldName => {
        const control = this.certificateForm.get(fieldName);
        if (control && control.invalid) {
            control.markAsTouched();
            isValid = false;
        }
    });
    return isValid;
}

onSubmitApplication(): void {
    if (this.certificateForm.valid && this.application) {
        this.isSubmitting = true;
        this.errorMessage = '';

        // Simulate form submission
        setTimeout(() => {
            this.application!.status = ApplicationStatus.PAYMENT_PENDING;

```

```

        this.isSubmitting = false;
        this.successMessage = 'Application submitted successfully!';
        this.currentStep = 3; // Move to payment step
    }, 1500);
} else {
    this.markFormGroupTouched();
}
}

private markFormGroupTouched(): void {
    Object.keys(this.certificateForm.controls).forEach(key => {
        const control = this.certificateForm.get(key);
        if (control) {
            control.markAsTouched();
        }
    });
}

onPayment(): void {
    if (this.application) {
        this.isLoading = true;
        this.errorMessage = '';

        const baseAmount = 200; // Base fee
        const urgentFee = this.certificateForm.get('urgentRequest')?.value ? 500
: 0;
        const totalAmount = baseAmount + urgentFee;

        this.applicationService.simulatePayment(
            this.application.id,
            totalAmount,
            this.selectedPaymentMethod
        ).subscribe({
            next: (payment) => {
                this.isLoading = false;
                if (payment.status === PaymentStatus.COMPLETED) {
                    this.successMessage = 'Payment completed successfully!';
                    this.currentStep = 4; // Move to confirmation step
                }
            },
            error: (error) => {
                this.isLoading = false;
                this.errorMessage = 'Payment failed. Please try again.';
            }
        });
    }
}

goToTracking(): void {

```

```

    if (this.application) {
      this.router.navigate(['/search-files'], {
        queryParams: { trackingNumber: this.application.trackingNumber }
      });
    }
  }

  calculateTotalAmount(): number {
    const baseAmount = 200;
    const urgentFee = this.certificateForm.get('urgentRequest')?.value ? 500 :
0;
    return baseAmount + urgentFee;
  }

  getExpectedDeliveryTime(): string {
    const isUrgent = this.certificateForm.get('urgentRequest')?.value;
    return isUrgent ? 'Same day (within 4 hours)' : '1-2 business days';
  }
}

```

- birth-certificate.component.html :src/app/pages/birth-certificate/birth-certificate.component.html:

```

<div class="birth-certificate-page">
  <div class="container py-4">

    <!-- Page Header -->
    <div class="page-header mb-4">
      <div class="row align-items-center">
        <div class="col-md-8">
          <h1 class="page-title">
            <i class="fas fa-certificate me-3"></i>
            Birth Certificate Copy
          </h1>
          <p class="text-muted">Request a certified copy of birth
certificate</p>
        </div>
        <div class="col-md-4 text-md-end">
          <div class="tracking-info" *ngIf="application">
            <strong>Tracking #:</strong> {{
application.trackingNumber }}
          </div>
        </div>
      </div>
    </div>

    <!-- Progress Indicator -->
    <div class="progress-container mb-5">

```

```

        <div class="progress-steps">
            <div class="step" [class.active]="currentStep >= 1"
[class.completed]="currentStep > 1">
                <div class="step-circle">1</div>
                <span class="step-label">Person Info</span>
            </div>
            <div class="step-line" [class.completed]="currentStep >
1"></div>
            <div class="step" [class.active]="currentStep >= 2"
[class.completed]="currentStep > 2">
                <div class="step-circle">2</div>
                <span class="step-label">Requester Info</span>
            </div>
            <div class="step-line" [class.completed]="currentStep >
2"></div>
            <div class="step" [class.active]="currentStep >= 3"
[class.completed]="currentStep > 3">
                <div class="step-circle">3</div>
                <span class="step-label">Payment</span>
            </div>
            <div class="step-line" [class.completed]="currentStep >
3"></div>
            <div class="step" [class.active]="currentStep >= 4">
                <div class="step-circle">4</div>
                <span class="step-label">Confirmation</span>
            </div>
        </div>
    </div>

    <!-- Application Form -->
    <div class="application-form">

        <!-- Error/Success Messages -->
        <div class="alert alert-danger" *ngIf="errorMessage" role="alert">
            <i class="fas fa-exclamation-triangle me-2"></i>
            {{ errorMessage }}
        </div>

        <div class="alert alert-success" *ngIf="successMessage"
role="alert">
            <i class="fas fa-check-circle me-2"></i>
            {{ successMessage }}
        </div>

        <form [formGroup]="certificateForm">

            <!-- Step 1: Person Information -->
            <div class="form-step" *ngIf="currentStep === 1">
                <div class="step-header mb-4">

```

```

        <h3>Birth Certificate Information</h3>
        <p class="text-muted">Provide details of the person
whose birth certificate is being requested
        </p>
    </div>

    <!-- Request Type -->
    <div class="mb-4">
        <label class="form-label">Who is this request for?
*</label>

        <div class="request-type-selection">
            <div class="form-check form-check-card">
                <input class="form-check-input" type="radio"
value="self" formControlName="requestType"
                id="self"
(change)="onRequestTypeChange()">
                <label class="form-check-label" for="self">
                    <div class="card-content">
                        <i class="fas fa-user fa-2x mb-2"></i>
                        <h6>For Myself</h6>
                        <small class="text-muted">My own birth
certificate</small>
                    </div>
                </label>
            </div>
            <div class="form-check form-check-card">
                <input class="form-check-input" type="radio"
value="other" formControlName="requestType"
                id="other"
(change)="onRequestTypeChange()">
                <label class="form-check-label" for="other">
                    <div class="card-content">
                        <i class="fas fa-users fa-2x mb-
2"></i>
                        <h6>For Someone Else</h6>
                        <small class="text-muted">Child,
relative, or other person</small>
                    </div>
                </label>
            </div>
        </div>
    </div>

    <!-- Person's Information -->
    <h5 class="mb-3">Person's Information</h5>
    <div class="row">
        <div class="col-md-4 mb-3">
            <label class="form-label">First Name *</label>

```



```

        <input type="text" class="form-control"
formControlName="personFirstName"
        [class.is-
invalid]="certificateForm.get('personFirstName')?.invalid &&
certificateForm.get('personFirstName')?.touched">
        <div class="invalid-feedback">First name is
required</div>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Middle Name</label>
        <input type="text" class="form-control"
formControlName="personMiddleName">
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Last Name *</label>
        <input type="text" class="form-control"
formControlName="personLastName"
        [class.is-
invalid]="certificateForm.get('personLastName')?.invalid &&
certificateForm.get('personLastName')?.touched">
        <div class="invalid-feedback">Last name is
required</div>
    </div>
</div>

<div class="row">
    <div class="col-md-4 mb-3">
        <label class="form-label">Date of Birth *</label>
        <input type="date" class="form-control"
formControlName="personDateOfBirth"
        [class.is-
invalid]="certificateForm.get('personDateOfBirth')?.invalid &&
certificateForm.get('personDateOfBirth')?.touched">
        <div class="invalid-feedback">Date of birth is
required</div>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Place of Birth *</label>
        <input type="text" class="form-control"
formControlName="personPlaceOfBirth"
        [class.is-
invalid]="certificateForm.get('personPlaceOfBirth')?.invalid &&
certificateForm.get('personPlaceOfBirth')?.touched">
        <div class="invalid-feedback">Place of birth is
required</div>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Gender *</label>

```

```

                <select class="form-select"
formControlName="personGender"
                [class.is-
invalid]="certificateForm.get('personGender')?.invalid &&
certificateForm.get('personGender')?.touched">
                <option value="">Select Gender</option>
                <option value="Male">Male</option>
                <option value="Female">Female</option>
                </select>
                <div class="invalid-feedback">Gender is
required</div>
            </div>
        </div>

        <!-- Parents Information -->
        <h5 class="mb-3 mt-4">Parents Information</h5>
        <div class="row">
            <div class="col-md-6 mb-3">
                <label class="form-label">Father's First Name
*</label>
                <input type="text" class="form-control"
formControlName="fatherFirstName"
                [class.is-
invalid]="certificateForm.get('fatherFirstName')?.invalid &&
certificateForm.get('fatherFirstName')?.touched">
                <div class="invalid-feedback">Father's first name
is required</div>
            </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Father's Last Name
*</label>
                <input type="text" class="form-control"
formControlName="fatherLastName"
                [class.is-
invalid]="certificateForm.get('fatherLastName')?.invalid &&
certificateForm.get('fatherLastName')?.touched">
                <div class="invalid-feedback">Father's last name
is required</div>
            </div>
        </div>

        <div class="row">
            <div class="col-md-4 mb-3">
                <label class="form-label">Mother's First Name
*</label>
                <input type="text" class="form-control"
formControlName="motherFirstName"

```

```

            [class.is-
invalid]="certificateForm.get('motherFirstName')?.invalid &&
certificateForm.get('motherFirstName')?.touched">
            <div class="invalid-feedback">Mother's first name
is required</div>
        </div>
        <div class="col-md-4 mb-3">
            <label class="form-label">Mother's Last Name
*</label>
            <input type="text" class="form-control"
formControlName="motherLastName"
            [class.is-
invalid]="certificateForm.get('motherLastName')?.invalid &&
certificateForm.get('motherLastName')?.touched">
            <div class="invalid-feedback">Mother's last name
is required</div>
        </div>
        <div class="col-md-4 mb-3">
            <label class="form-label">Mother's Maiden
Name</label>
            <input type="text" class="form-control"
formControlName="motherMaidenName">
            <small class="form-text text-muted">If different
from last name</small>
        </div>
    </div>
</div>
</div>

<!-- Step 2: Requester Information -->
<div class="form-step" *ngIf="currentStep === 2">
    <div class="step-header mb-4">
        <h3>Requester Information</h3>
        <p class="text-muted"
*ngIf="certificateForm.get('requestType')?.value === 'self'">
            Confirm your contact information
        </p>
        <p class="text-muted"
*ngIf="certificateForm.get('requestType')?.value === 'other'">
            Provide your information as the person requesting
this certificate
        </p>
    </div>

    <div class="row">
        <div class="col-md-6 mb-3">
            <label class="form-label">Your First Name
*</label>
            <input type="text" class="form-control"
formControlName="requesterFirstName"

```

```

            [class.is-
invalid]="certificateForm.get('requesterFirstName')?.invalid &&
certificateForm.get('requesterFirstName')?.touched">
                <div class="invalid-feedback">First name is
required</div>
            </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Your Last Name *</label>
                <input type="text" class="form-control"
formControlName="requesterLastName"
            [class.is-
invalid]="certificateForm.get('requesterLastName')?.invalid &&
certificateForm.get('requesterLastName')?.touched">
                <div class="invalid-feedback">Last name is
required</div>
            </div>
        </div>
        <div class="row">
            <div class="col-md-6 mb-3">
                <label class="form-label">Phone Number *</label>
                <input type="tel" class="form-control"
formControlName="requesterPhoneNumber"
            [class.is-
invalid]="certificateForm.get('requesterPhoneNumber')?.invalid &&
certificateForm.get('requesterPhoneNumber')?.touched">
                <div class="invalid-feedback">Phone number is
required</div>
            </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Email Address *</label>
                <input type="email" class="form-control"
formControlName="requesterEmail"
            [class.is-
invalid]="certificateForm.get('requesterEmail')?.invalid &&
certificateForm.get('requesterEmail')?.touched">
                <div class="invalid-feedback">Valid email is
required</div>
            </div>
        </div>

        <!-- Additional fields for requests on behalf of others --
>
        <div *ngIf="certificateForm.get('requestType')?.value ===
'other'">
            <div class="row">
                <div class="col-md-6 mb-3">
                    <label class="form-label">Your ID Number
*</label>

```

```

<input type="text" class="form-control"
formControlName="requesterIdNumber"
[class.is-
invalid]="certificateForm.get('requesterIdNumber')?.invalid &&
certificateForm.get('requesterIdNumber')?.touched">
<div class="invalid-feedback">ID number is
required</div>
</div>
<div class="col-md-6 mb-3">
<label class="form-label">Relationship to
Person *</label>
<select class="form-select"
formControlName="relationshipToPerson"
[class.is-
invalid]="certificateForm.get('relationshipToPerson')?.invalid &&
certificateForm.get('relationshipToPerson')?.touched">
<option value="">Select
Relationship</option>
<option value="Parent">Parent</option>
<option value="Child">Child</option>
<option value="Spouse">Spouse</option>
<option value="Sibling">Sibling</option>
<option value="Legal Guardian">Legal
Guardian</option>
<option value="Legal Representative">Legal
Representative</option>
<option value="Other">Other</option>
</select>
<div class="invalid-feedback">Relationship is
required</div>
</div>
</div>
<div class="mb-3">
<label class="form-label">Your Address *</label>
<textarea class="form-control" rows="3"
formControlName="requesterAddress"
[class.is-
invalid]="certificateForm.get('requesterAddress')?.invalid &&
certificateForm.get('requesterAddress')?.touched"></textarea>
<div class="invalid-feedback">Address is
required</div>
</div>
<div class="mb-3">
<label class="form-label">Reason for Request
*</label>
<textarea class="form-control" rows="2"
formControlName="reasonForRequest"

```

```

placeholder="e.g., School enrollment, passport
application, legal proceedings"
[class.is-
invalid]="certificateForm.get('reasonForRequest')?.invalid &&
certificateForm.get('reasonForRequest')?.touched"></textarea>
<div class="invalid-feedback">Reason for request
is required</div>
</div>
</div>

<!-- Collection Method -->
<h5 class="mb-3 mt-4">Collection Information</h5>
<div class="row">
<div class="col-md-6 mb-3">
<label class="form-label">Collection Method
*</label>
<select class="form-select"
formControlName="collectionMethod">
<option value="pickup">Pickup at Registry
Office</option>
<option value="mail">Mail Delivery</option>
</select>
</div>
<div class="col-md-6 mb-3">
<label class="form-label">Registry Office
*</label>
<select class="form-select"
formControlName="collectionLocation"
[class.is-
invalid]="certificateForm.get('collectionLocation')?.invalid &&
certificateForm.get('collectionLocation')?.touched">
<option value="">Select Registry
Office</option>
<option value="Yaoundé Central
Registry">Yaoundé Central Registry</option>
<option value="Douala Registry Office">Douala
Registry Office</option>
<option value="Bamenda Registry
Office">Bamenda Registry Office</option>
<option value="Bafoussam Registry
Office">Bafoussam Registry Office</option>
<option value="Garoua Registry Office">Garoua
Registry Office</option>
</select>
<div class="invalid-feedback">Registry office
selection is required</div>
</div>
</div>

```

```

        <!-- Urgent Request Option -->
        <div class="mb-3">
            <div class="form-check form-check-premium">
                <input class="form-check-input" type="checkbox"
formControlName="urgentRequest"
                id="urgentRequest">
                <label class="form-check-label"
for="urgentRequest">
                    <div class="premium-option">
                        <div class="premium-header">
                            <i class="fas fa-bolt text-warning me-
2"></i>
                            <strong>Urgent Processing</strong>
                            <span class="badge bg-warning text-
dark ms-2">+500 XAF</span>
                        </div>
                        <small class="text-muted">Same-day
processing (within 4 hours)</small>
                    </div>
                </label>
            </div>
        </div>

        <!-- Step 3: Payment -->
        <div class="form-step" *ngIf="currentStep === 3">
            <div class="step-header mb-4">
                <h3>Payment</h3>
                <p class="text-muted">Complete your payment to process
your certificate request</p>
            </div>

            <div class="row">
                <div class="col-md-8">
                    <div class="payment-summary mb-4">
                        <h5>Payment Summary</h5>
                        <div class="fee-item">
                            <div class="d-flex justify-content-
between">
                                <span>Birth Certificate Copy</span>
                                <span>200 XAF</span>
                            </div>
                        </div>
                        <div class="fee-item"
*ngIf="certificateForm.get('urgentRequest')?.value">
                            <div class="d-flex justify-content-
between">
                                <span>Urgent Processing Fee</span>
                                <span>500 XAF</span>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>
    </div>
    <hr>
    <div class="total-amount">
        <div class="d-flex justify-content-
between">
            <strong>Total Amount</strong>
            <strong class="text-primary">{{
calculateTotalAmount() }} XAF</strong>
        </div>
    </div>
</div>

<div class="payment-methods">
    <h5 class="mb-3">Select Payment Method</h5>
    <div class="payment-options">
        <div class="payment-option" *ngFor="let
method of paymentMethods">
            <input type="radio" [id]='payment-' +
method.value"
                [(ngModel)]="selectedPaymentMethod
" [value]="method.value"
                [ngModelOptions]="{standalone:
true}">
            <label [for]='payment-' +
method.value" class="payment-label">
                <i [class]="method.icon"
class="me-3"></i>
                {{ method.label }}
            </label>
        </div>
    </div>
</div>

<div class="alert alert-info mt-3">
    <i class="fas fa-info-circle me-2"></i>
    {{ 'messages.payment_simulation' | translate
}}
</div>
</div>
<div class="col-md-4">
    <div class="delivery-info-card">
        <h6>Delivery Information</h6>
        <div class="info-item">
            <i class="fas fa-clock text-primary me-
2"></i>
            <strong>Expected Delivery:</strong>
            <br>

```



```

        <span class="text-muted">{{
getExpectedDeliveryTime() }}</span>
        </div>
        <div class="info-item">
            <i class="fas fa-map-marker-alt text-
primary me-2"></i>
            <strong>Collection Point:</strong>
            <br>
            <span class="text-muted">{{
certificateForm.get('collectionLocation')?.value
            }}</span>
        </div>
        <div class="info-item">
            <i class="fas fa-shield-alt text-success
me-2"></i>
            <strong>Certified Document:</strong>
            <br>
            <span class="text-muted">Official
government seal and signature</span>
        </div>
    </div>
</div>

<!-- Step 4: Confirmation -->
<div class="form-step" *ngIf="currentStep === 4">
    <div class="confirmation-content text-center">
        <div class="success-icon mb-4">
            <i class="fas fa-check-circle fa-4x text-
success"></i>
        </div>
        <h3>Request Submitted Successfully!</h3>
        <p class="text-muted mb-4">Your birth certificate
request has been submitted and payment
        processed.</p>

        <div class="confirmation-details">
            <div class="detail-item">
                <strong>Tracking Number:</strong>
                <span class="text-primary">{{
application?.trackingNumber }}</span>
            </div>
            <div class="detail-item">
                <strong>Expected Completion:</strong> {{
getExpectedDeliveryTime() }}
            </div>
            <div class="detail-item">

```

```

                <strong>Collection Point:</strong> {{
certificateForm.get('collectionLocation')?.value
                }}
            </div>
            <div class="detail-item">
                <strong>Total Paid:</strong> {{
calculateTotalAmount() }} XAF
            </div>
        </div>
    </div>
</div>

</form>

<!-- Navigation Buttons -->
<div class="form-navigation mt-4">
    <div class="d-flex justify-content-between">
        <button type="button" class="btn btn-outline-secondary"
*ngIf="currentStep > 1 && currentStep < 4"
            (click)="previousStep()">
            <i class="fas fa-arrow-left me-2"></i>
            Previous
        </button>

        <div class="ms-auto">
            <button type="button" class="btn btn-primary"
*ngIf="currentStep < 2" (click)="nextStep()">
                Next
                <i class="fas fa-arrow-right ms-2"></i>
            </button>

            <button type="button" class="btn btn-success"
*ngIf="currentStep === 2"
                [disabled]="isSubmitting"
            (click)="onSubmitApplication()">
                <span *ngIf="isSubmitting" class="spinner-border
spinner-border-sm me-2"></span>
                Submit Request
            </button>

            <button type="button" class="btn btn-warning"
*ngIf="currentStep === 3" [disabled]="isLoading"
            (click)="onPayment()">
                <span *ngIf="isLoading" class="spinner-border
spinner-border-sm me-2"></span>
                Pay Now
            </button>
        </div>
    </div>

```

```

                <button type="button" class="btn btn-primary"
*ngIf="currentStep === 4"
                (click)="goToTracking()">
                <i class="fas fa-search me-2"></i>
                Track Request
            </button>
        </div>
    </div>
</div>
</div>
</div>
</div>

```

- birth-certificate.component.scss :src/app/pages/birth-certificate/birth-certificate.component.scss:

```

.birth-certificate-page {
    background: #f8f9fa;
    min-height: 100vh;

    .page-header {
        .page-title {
            font-size: 2.2rem;
            font-weight: 700;
            color: var(--dark-color);
        }
    }

    .tracking-info {
        background: var(--success-color);
        color: white;
        padding: 0.5rem 1rem;
        border-radius: 20px;
        font-size: 0.9rem;
    }
}

.progress-container {
    background: white;
    padding: 2rem;
    border-radius: 15px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);

    .progress-steps {
        display: flex;
        align-items: center;
        justify-content: center;

        .step {

```

```

    display: flex;
    flex-direction: column;
    align-items: center;
    opacity: 0.5;
    transition: all 0.3s ease;

    &.active,
    &.completed {
      opacity: 1;
    }

    &.completed .step-circle {
      background-color: var(--success-color);
    }

    .step-circle {
      width: 45px;
      height: 45px;
      border-radius: 50%;
      background-color: var(--success-color);
      color: white;
      display: flex;
      align-items: center;
      justify-content: center;
      font-weight: bold;
      margin-bottom: 0.5rem;
    }

    .step-label {
      font-size: 0.85rem;
      text-align: center;
      font-weight: 500;
    }
  }

  .step-line {
    width: 80px;
    height: 3px;
    background-color: #dee2e6;
    margin: 0 1rem;

    &.completed {
      background-color: var(--success-color);
    }
  }
}

}

}

.application-form {
```

```

background: white;
padding: 2.5rem;
border-radius: 15px;
box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);

.form-step {
  .step-header {
    h3 {
      color: var(--success-color);
      font-weight: 600;
      margin-bottom: 0.5rem;
    }
  }
}

.request-type-selection {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;

  .form-check-card {
    margin: 0;

    .form-check-input {
      display: none;

      &:checked+.form-check-label {
        border-color: var(--success-color);
        background-color: rgba(52, 168, 83, 0.05);

        .card-content {
          color: var(--success-color);
        }
      }
    }

    .form-check-label {
      border: 2px solid #dee2e6;
      border-radius: 12px;
      padding: 1.5rem;
      cursor: pointer;
      transition: all 0.3s ease;
      display: block;
      text-align: center;

      &:hover {
        border-color: var(--success-color);
        transform: translateY(-2px);
      }
    }
  }
}

```

```

        .card-content {
            h6 {
                margin-bottom: 0.5rem;
                font-weight: 600;
            }
        }
    }
}

.form-check-premium {
    .form-check-label {
        cursor: pointer;

        .premium-option {
            background: #fff8e1;
            border: 2px solid #ffc107;
            border-radius: 12px;
            padding: 1rem;
            transition: all 0.3s ease;

            &:hover {
                background: #fff3c4;
            }

            .premium-header {
                display: flex;
                align-items: center;
                margin-bottom: 0.5rem;
            }
        }
    }
}

.form-check-input:checked+.form-check-label .premium-option {
    background: #e8f5e8;
    border-color: var(--success-color);
}

}

.payment-summary {
    background: #f8f9fa;
    padding: 1.5rem;
    border-radius: 12px;

    .fee-item {
        margin-bottom: 0.5rem;
    }
}

```

```

        .total-amount {
            font-size: 1.1rem;
        }
    }

    .payment-options {
        .payment-option {
            margin-bottom: 1rem;

            input[type="radio"] {
                display: none;

                &:checked+.payment-label {
                    background-color: rgba(52, 168, 83, 0.1);
                    border-color: var(--success-color);
                    color: var(--success-color);
                }
            }

            .payment-label {
                display: block;
                padding: 1rem 1.5rem;
                border: 2px solid #dee2e6;
                border-radius: 8px;
                cursor: pointer;
                transition: all 0.3s ease;

                &:hover {
                    border-color: var(--success-color);
                }
            }
        }
    }

    .delivery-info-card {
        background: #f8f9fa;
        padding: 1.5rem;
        border-radius: 12px;

        h6 {
            font-weight: 600;
            margin-bottom: 1rem;
        }

        .info-item {
            margin-bottom: 1.5rem;

            &:last-child {
                margin-bottom: 0;
            }
        }
    }

```

```

    }
  }
}

.confirmation-content {
  padding: 3rem 2rem;

  .success-icon {
    animation: bounceIn 0.8s ease;
  }

  .confirmation-details {
    background: #f8f9fa;
    padding: 2rem;
    border-radius: 12px;
    margin: 2rem 0;

    .detail-item {
      margin-bottom: 1rem;

      &.last-child {
        margin-bottom: 0;
      }
    }
  }
}

.form-navigation {
  background: white;
  padding: 1.5rem 2.5rem;
  border-radius: 0 0 15px 15px;
  margin-top: -1px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

@media (max-width: 768px) {
  .birth-certificate-page {
    .request-type-selection {
      grid-template-columns: 1fr;
    }

    .progress-steps {
      .step-label {
        font-size: 0.7rem;
      }

      .step-line {

```



```

        width: 40px;
    }
}
}
}

```

- home.component.ts :src/app/pages/home/home.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { TranslateService } from '@ngx-translate/core';
import { ServicesDataService } from '../../services/services-data.service';
import { AuthService } from '../../services/auth.service';
import { ServiceCategory, Service } from '../../models/service.model';
import { ServiceModalComponent } from '../../components/modals/service-modal/service-modal.component';
import { LoginModalComponent } from '../../components/modals/login-modal/login-modal.component';

@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.scss']
})
export class HomeComponent implements OnInit {
  serviceCategories: ServiceCategory[] = [];
  searchQuery = '';
  isLoading = true;

  constructor(
    private router: Router,
    private modalService: NgbModal,
    private servicesDataService: ServicesDataService,
    private authService: AuthService,
    private translate: TranslateService
  ) { }

  ngOnInit(): void {
    this.loadServiceCategories();
  }

  private loadServiceCategories(): void {
    this.servicesDataService.getServiceCategories().subscribe({
      next: (categories) => {
        this.serviceCategories = categories;
        this.isLoading = false;
      },
    },
  },

```

```

        error: (error) => {
            console.error('Error loading service categories:', error);
            this.isLoading = false;
        }
    });
}

onServiceClick(service: Service): void {
    if (!this.authService.isAuthenticated()) {
        const loginModal = this.modalService.open(LoginModalComponent, {
            centered: true,
            backdrop: 'static'
        });

        loginModal.result.then((result) => {
            if (result === 'login_success') {
                this.openServiceModal(service);
            }
        });
    } else {
        this.openServiceModal(service);
    }
}

private openServiceModal(service: Service): void {
    const modalRef = this.modalService.open(ServiceModalComponent, {
        centered: true,
        size: 'lg'
    });

    modalRef.componentInstance.service = service;
}

onCategoryClick(categoryId: string): void {
    this.router.navigate(['/procedures', categoryId]);
}

onSearch(): void {
    if (this.searchQuery.trim()) {
        this.router.navigate(['/procedures'], {
            queryParams: { search: this.searchQuery.trim() }
        });
    }
}

getCategoryIcon(icon: string): string {
    return icon || 'fas fa-cog';
}

```

```

getPassportService(): Service | null {
  for (const category of this.serviceCategories) {
    const passportService = category.services.find(service => service.id ===
'passport');
    if (passportService) {
      return passportService;
    }
  }
  return null;
}
}

```

- home.component.html :src/app/pages/home/home.component.html:

```

<div class="home-page">
  <!-- Hero Section -->
  <section class="hero-section">
    <div class="container">
      <div class="row align-items-center min-vh-50">
        <div class="col-lg-6">
          <div class="hero-content">
            <h1 class="hero-title">{{ 'app.welcome' | translate
}}</h1>
            <p class="hero-subtitle">{{ 'app.subtitle' | translate
}}</p>
            <div class="search-container mt-4">
              <div class="input-group input-group-lg">
                <input type="text" class="form-control search-
input" [(ngModel)]="searchQuery"
                    (keyup.enter)="onSearch()"
placeholder="Search for services...">
                <button class="btn btn-primary search-btn"
(click)="onSearch()">
                  <i class="fas fa-search"></i>
                  {{ 'buttons.search' | translate }}
                </button>
              </div>
            </div>
          </div>
        </div>
        <div class="col-lg-6">
          <div class="hero-image text-center"></div>
        </div>
      </div>
    </div>
  </section>

  <!-- Service Categories Section -->

```

```

<section class="categories-section py-5">
  <div class="container">
    <div class="row">
      <div class="col-12">
        <h2 class="section-title text-center mb-5">Government
Services</h2>
      </div>
    </div>

    <!-- Loading State -->
    <div class="text-center" *ngIf="isLoading">
      <div class="spinner-border text-primary" role="status">
        <span class="visually-hidden">Loading...</span>
      </div>
    </div>

    <!-- Service Categories Grid -->
    <div class="row g-4" *ngIf="!isLoading">
      <div class="col-lg-4 col-md-6" *ngFor="let category of
serviceCategories">
        <div class="category-card"
(click)="onCategoryClick(category.id)">
          <div class="category-header">
            <div class="category-icon">
              <i
[class]="getCategoryIcon(category.icon)"></i>
            </div>
            <h4 class="category-title">{{ category.nameKey |
translate }}</h4>
          </div>
          <p class="category-description">{{
category.descriptionKey | translate }}</p>

          <!-- Services List -->
          <div class="services-list" *ngIf="category.services &&
category.services.length > 0">
            <h6 class="services-title">Available
Services:</h6>
            <ul class="list-unstyled">
              <li *ngFor="let service of category.services |
slice:0:3" class="service-link"
                (click)="onServiceClick(service);
$event.stopPropagation()">
                <i class="fas fa-chevron-right me-2"></i>
                {{ service.nameKey | translate }}
              </li>
              <li *ngIf="category.services.length > 3"
class="text-muted small">

```

```

+{{ category.services.length - 3 }} more
services
        </li>
    </ul>
</div>
</div>
</div>
</div>
</div>
</div>
</section>

<!-- Quick Actions Section -->
<section class="quick-actions-section py-5 bg-light">
    <div class="container">
        <div class="row">
            <div class="col-12 text-center">
                <h3 class="mb-4">Quick Actions</h3>
            </div>
            <div class="row g-4 justify-content-center">
                <div class="col-md-3 col-sm-6">
                    <div class="quick-action-card" routerLink="/procedures">
                        <i class="fas fa-list-alt"></i>
                        <h5>Browse Procedures</h5>
                        <p>Explore all available government services</p>
                    </div>
                </div>
                <div class="col-md-3 col-sm-6">
                    <div class="quick-action-card" routerLink="/search-files">
                        <i class="fas fa-search"></i>
                        <h5>Track Application</h5>
                        <p>Search and track your submitted applications</p>
                    </div>
                </div>
                <div class="col-md-3 col-sm-6">
                    <div class="quick-action-card" routerLink="/registration">
                        <i class="fas fa-user-plus"></i>
                        <h5>Create Account</h5>
                        <p>Register to access government services</p>
                    </div>
                </div>
                <div class="col-md-3 col-sm-6">
                    <div class="quick-action-card"
                        (click)="getPassportService() &&
onServiceClick(getPassportService())">
                        <i class="fas fa-passport"></i>
                        <h5>Apply for Passport</h5>
                        <p>Start your passport application process</p>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>
      </div>
    </div>
  </section>
</div>

```

- home.component.scss :src/app/pages/home/home.component.scss:

```

.hero-section {
  background: linear-gradient(135deg, var(--primary-color) 0%, var(--secondary-color) 100%);
  color: white;
  min-height: 70vh;
  display: flex;
  align-items: center;

  .hero-title {
    font-size: 3.5rem;
    font-weight: 700;
    margin-bottom: 1rem;

    @media (max-width: 768px) {
      font-size: 2.5rem;
    }
  }

  .hero-subtitle {
    font-size: 1.2rem;
    opacity: 0.9;
    margin-bottom: 2rem;
  }

  .search-container {
    .search-input {
      border: none;
      border-radius: 50px 0 0 50px;
      padding: 1rem 1.5rem;
      font-size: 1.1rem;

      &:focus {
        box-shadow: none;
        border-color: white;
      }
    }

    .search-btn {
      border-radius: 0 50px 50px 0;
      padding: 1rem 2rem;
    }
  }
}

```

```

        font-weight: 600;
        background-color: var(--warning-color);
        border-color: var(--warning-color);
        color: var(--dark-color);

        &:hover {
            background-color: var(--warning-color-dark);
            transform: translateY(-1px);
        }
    }
}

.hero-image img {
    max-height: 300px;
    filter: drop-shadow(0 10px 20px rgba(0, 0, 0, 0.2));
}

.categories-section {
    .section-title {
        font-size: 2.5rem;
        font-weight: 700;
        color: var(--dark-color);
        margin-bottom: 3rem;
    }

    .category-card {
        background: white;
        border-radius: 15px;
        padding: 2rem;
        box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
        cursor: pointer;
        transition: all 0.3s ease;
        height: 100%;

        &:hover {
            transform: translateY(-10px);
            box-shadow: 0 15px 35px rgba(0, 0, 0, 0.2);
        }

        .category-header {
            text-align: center;
            margin-bottom: 1.5rem;
        }

        .category-icon {
            width: 80px;
            height: 80px;
            margin: 0 auto 1rem;

```

```

        background: linear-gradient(135deg, var(--primary-color), var(--
secondary-color));
        border-radius: 50%;
        display: flex;
        align-items: center;
        justify-content: center;
        color: white;
        font-size: 2rem;
    }

    .category-title {
        font-size: 1.3rem;
        font-weight: 600;
        color: var(--dark-color);
        margin-bottom: 0;
    }

    .category-description {
        color: #666;
        text-align: center;
        margin-bottom: 1.5rem;
    }

    .services-list {
        border-top: 1px solid #eee;
        padding-top: 1rem;

        .services-title {
            font-size: 0.9rem;
            font-weight: 600;
            color: var(--primary-color);
            margin-bottom: 0.5rem;
        }

        .service-link {
            padding: 0.25rem 0;
            cursor: pointer;
            transition: all 0.3s ease;
            border-radius: 4px;
            padding-left: 0.5rem;

            &:hover {
                background-color: #f8f9fa;
                color: var(--primary-color);
                padding-left: 1rem;
            }
        }
    }
}

```



```

}

.quick-actions-section {
  .quick-action-card {
    background: white;
    padding: 2rem;
    border-radius: 15px;
    text-align: center;
    cursor: pointer;
    transition: all 0.3s ease;
    box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);

    &:hover {
      transform: translateY(-5px);
      box-shadow: 0 10px 25px rgba(0, 0, 0, 0.15);
    }

    i {
      font-size: 3rem;
      color: var(--primary-color);
      margin-bottom: 1rem;
    }

    h5 {
      font-weight: 600;
      margin-bottom: 0.5rem;
    }

    p {
      color: #666;
      margin-bottom: 0;
      font-size: 0.9rem;
    }
  }
}

```

- passport-application.component.ts :src/app/pages/passport-application/passport-application.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { TranslateService } from '@ngx-translate/core';
import { ApplicationService } from '../../services/application.service';
import { AuthService } from '../../services/auth.service';
import { Application, ApplicationStatus, PaymentMethod, PaymentStatus } from '../../models/application.model';
import { ApplicantType } from '../../models/service.model';

```

```

@Component({
  selector: 'app-passport-application',
  templateUrl: './passport-application.component.html',
  styleUrls: ['./passport-application.component.scss']
})
export class PassportApplicationComponent implements OnInit {
  application: Application | null = null;
  passportForm: FormGroup;
  currentStep = 1;
  totalSteps = 5;
  isLoading = false;
  isSubmitting = false;
  errorMessage = '';
  successMessage = '';

  // File upload properties
  uploadedFiles: { [key: string]: File } = {};
  requiredDocuments = [
    { id: 'birth_certificate', name: 'Certified Copy of Birth Certificate',
required: true },
    { id: 'national_id', name: 'National Identity Card Copy', required: true
},
    { id: 'passport_photos', name: '12 Passport Photos', required: true },
    { id: 'residence_proof', name: 'Proof of Residence', required: true },
    { id: 'marriage_certificate', name: 'Marriage Certificate (if married)',
required: false }
  ];

  // Payment properties
  selectedPaymentMethod: PaymentMethod = PaymentMethod.MTN_MONEY;
  paymentMethods = [
    { value: PaymentMethod.MTN_MONEY, label: 'MTN Mobile Money', icon: 'fas
fa-mobile-alt' },
    { value: PaymentMethod.ORANGE_MONEY, label: 'Orange Money', icon: 'fas fa-
mobile-alt' },
    { value: PaymentMethod.CREDIT_CARD, label: 'Credit Card', icon: 'fas fa-
credit-card' },
    { value: PaymentMethod.BANK_TRANSFER, label: 'Bank Transfer', icon: 'fas
fa-university' }
  ];

  constructor(
    private fb: FormBuilder,
    private route: ActivatedRoute,
    private router: Router,
    private applicationService: ApplicationService,
    private authService: AuthService,
    private translate: TranslateService
  ) {

```

```

) {
  this.passportForm = this.fb.group({
    // Personal Information
    firstName: ['', Validators.required],
    lastName: ['', Validators.required],
    middleName: [''],
    dateOfBirth: ['', Validators.required],
    placeOfBirth: ['', Validators.required],
    nationality: ['Cameroonian', Validators.required],
    gender: ['', Validators.required],
    maritalStatus: ['', Validators.required],

    // Contact Information
    phoneNumber: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    currentAddress: ['', Validators.required],
    permanentAddress: ['', Validators.required],

    // Emergency Contact
    emergencyContactName: ['', Validators.required],
    emergencyContactPhone: ['', Validators.required],
    emergencyContactRelation: ['', Validators.required],

    // Previous Passport Information (if applicable)
    hasPreviousPassport: [false],
    previousPassportNumber: [''],
    previousPassportIssueDate: [''],
    previousPassportExpiryDate: [''],

    // Collection Method
    collectionMethod: ['pickup', Validators.required],
    collectionAddress: ['']
  });
}

```

```

ngOnInit(): void {
  this.route.queryParams.subscribe(params => {
    if (params['applicationId']) {
      this.loadApplication(params['applicationId']);
    } else {
      this.createNewApplication();
    }
  });
}

this.prefillUserData();
}

```

```

private loadApplication(applicationId: string): void {
  this.applicationService.getApplicationById(applicationId).subscribe({

```

```

    next: (application) => {
      if (application) {
        this.application = application;
        this.determineCurrentStep();
      }
    },
    error: (error) => {
      console.error('Error loading application:', error);
      this.errorMessage = 'Failed to load application details.';
    }
  });
}

```

```

private createNewApplication(): void {
  const currentUser = this.authService.getCurrentUser();
  if (currentUser) {
    this.applicationService.createApplication(
      'passport',
      'Passport Application',
      ApplicantType.SELF,
      currentUser.id!
    ).subscribe({
      next: (application) => {
        this.application = application;
      },
      error: (error) => {
        console.error('Error creating application:', error);
        this.errorMessage = 'Failed to create application.';
      }
    });
  }
}

```

```

private prefillUserData(): void {
  const currentUser = this.authService.getCurrentUser();
  if (currentUser) {
    this.passportForm.patchValue({
      phoneNumber: currentUser.phoneNumber,
      email: currentUser.email,
      firstName: currentUser.firstName || '',
      lastName: currentUser.lastName || ''
    });
  }
}

```

```

private determineCurrentStep(): void {
  if (!this.application) return;

  switch (this.application.status) {

```

```

        case ApplicationStatus.DRAFT:
            this.currentStep = 1;
            break;
        case ApplicationStatus.SUBMITTED:
            this.currentStep = 3;
            break;
        case ApplicationStatus.PAYMENT_PENDING:
            this.currentStep = 4;
            break;
        case ApplicationStatus.PROCESSING:
            this.currentStep = 5;
            break;
        default:
            this.currentStep = 1;
    }
}

nextStep(): void {
    if (this.currentStep < this.totalSteps) {
        if (this.validateCurrentStep()) {
            this.currentStep++;
        }
    }
}

previousStep(): void {
    if (this.currentStep > 1) {
        this.currentStep--;
    }
}

private validateCurrentStep(): boolean {
    switch (this.currentStep) {
        case 1: // Personal Information
            const personalFields = ['firstName', 'lastName', 'dateOfBirth',
'placeOfBirth', 'gender', 'maritalStatus'];
            return this.validateFields(personalFields);
        case 2: // Contact & Emergency Information
            const contactFields = ['phoneNumber', 'email', 'currentAddress',
'permanentAddress',
            'emergencyContactName', 'emergencyContactPhone',
'emergencyContactRelation'];
            return this.validateFields(contactFields);
        case 3: // Document Upload
            return this.validateDocumentUploads();
        default:
            return true;
    }
}

```

```

private validateFields(fieldNames: string[]): boolean {
  let isValid = true;
  fieldNames.forEach(fieldName => {
    const control = this.passportForm.get(fieldName);
    if (control && control.invalid) {
      control.markAsTouched();
      isValid = false;
    }
  });
  return isValid;
}

// Rendre cette méthode publique pour l'utiliser dans le template
validateDocumentUploads(): boolean {
  const requiredDocs = this.requiredDocuments.filter(doc => doc.required);
  const uploadedRequiredDocs = requiredDocs.filter(doc =>
this.uploadedFiles[doc.id]);
  return uploadedRequiredDocs.length === requiredDocs.length;
}

onFileUpload(event: any, documentId: string): void {
  const file = event.target.files[0];
  if (file) {
    // Validate file type and size
    const allowedTypes = ['application/pdf', 'image/jpeg', 'image/png'];
    const maxSize = 5 * 1024 * 1024; // 5MB

    if (!allowedTypes.includes(file.type)) {
      this.errorMessage = 'Invalid file type. Please upload PDF, JPEG, or
PNG files only.';
      return;
    }

    if (file.size > maxSize) {
      this.errorMessage = 'File size too large. Please upload files smaller
than 5MB.';
      return;
    }

    this.uploadedFiles[documentId] = file;
    this.errorMessage = '';
  }
}

removeFile(documentId: string): void {
  delete this.uploadedFiles[documentId];
}

```

```

onSubmitApplication(): void {
  if (this.passportForm.valid && this.application) {
    this.isSubmitting = true;
    this.errorMessage = '';

    // Simulate form submission
    setTimeout(() => {
      this.application!.status = ApplicationStatus.SUBMITTED;
      this.isSubmitting = false;
      this.successMessage = 'Application submitted successfully!';
      this.currentStep = 4; // Move to payment step
    }, 2000);
  } else {
    this.markFormGroupTouched();
  }
}

private markFormGroupTouched(): void {
  Object.keys(this.passportForm.controls).forEach(key => {
    const control = this.passportForm.get(key);
    if (control) {
      control.markAsTouched();
    }
  });
}

onPayment(): void {
  if (this.application) {
    this.isLoading = true;
    this.errorMessage = '';

    this.applicationService.simulatePayment(
      this.application.id,
      110000, // Passport fee in XAF
      this.selectedPaymentMethod
    ).subscribe({
      next: (payment) => {
        this.isLoading = false;
        if (payment.status === PaymentStatus.COMPLETED) {
          this.successMessage = 'Payment completed successfully!';
          this.currentStep = 5; // Move to confirmation step
        }
      },
      error: (error) => {
        this.isLoading = false;
        this.errorMessage = 'Payment failed. Please try again.';
      }
    });
  }
}

```

```

}

goToTracking(): void {
  if (this.application) {
    this.router.navigate(['/search-files'], {
      queryParams: { trackingNumber: this.application.trackingNumber }
    });
  }
}

calculateAge(dateOfBirth: string): number {
  if (!dateOfBirth) return 0;
  const today = new Date();
  const birthDate = new Date(dateOfBirth);
  let age = today.getFullYear() - birthDate.getFullYear();
  const monthDiff = today.getMonth() - birthDate.getMonth();
  if (monthDiff < 0 || (monthDiff === 0 && today.getDate() <
  birthDate.getDate())) {
    age--;
  }
  return age;
}
}

```

- passport-application.component.html :src/app/pages/passport-application/passport-application.component.html:

```

<div class="passport-application-page">
  <div class="container py-4">

    <!-- Page Header -->
    <div class="page-header mb-4">
      <div class="row align-items-center">
        <div class="col-md-8">
          <h1 class="page-title">
            <i class="fas fa-passport me-3"></i>
            Passport Application
          </h1>
          <p class="text-muted">Apply for a Cameroonian passport
online</p>
        </div>
        <div class="col-md-4 text-md-end">
          <div class="tracking-info" *ngIf="application">
            <strong>Tracking #:</strong> {{
application.trackingNumber }}
          </div>
        </div>
      </div>
    </div>
  </div>

```



```

</div>

<!-- Progress Indicator -->
<div class="progress-container mb-5">
    <div class="progress-steps">
        <div class="step" [class.active]="currentStep >= 1"
[class.completed]="currentStep > 1">
            <div class="step-circle">1</div>
            <span class="step-label">Personal Info</span>
        </div>
        <div class="step-line" [class.completed]="currentStep >
1"></div>
        <div class="step" [class.active]="currentStep >= 2"
[class.completed]="currentStep > 2">
            <div class="step-circle">2</div>
            <span class="step-label">Contact Details</span>
        </div>
        <div class="step-line" [class.completed]="currentStep >
2"></div>
        <div class="step" [class.active]="currentStep >= 3"
[class.completed]="currentStep > 3">
            <div class="step-circle">3</div>
            <span class="step-label">Documents</span>
        </div>
        <div class="step-line" [class.completed]="currentStep >
3"></div>
        <div class="step" [class.active]="currentStep >= 4"
[class.completed]="currentStep > 4">
            <div class="step-circle">4</div>
            <span class="step-label">Payment</span>
        </div>
        <div class="step-line" [class.completed]="currentStep >
4"></div>
        <div class="step" [class.active]="currentStep >= 5">
            <div class="step-circle">5</div>
            <span class="step-label">Confirmation</span>
        </div>
    </div>
</div>

<!-- Application Form -->
<div class="application-form">

    <!-- Error/Success Messages -->
    <div class="alert alert-danger" *ngIf="errorMessage" role="alert">
        <i class="fas fa-exclamation-triangle me-2"></i>
        {{ errorMessage }}
    </div>

```

```

        <div class="alert alert-success" *ngIf="successMessage"
role="alert">
            <i class="fas fa-check-circle me-2"></i>
            {{ successMessage }}
        </div>

        <form [formGroup]="passportForm">

            <!-- Step 1: Personal Information -->
            <div class="form-step" *ngIf="currentStep === 1">
                <div class="step-header mb-4">
                    <h3>Personal Information</h3>
                    <p class="text-muted">Please provide your personal
details as they appear on your birth
                    certificate</p>
                </div>

                <div class="row">
                    <div class="col-md-4 mb-3">
                        <label class="form-label">First Name *</label>
                        <input type="text" class="form-control"
formControlName="firstName"
                        [class.is-
invalid]="passportForm.get('firstName')?.invalid &&
passportForm.get('firstName')?.touched">
                        <div class="invalid-feedback">First name is
required</div>
                    </div>
                    <div class="col-md-4 mb-3">
                        <label class="form-label">Middle Name</label>
                        <input type="text" class="form-control"
formControlName="middleName">
                    </div>
                    <div class="col-md-4 mb-3">
                        <label class="form-label">Last Name *</label>
                        <input type="text" class="form-control"
formControlName="lastName"
                        [class.is-
invalid]="passportForm.get('lastName')?.invalid &&
passportForm.get('lastName')?.touched">
                        <div class="invalid-feedback">Last name is
required</div>
                    </div>
                </div>

                <div class="row">
                    <div class="col-md-4 mb-3">
                        <label class="form-label">Date of Birth *</label>

```

```

        <input type="date" class="form-control"
formControlName="dateOfBirth"
        [class.is-
invalid]="passportForm.get('dateOfBirth')?.invalid &&
passportForm.get('dateOfBirth')?.touched">
        <div class="invalid-feedback">Date of birth is
required</div>
        <small class="form-text text-muted"
*ngIf="passportForm.get('dateOfBirth')?.value">
        Age: {{
calculateAge(passportForm.get('dateOfBirth')?.value) }} years
        </small>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Place of Birth *</label>
        <input type="text" class="form-control"
formControlName="placeOfBirth"
        [class.is-
invalid]="passportForm.get('placeOfBirth')?.invalid &&
passportForm.get('placeOfBirth')?.touched">
        <div class="invalid-feedback">Place of birth is
required</div>
    </div>
    <div class="col-md-4 mb-3">
        <label class="form-label">Nationality *</label>
        <input type="text" class="form-control"
formControlName="nationality" readonly>
    </div>
</div>

<div class="row">
    <div class="col-md-6 mb-3">
        <label class="form-label">Gender *</label>
        <select class="form-select"
formControlName="gender"
        [class.is-
invalid]="passportForm.get('gender')?.invalid &&
passportForm.get('gender')?.touched">
            <option value="">Select Gender</option>
            <option value="Male">Male</option>
            <option value="Female">Female</option>
        </select>
        <div class="invalid-feedback">Gender is
required</div>
    </div>
    <div class="col-md-6 mb-3">
        <label class="form-label">Marital Status *</label>
        <select class="form-select"
formControlName="maritalStatus"

```

```

                                [class.is-
invalid]="passportForm.get('maritalStatus')?.invalid &&
passportForm.get('maritalStatus')?.touched">
                                <option value="">Select Marital
Status</option>
                                <option value="Single">Single</option>
                                <option value="Married">Married</option>
                                <option value="Divorced">Divorced</option>
                                <option value="Widowed">Widowed</option>
                                </select>
                                <div class="invalid-feedback">Marital status is
required</div>
                                </div>
                            </div>
                        </div>

<!-- Step 2: Contact Information -->
<div class="form-step" *ngIf="currentStep === 2">
    <div class="step-header mb-4">
        <h3>Contact & Emergency Information</h3>
        <p class="text-muted">Provide your current contact
details and emergency contact</p>
    </div>

    <!-- Contact Information -->
    <h5 class="mb-3">Contact Information</h5>
    <div class="row">
        <div class="col-md-6 mb-3">
            <label class="form-label">Phone Number *</label>
            <input type="tel" class="form-control"
formControlName="phoneNumber"
                                [class.is-
invalid]="passportForm.get('phoneNumber')?.invalid &&
passportForm.get('phoneNumber')?.touched">
                                <div class="invalid-feedback">Phone number is
required</div>
                                </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Email Address *</label>
                <input type="email" class="form-control"
formControlName="email"
                                [class.is-
invalid]="passportForm.get('email')?.invalid &&
passportForm.get('email')?.touched">
                                <div class="invalid-feedback">Valid email is
required</div>
                                </div>
            </div>
        </div>

```

```

        <div class="row">
            <div class="col-md-6 mb-3">
                <label class="form-label">Current Address
*</label>
                <textarea class="form-control" rows="3"
formControlName="currentAddress"
                [class.is-
invalid]="passportForm.get('currentAddress')?.invalid &&
passportForm.get('currentAddress')?.touched"></textarea>
                <div class="invalid-feedback">Current address is
required</div>
            </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Permanent Address
*</label>
                <textarea class="form-control" rows="3"
formControlName="permanentAddress"
                [class.is-
invalid]="passportForm.get('permanentAddress')?.invalid &&
passportForm.get('permanentAddress')?.touched"></textarea>
                <div class="invalid-feedback">Permanent address is
required</div>
            </div>
        </div>

        <!-- Emergency Contact -->
        <h5 class="mb-3 mt-4">Emergency Contact</h5>
        <div class="row">
            <div class="col-md-4 mb-3">
                <label class="form-label">Full Name *</label>
                <input type="text" class="form-control"
formControlName="emergencyContactName"
                [class.is-
invalid]="passportForm.get('emergencyContactName')?.invalid &&
passportForm.get('emergencyContactName')?.touched">
                <div class="invalid-feedback">Emergency contact
name is required</div>
            </div>
            <div class="col-md-4 mb-3">
                <label class="form-label">Phone Number *</label>
                <input type="tel" class="form-control"
formControlName="emergencyContactPhone"
                [class.is-
invalid]="passportForm.get('emergencyContactPhone')?.invalid &&
passportForm.get('emergencyContactPhone')?.touched">
                <div class="invalid-feedback">Emergency contact
phone is required</div>
            </div>
        </div>
    </div>

```

```

        <label class="form-label">Relationship *</label>
        <select class="form-select"
formControlName="emergencyContactRelation"
        [class.is-
invalid]="passportForm.get('emergencyContactRelation')?.invalid &&
passportForm.get('emergencyContactRelation')?.touched">
            <option value="">Select Relationship</option>
            <option value="Parent">Parent</option>
            <option value="Spouse">Spouse</option>
            <option value="Sibling">Sibling</option>
            <option value="Child">Child</option>
            <option value="Friend">Friend</option>
            <option value="Other">Other</option>
        </select>
        <div class="invalid-feedback">Relationship is
required</div>
    </div>
</div>
</div>

<!-- Step 3: Document Upload -->
<div class="form-step" *ngIf="currentStep === 3">
    <div class="step-header mb-4">
        <h3>Required Documents</h3>
        <p class="text-muted">Upload all required documents.
Files must be in PDF, JPEG, or PNG format
(max 5MB each)</p>
    </div>

    <div class="documents-grid">
        <div class="document-item" *ngFor="let doc of
requiredDocuments">
            <div class="document-header">
                <h6 class="document-name">
                    {{ doc.name }}
                    <span class="badge bg-danger ms-2"
*ngIf="doc.required">Required</span>
                    <span class="badge bg-secondary ms-2"
*ngIf="!doc.required">Optional</span>
                </h6>
            </div>

            <div class="document-upload"
*ngIf="!uploadedFiles[doc.id]">
                <input type="file" [id]='`file-` + doc.id'
(change)="onFileUpload($event, doc.id)"
                accept=".pdf,.jpg,.jpeg,.png" class="d-
none">

```

```

        <label [for]="file-' + doc.id" class="upload-
label">
            <i class="fas fa-cloud-upload-alt fa-2x
mb-2"></i>
            <span>Click to upload or drag and
drop</span>
            <small class="text-muted d-block">PDF,
JPEG, PNG (max 5MB)</small>
        </label>
    </div>

    <div class="document-uploaded"
*ngIf="uploadedFiles[doc.id]">
        <div class="uploaded-file">
            <div class="file-info">
                <i class="fas fa-file-alt fa-2x text-
success me-3"></i>
                <div>
                    <strong>{{
uploadedFiles[doc.id].name }}</strong>
                    <small class="text-muted d-
block">{{ (uploadedFiles[doc.id].size /
1024).toFixed(1) }} KB</small>
                </div>
            </div>
            <button type="button" class="btn btn-
outline-danger btn-sm"
                (click)="removeFile(doc.id)">
                <i class="fas fa-trash"></i>
            </button>
        </div>
    </div>
</div>
</div>
</div>
</div>

<!-- Step 4: Payment -->
<div class="form-step" *ngIf="currentStep === 4">
    <div class="step-header mb-4">
        <h3>Payment</h3>
        <p class="text-muted">Complete your payment to process
your passport application</p>
    </div>

    <div class="row">
        <div class="col-md-8">
            <div class="payment-summary mb-4">
                <h5>Payment Summary</h5>
                <div class="fee-item">

```

```

                <div class="d-flex justify-content-
between">
                    <span>Passport Application Fee</span>
                    <strong>110,000 XAF</strong>
                </div>
            </div>
            <hr>
            <div class="total-amount">
                <div class="d-flex justify-content-
between">
                    <strong>Total Amount</strong>
                    <strong class="text-primary">110,000
XAF</strong>
                </div>
            </div>
        </div>
        <div class="payment-methods">
            <h5 class="mb-3">Select Payment Method</h5>
            <div class="payment-options">
                <div class="payment-option" *ngFor="let
method of paymentMethods">
                    <input type="radio" [id]='payment-' +
method.value"
                        [(ngModel)]="selectedPaymentMethod
" [value]="method.value"
                        [ngModelOptions]="{standalone:
true}">
                    <label [for]='payment-' +
method.value" class="payment-label">
                        <i [class]="method.icon"
class="me-3"></i>
                        {{ method.label }}
                    </label>
                </div>
            </div>
        </div>
        <div class="alert alert-info mt-3">
            <i class="fas fa-info-circle me-2"></i>
            {{ 'messages.payment_simulation' | translate
}}
        </div>
    </div>
    <div class="col-md-4">
        <div class="payment-info-card">
            <h6>Important Information</h6>
            <ul class="list-unstyled">

```



```

                <li><i class="fas fa-check text-success
me-2"></i>Secure payment processing</li>
                <li><i class="fas fa-check text-success
me-2"></i>Instant payment confirmation</li>
                <li><i class="fas fa-check text-success
me-2"></i>48-hour processing time</li>
                <li><i class="fas fa-check text-success
me-2"></i>SMS and email notifications</li>
            </ul>
        </div>
    </div>
</div>
</div>
</div>

<!-- Step 5: Confirmation -->
<div class="form-step" *ngIf="currentStep === 5">
    <div class="confirmation-content text-center">
        <div class="success-icon mb-4">
            <i class="fas fa-check-circle fa-4x text-
success"></i>
        </div>
        <h3>Application Submitted Successfully!</h3>
        <p class="text-muted mb-4">Your passport application
has been submitted and payment processed.
        </p>

        <div class="confirmation-details">
            <div class="detail-item">
                <strong>Tracking Number:</strong>
                <span class="text-primary">{{
application?.trackingNumber }}</span>
            </div>
            <div class="detail-item">
                <strong>Expected Processing Time:</strong> 48
hours to 3 months
            </div>
            <div class="detail-item">
                <strong>Next Steps:</strong> Visit DGSN center
for biometric enrollment
            </div>
        </div>
    </div>
</div>
</div>
</div>

</form>

<!-- Navigation Buttons -->
<div class="form-navigation mt-4">
    <div class="d-flex justify-content-between">

```

```

        <button type="button" class="btn btn-outline-secondary"
*ngIf="currentStep > 1 && currentStep < 5"
        (click)="previousStep()">
        <i class="fas fa-arrow-left me-2"></i>
        Previous
    </button>

    <div class="ms-auto">
        <button type="button" class="btn btn-primary"
*ngIf="currentStep < 3" (click)="nextStep()">
        Next
        <i class="fas fa-arrow-right ms-2"></i>
    </button>

        <button type="button" class="btn btn-success"
*ngIf="currentStep === 3"
        [disabled]="isSubmitting ||
!validateDocumentUploads()" (click)="onSubmitApplication()">
        <span *ngIf="isSubmitting" class="spinner-border
spinner-border-sm me-2"></span>
        Submit Application
    </button>

        <button type="button" class="btn btn-warning"
*ngIf="currentStep === 4" [disabled]="isLoading"
        (click)="onPayment()">
        <span *ngIf="isLoading" class="spinner-border
spinner-border-sm me-2"></span>
        Pay Now
    </button>

        <button type="button" class="btn btn-primary"
*ngIf="currentStep === 5"
        (click)="goToTracking()">
        <i class="fas fa-search me-2"></i>
        Track Application
    </button>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>

```

- passport-application.component.scss :src/app/pages/passport-application/passport-application.component.scss:

```
.passport-application-page {
```

```
background: #f8f9fa;
min-height: 100vh;

.page-header {
  .page-title {
    font-size: 2.2rem;
    font-weight: 700;
    color: var(--dark-color);
  }

  .tracking-info {
    background: var(--primary-color);
    color: white;
    padding: 0.5rem 1rem;
    border-radius: 20px;
    font-size: 0.9rem;
  }
}

.progress-container {
  background: white;
  padding: 2rem;
  border-radius: 15px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);

  .progress-steps {
    display: flex;
    align-items: center;
    justify-content: center;

    .step {
      display: flex;
      flex-direction: column;
      align-items: center;
      opacity: 0.5;
      transition: all 0.3s ease;

      &.active,
      &.completed {
        opacity: 1;
      }

      &.completed .step-circle {
        background-color: var(--success-color);
      }

      .step-circle {
        width: 50px;
        height: 50px;
      }
    }
  }
}
```

```

        border-radius: 50%;
        background-color: var(--primary-color);
        color: white;
        display: flex;
        align-items: center;
        justify-content: center;
        font-weight: bold;
        font-size: 1.1rem;
        margin-bottom: 0.5rem;
    }

    .step-label {
        font-size: 0.85rem;
        text-align: center;
        font-weight: 500;
    }
}

.step-line {
    width: 80px;
    height: 3px;
    background-color: #dee2e6;
    margin: 0 1rem;

    &.completed {
        background-color: var(--success-color);
    }
}

}

}

.application-form {
    background: white;
    padding: 2.5rem;
    border-radius: 15px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);

    .form-step {
        .step-header {
            h3 {
                color: var(--primary-color);
                font-weight: 600;
                margin-bottom: 0.5rem;
            }
        }
    }
}

.form-control,
.form-select {

```

```

border-radius: 8px;
padding: 0.75rem 1rem;
border: 1px solid #dee2e6;

&:focus {
  border-color: var(--primary-color);
  box-shadow: 0 0 0 0.2rem rgba(26, 115, 232, 0.25);
}
}

.documents-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 1.5rem;

  .document-item {
    border: 2px dashed #dee2e6;
    border-radius: 12px;
    padding: 1.5rem;
    text-align: center;
    transition: all 0.3s ease;

    &:hover {
      border-color: var(--primary-color);
    }

    .document-header {
      margin-bottom: 1rem;

      .document-name {
        font-weight: 600;
        margin-bottom: 0;
      }
    }

    .upload-label {
      display: block;
      cursor: pointer;
      padding: 2rem 1rem;
      color: #666;
      transition: all 0.3s ease;

      &:hover {
        color: var(--primary-color);
      }
    }

    .document-uploaded {
      .uploaded-file {

```

```

        display: flex;
        align-items: center;
        justify-content: space-between;
        background: #f8f9fa;
        padding: 1rem;
        border-radius: 8px;

        .file-info {
            display: flex;
            align-items: center;
        }
    }
}

.payment-summary {
    background: #f8f9fa;
    padding: 1.5rem;
    border-radius: 12px;

    .fee-item {
        margin-bottom: 0.5rem;
    }

    .total-amount {
        font-size: 1.1rem;
    }
}

.payment-options {
    .payment-option {
        margin-bottom: 1rem;

        input[type="radio"] {
            display: none;

            &.checked+.payment-label {
                background-color: rgba(26, 115, 232, 0.1);
                border-color: var(--primary-color);
                color: var(--primary-color);
            }
        }

        .payment-label {
            display: block;
            padding: 1rem 1.5rem;
            border: 2px solid #dee2e6;
            border-radius: 8px;

```

```

        cursor: pointer;
        transition: all 0.3s ease;

        &:hover {
            border-color: var(--primary-color);
        }
    }
}

.payment-info-card {
    background: #f8f9fa;
    padding: 1.5rem;
    border-radius: 12px;

    h6 {
        font-weight: 600;
        margin-bottom: 1rem;
    }

    li {
        margin-bottom: 0.5rem;
    }
}

.confirmation-content {
    padding: 3rem 2rem;

    .success-icon {
        animation: bounceIn 0.8s ease;
    }

    .confirmation-details {
        background: #f8f9fa;
        padding: 2rem;
        border-radius: 12px;
        margin: 2rem 0;

        .detail-item {
            margin-bottom: 1rem;

            &:last-child {
                margin-bottom: 0;
            }
        }
    }
}
}

```

```

    .form-navigation {
      background: white;
      padding: 1.5rem 2.5rem;
      border-radius: 0 0 15px 15px;
      margin-top: -1px;
      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    }
  }

  @keyframes bounceIn {
    0% {
      opacity: 0;
      transform: scale(0.3);
    }

    50% {
      opacity: 1;
      transform: scale(1.05);
    }

    70% {
      transform: scale(0.9);
    }

    100% {
      opacity: 1;
      transform: scale(1);
    }
  }

  @media (max-width: 768px) {
    .passport-application-page {
      .progress-steps {
        .step-label {
          font-size: 0.7rem;
        }

        .step-line {
          width: 40px;
        }
      }

      .documents-grid {
        grid-template-columns: 1fr;
      }
    }
  }
}

```



- procedures.component.ts :src/app/pages/procedures/procedures.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { TranslateService } from '@ngx-translate/core';
import { ServicesDataService } from '../../services/services-data.service';
import { AuthService } from '../../services/auth.service';
import { ServiceCategory, Service } from '../../models/service.model';
import { ServiceModalComponent } from '../../components/modals/service-modal/service-modal.component';

@Component({
  selector: 'app-procedures',
  templateUrl: './procedures.component.html',
  styleUrls: ['./procedures.component.scss']
})
export class ProceduresComponent implements OnInit {
  serviceCategories: ServiceCategory[] = [];
  filteredServices: Service[] = [];
  selectedCategory: string | null = null;
  searchQuery = '';
  isLoading = true;
  viewMode: 'categories' | 'services' = 'categories';

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private modalService: NgbModal,
    private servicesDataService: ServicesDataService,
    private authService: AuthService,
    private translate: TranslateService
  ) { }

  ngOnInit(): void {
    this.loadServiceCategories();

    // Handle route parameters
    this.route.params.subscribe(params => {
      if (params['category']) {
        this.selectedCategory = params['category'];
        this.viewMode = 'services';
        this.loadCategoryServices(params['category']);
      }
    });

    // Handle query parameters for search
    this.route.queryParams.subscribe(queryParams => {
      if (queryParams['search']) {

```

```

        this.searchQuery = queryParams['search'];
        this.performSearch();
    }
    });
}

private loadServiceCategories(): void {
    this.servicesDataService.getServiceCategories().subscribe({
        next: (categories) => {
            this.serviceCategories = categories;
            this.isLoading = false;
        },
        error: (error) => {
            console.error('Error loading service categories:', error);
            this.isLoading = false;
        }
    });
}

private loadCategoryServices(categoryId: string): void {
    this.servicesDataService.getServicesByCategory(categoryId).subscribe({
        next: (services) => {
            this.filteredServices = services;
            this.isLoading = false;
        },
        error: (error) => {
            console.error('Error loading category services:', error);
            this.isLoading = false;
        }
    });
}

onCategoryClick(categoryId: string): void {
    this.router.navigate(['/procedures', categoryId]);
}

onServiceClick(service: Service): void {
    if (service.isImplemented) {
        // Navigate to service detail page first
        this.router.navigate(['/service-detail', service.id]);
    } else {
        // Show modal for non-implemented services
        const modalRef = this.modalService.open(ServiceModalComponent, {
            centered: true,
            size: 'lg'
        });
        modalRef.componentInstance.service = service;
    }
}
}

```

```

onSearch(): void {
  if (this.searchQuery.trim()) {
    this.performSearch();
  } else {
    this.backToCategories();
  }
}

private performSearch(): void {
  this.isLoading = true;
  this.viewMode = 'services';

  this.servicesDataService.searchServices(this.searchQuery).subscribe({
    next: (services) => {
      this.filteredServices = services;
      this.selectedCategory = null;
      this.isLoading = false;
    },
    error: (error) => {
      console.error('Error searching services:', error);
      this.isLoading = false;
    }
  });
}

backToCategories(): void {
  this.viewMode = 'categories';
  this.selectedCategory = null;
  this.filteredServices = [];
  this.searchQuery = '';
  this.router.navigate(['/procedures']);
}

getCategoryById(categoryId: string | null): ServiceCategory | undefined {
  if (!categoryId) {
    return undefined;
  }
  return this.serviceCategories.find(cat => cat.id === categoryId);
}

getCategoryIcon(icon: string): string {
  return icon || 'fas fa-cog';
}

// Helper method for template to safely get category name
getCategoryName(categoryId: string | null): string {
  const category = this.getCategoryById(categoryId);
  return category ? category.nameKey : '';
}

```

```
}  
}
```

- procedures.component.html :src/app/pages/procedures/procedures.component.html:

```
<div class="procedures-page">  
  <div class="container py-4">  
  
    <!-- Page Header -->  
    <div class="page-header mb-4">  
      <div class="row align-items-center">  
        <div class="col-md-6">  
          <h1 class="page-title">  
            <i class="fas fa-list-alt me-3"></i>  
            {{ 'navigation.procedures' | translate }}  
          </h1>  
          <p class="text-muted" *ngIf="viewMode === 'categories'">  
            Browse government services by category  
          </p>  
          <div class="breadcrumb-nav" *ngIf="viewMode === 'services'  
&& selectedCategory">  
            <nav aria-label="breadcrumb">  
              <ol class="breadcrumb">  
                <li class="breadcrumb-item">  
                  <a href="#" (click)="backToCategories();  
$event.preventDefault()">  
                      {{ 'navigation.procedures' | translate  
}}  
                  </a>  
                </li>  
                <li class="breadcrumb-item active"  
*ngIf="getCategoryById(selectedCategory)">  
                  {{  
getCategoryById(selectedCategory)!.nameKey | translate }}  
                </li>  
              </ol>  
            </nav>  
          </div>  
        </div>  
        <div class="col-md-6">  
          <!-- Search Bar -->  
          <div class="search-container">  
            <div class="input-group">  
              <input type="text" class="form-control"  
[(ngModel)]="searchQuery" (keyup.enter)="onSearch()"  
                placeholder="Search procedures...">  
              <button class="btn btn-primary"  
(click)="onSearch()">
```

```

        <i class="fas fa-search"></i>
      </button>
      <button class="btn btn-outline-secondary"
*ngIf="searchQuery || selectedCategory"
        (click)="backToCategories()">
        <i class="fas fa-times"></i>
      </button>
    </div>
  </div>
</div>
</div>
</div>
</div>

<!-- Loading State -->
<div class="text-center py-5" *ngIf="isLoading">
  <div class="spinner-border text-primary" role="status">
    <span class="visually-hidden">Loading...</span>
  </div>
</div>

<!-- Categories View -->
<div *ngIf="!isLoading && viewMode === 'categories'">
  <div class="row g-4">
    <div class="col-lg-4 col-md-6" *ngFor="let category of
serviceCategories">
      <div class="category-card"
(click)="onCategoryClick(category.id)">
        <div class="category-icon">
          <i [class]="getCategoryIcon(category.icon)"></i>
        </div>
        <h4 class="category-name">{{ category.nameKey |
translate }}</h4>
        <p class="category-description">{{
category.descriptionKey | translate }}</p>
        <div class="category-meta">
          <span class="service-count">
            {{ category.services.length || 0 }} services
          </span>
        </div>
      </div>
    </div>
  </div>
</div>

<!-- Services View -->
<div *ngIf="!isLoading && viewMode === 'services'">
  <div class="row">
    <div class="col-12">
      <div class="services-header mb-4">

```

```

        <h3 *ngIf="selectedCategory &&
getCategoryById(selectedCategory)">
            {{ getCategoryById(selectedCategory)!.nameKey |
translate }}
        </h3>
        <h3 *ngIf="!selectedCategory && searchQuery">
            Search Results for "{{ searchQuery }}"
        </h3>
        <p class="text-muted">{{ filteredServices.length }}
service(s) found</p>
    </div>

    <!-- No Results -->
    <div class="text-center py-5"
*ngIf="filteredServices.length === 0">
        <i class="fas fa-search fa-3x text-muted mb-3"></i>
        <h4>{{ 'messages.no_results' | translate }}</h4>
        <p class="text-muted">Try adjusting your search
criteria</p>
    </div>

    <!-- Services List -->
    <div class="services-grid">
        <div class="service-item" *ngFor="let service of
filteredServices"
            (click)="onServiceClick(service)">
            <div class="service-header">
                <h5 class="service-name">{{ service.nameKey |
translate }}</h5>
                <div class="service-badges"></div>
            </div>
            <p class="service-description">{{
service.descriptionKey | translate }}</p>
            <div class="service-meta">
                <div class="meta-item">
                    <i class="fas fa-clock me-1"></i>
                    <small>{{ service.processingTime
}}</small>
                </div>
                <div class="meta-item">
                    <i class="fas fa-money-bill-wave me-
1"></i>
                    <small>{{ service.cost |
currency:service.currency:'symbol':'1.0-0' }}</small>
                </div>
                <div class="meta-item">
                    <i class="fas fa-building me-1"></i>
                    <small>{{ service.providedBy }}</small>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>
      </div>
    </div>
  </div>
</div>
</div>
</div>

```

- `procedures.component.scss :src/app/pages/procedures/procedures.component.scss:`

```

.procedures-page {
  min-height: 80vh;

  .page-header {
    .page-title {
      font-size: 2.5rem;
      font-weight: 700;
      color: var(--dark-color);
      margin-bottom: 0.5rem;
    }

    .search-container {
      .input-group {
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
        border-radius: 25px;
        overflow: hidden;

        .form-control {
          border: none;
          padding: 0.75rem 1.5rem;

          &:focus {
            box-shadow: none;
          }
        }

        .btn {
          border: none;
          padding: 0.75rem 1.5rem;
        }
      }
    }
  }

  .category-card {
    background: white;
    border-radius: 12px;
  }
}

```

```
padding: 2rem;
text-align: center;
cursor: pointer;
transition: all 0.3s ease;
box-shadow: 0 3px 15px rgba(0, 0, 0, 0.1);
height: 100%;

&:hover {
  transform: translateY(-8px);
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
}

.category-icon {
  width: 70px;
  height: 70px;
  margin: 0 auto 1.5rem;
  background: var(--primary-color);
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  font-size: 1.8rem;
}

.category-name {
  font-size: 1.25rem;
  font-weight: 600;
  margin-bottom: 1rem;
  color: var(--dark-color);
}

.category-description {
  color: #666;
  margin-bottom: 1.5rem;
  line-height: 1.5;
}

.category-meta {
  .service-count {
    background: #f8f9fa;
    padding: 0.25rem 0.75rem;
    border-radius: 15px;
    font-size: 0.8rem;
    color: #666;
  }
}
}
```



```
.services-grid {
  .service-item {
    background: white;
    border-radius: 12px;
    padding: 1.5rem;
    margin-bottom: 1rem;
    cursor: pointer;
    transition: all 0.3s ease;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);

    &:hover {
      transform: translateX(8px);
      box-shadow: 0 5px 20px rgba(0, 0, 0, 0.15);
    }

    .service-header {
      display: flex;
      justify-content: space-between;
      align-items: flex-start;
      margin-bottom: 1rem;

      .service-name {
        font-size: 1.2rem;
        font-weight: 600;
        color: var(--primary-color);
        margin-bottom: 0;
      }

      .service-badges {
        .badge {
          font-size: 0.7rem;
        }
      }
    }

    .service-description {
      color: #666;
      margin-bottom: 1rem;
      line-height: 1.5;
    }

    .service-meta {
      display: flex;
      flex-wrap: wrap;
      gap: 1rem;
      padding-top: 1rem;
      border-top: 1px solid #eee;

      .meta-item {
```

```
        color: #888;
        font-size: 0.85rem;

        i {
            color: var(--primary-color);
        }
    }
}

@media (max-width: 768px) {
    .procedures-page {
        .page-header {
            .page-title {
                font-size: 2rem;
            }

            .search-container {
                margin-top: 1rem;
            }

            .service-item .service-header {
                flex-direction: column;
                align-items: flex-start;

                .service-badges {
                    margin-top: 0.5rem;
                }
            }
        }
    }
}
```

- registration.component.ts :src/app/pages/registration/registration.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators, AbstractControl } from
'@angular/forms';
import { Router } from '@angular/router';
import { TranslateService } from '@ngx-translate/core';
import { AuthService } from '../services/auth.service';
import { RegistrationData } from '../models/user.model';

@Component({
  selector: 'app-registration',
  templateUrl: './registration.component.html',
```

```

    styleUrls: ['./registration.component.scss']
  })
}
export class RegistrationComponent implements OnInit {
  registrationForm: FormGroup;
  currentStep = 1;
  totalSteps = 2; // We're not implementing OTP for now
  isLoading = false;
  errorMessage = '';
  successMessage = '';
  showPassword = false;
  showConfirmPassword = false;

  constructor(
    private fb: FormBuilder,
    private router: Router,
    private authService: AuthService,
    private translate: TranslateService
  ) {
    this.registrationForm = this.fb.group({
      accountType: ['local', Validators.required],
      cniNumber: [''],
      consularCardNumber: [''],
      phoneNumber: ['', [Validators.required, Validators.pattern(/^\+[+]?[0-9]{8,15}$/)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]],
      confirmPassword: ['', Validators.required],
      acceptedTerms: [false, Validators.requiredTrue]
    }, { validators: this.passwordMatchValidator });
  }

  ngOnInit(): void {
    this.onAccountTypeChange();
  }

  passwordMatchValidator(control: AbstractControl): { [key: string]: any } | null {
    const password = control.get('password');
    const confirmPassword = control.get('confirmPassword');

    if (!password || !confirmPassword) {
      return null;
    }

    return password.value !== confirmPassword.value ? { 'passwordMismatch': true } : null;
  }

  onAccountTypeChange(): void {

```

```

const accountType = this.registrationForm.get('accountType')?.value;

if (accountType === 'local') {
  this.registrationForm.get('cniNumber')?.setValidators([Validators.required, Validators.pattern(/^[\d]{8,12}$/)]);
  this.registrationForm.get('consularCardNumber')?.clearValidators();
} else {
  this.registrationForm.get('consularCardNumber')?.setValidators([Validators.required, Validators.pattern(/^[\dA-Z]{8,15}$/)]);
  this.registrationForm.get('cniNumber')?.clearValidators();
}

this.registrationForm.get('cniNumber')?.updateValueAndValidity();
this.registrationForm.get('consularCardNumber')?.updateValueAndValidity();
}

nextStep(): void {
  if (this.currentStep === 1) {
    // Validate step 1 fields
    const step1Fields = ['accountType', 'phoneNumber', 'email'];
    if (this.registrationForm.get('accountType')?.value === 'local') {
      step1Fields.push('cniNumber');
    } else {
      step1Fields.push('consularCardNumber');
    }

    let isStep1Valid = true;
    step1Fields.forEach(field => {
      const control = this.registrationForm.get(field);
      if (control && control.invalid) {
        control.markAsTouched();
        isStep1Valid = false;
      }
    });

    if (isStep1Valid) {
      this.currentStep = 2;
    }
  }
}

previousStep(): void {
  if (this.currentStep > 1) {
    this.currentStep--;
  }
}

onSubmit(): void {
  if (this.registrationForm.valid) {

```

```

this.isLoading = true;
this.errorMessage = '';

const registrationData: RegistrationData = {
  accountType: this.registrationForm.value.accountType,
  cniNumber: this.registrationForm.value.cniNumber,
  consularCardNumber: this.registrationForm.value.consularCardNumber,
  phoneNumber: this.registrationForm.value.phoneNumber,
  email: this.registrationForm.value.email,
  password: this.registrationForm.value.password,
  confirmPassword: this.registrationForm.value.confirmPassword,
  acceptedTerms: this.registrationForm.value.acceptedTerms
};

this.authService.register(registrationData).subscribe({
  next: (response) => {
    this.isLoading = false;
    if (response.success) {
      this.successMessage = response.message;
      setTimeout(() => {
        this.router.navigate(['/home']);
      }, 2000);
    } else {
      this.errorMessage = response.message;
    }
  },
  error: (error) => {
    this.isLoading = false;
    this.errorMessage = 'Registration failed. Please try again.';
  }
});
} else {
  this.markFormGroupTouched();
}
}

private markFormGroupTouched(): void {
  Object.keys(this.registrationForm.controls).forEach(key => {
    const control = this.registrationForm.get(key);
    if (control) {
      control.markAsTouched();
    }
  });
}

togglePasswordVisibility(field: 'password' | 'confirmPassword'): void {
  if (field === 'password') {
    this.showPassword = !this.showPassword;
  } else {

```

```

    this.showConfirmPassword = !this.showConfirmPassword;
  }
}
}

```

- registration.component.html :src/app/pages/registration/registration.component.html:

```

<div class="registration-page">
  <div class="container py-5">
    <div class="row justify-content-center">
      <div class="col-lg-8">
        <div class="registration-card">

          <!-- Header -->
          <div class="card-header text-center">

            <h2 class="card-title">{{ 'forms.registration.title' |
translate }}</h2>

            <!-- Progress Steps -->
            <div class="progress-steps mb-4">
              <div class="step" [class.active]="currentStep >=
1" [class.completed]="currentStep > 1">
                <div class="step-circle">1</div>
                <span class="step-label">{{
'forms.registration.step1' | translate }}</span>
              </div>
              <div class="step-line"
[class.completed]="currentStep > 1"></div>
              <div class="step" [class.active]="currentStep >=
2">
                <div class="step-circle">2</div>
                <span class="step-label">{{
'forms.registration.step3' | translate }}</span>
              </div>
            </div>

            <!-- Form -->
            <div class="card-body">

              <!-- Success Message -->
              <div class="alert alert-success text-center"
*ngIf="successMessage" role="alert">
                <i class="fas fa-check-circle me-2"></i>
                {{ successMessage }}
              </div>

```

```

        <!-- Error Message -->
        <div class="alert alert-danger" *ngIf="errorMessage"
role="alert">
            <i class="fas fa-exclamation-triangle me-2"></i>
            {{ errorMessage }}
        </div>

        <form [formGroup]="registrationForm"
(ngSubmit)="onSubmit()">

            <!-- Step 1: Basic Information -->
            <div *ngIf="currentStep === 1" class="step-
content">

                <h4 class="step-title mb-4">Account
Information</h4>

                <!-- Account Type -->
                <div class="mb-4">
                    <label class="form-label">{{
'forms.registration.account_type' | translate
                    }}</label>
                    <div class="account-type-selection">
                        <div class="form-check form-check-
card">
                            <input class="form-check-input"
type="radio" value="local"
                            formControlName="accountType"
id="local"
                            (change)="onAccountTypeChange(
)">
                            <label class="form-check-label"
for="local">
                                <div class="card-content">
                                    <i class="fas fa-home fa-
2x mb-2"></i>
                                    <h6>{{
'forms.registration.local_citizen' | translate }}</h6>
                                    <small class="text-
muted">Resident in Cameroon</small>
                                </div>
                            </label>
                        </div>
                        <div class="form-check form-check-
card">
                            <input class="form-check-input"
type="radio" value="diaspora"
                            formControlName="accountType"
id="diaspora"

```

```

                                (change)="onAccountTypeChange(
)">
                                <label class="form-check-label"
for="diaspora">
                                <div class="card-content">
                                    <i class="fas fa-globe fa-
2x mb-2"></i>
                                    <h6>{{
'forms.registration.diaspora_citizen' | translate }}</h6>
                                    <small class="text-
muted">Living abroad</small>
                                </div>
                                </label>
                                </div>
                                </div>
                                </div>
                                <!-- CNI Number (for local citizens) -->
                                <div class="mb-3"
*ngIf="registrationForm.get('accountType')?.value === 'local'">
                                    <label class="form-label">{{
'forms.registration.cni_number' | translate }}</label>
                                    <input type="text" class="form-control"
formControlName="cniNumber"
                                    [class.is-
invalid]="registrationForm.get('cniNumber')?.invalid &&
registrationForm.get('cniNumber')?.touched">
                                        <div class="invalid-feedback"
*ngIf="registrationForm.get('cniNumber'
)?.invalid && registrationForm.get('cniNumber')?.touched">
                                            <small
*ngIf="registrationForm.get('cniNumber')?.errors?.['required']">CNI
number is required</small>
                                            <small
*ngIf="registrationForm.get('cniNumber')?.errors?.['pattern']">Invalid
CNI number format</small>
                                        </div>
                                    </div>
                                <!-- Consular Card Number (for diaspora
citizens) -->
                                <div class="mb-3"
*ngIf="registrationForm.get('accountType')?.value === 'diaspora'">
                                    <label class="form-label">{{
'forms.registration.consular_card' | translate
                                    }}</label>
                                    <input type="text" class="form-control"
formControlName="consularCardNumber"

```



```

        [class.is-
invalid]="registrationForm.get('consularCardNumber')?.invalid &&
registrationForm.get('consularCardNumber')?.touched">
            <div class="invalid-feedback"
                *ngIf="registrationForm.get('consularC
ardNumber')?.invalid && registrationForm.get('consularCardNumber')?.touched">
                    <small
                        *ngIf="registrationForm.get('consu
larCardNumber')?.errors?.['required']">Consular
                        card number is required</small>
                    <small
                        *ngIf="registrationForm.get('consu
larCardNumber')?.errors?.['pattern']">Invalid
                        consular card format</small>
                    </div>
                </div>

        <!-- Phone Number -->
        <div class="mb-3">
            <label class="form-label">{{
'forms.registration.phone_number' | translate
            }}</label>
            <input type="tel" class="form-control"
formControlName="phoneNumber"
                [class.is-
invalid]="registrationForm.get('phoneNumber')?.invalid &&
registrationForm.get('phoneNumber')?.touched">
                    <div class="invalid-feedback"
                        *ngIf="registrationForm.get('phoneNumb
er')?.invalid && registrationForm.get('phoneNumber')?.touched">
                            <small
                                *ngIf="registrationForm.get('phoneNumber')?.errors?.['required']">Phone
                                number is required</small>
                            <small
                                *ngIf="registrationForm.get('phoneNumber')?.errors?.['pattern']">Invalid
                                phone number format</small>
                            </div>
                        </div>

        <!-- Email -->
        <div class="mb-4">
            <label class="form-label">{{
'forms.registration.email' | translate }}</label>
            <input type="email" class="form-control"
formControlName="email"
                [class.is-
invalid]="registrationForm.get('email')?.invalid &&
registrationForm.get('email')?.touched">
                    <div class="invalid-feedback"

```

```

                *ngIf="registrationForm.get('email')?.
invalid && registrationForm.get('email')?.touched">
                <small
*ngIf="registrationForm.get('email')?.errors?.['required']">Email is
                required</small>
                <small
*ngIf="registrationForm.get('email')?.errors?.['email']">Invalid email
                format</small>
            </div>
        </div>

        <button type="button" class="btn btn-primary
w-100" (click)="nextStep()">
            {{ 'buttons.continue' | translate }}
            <i class="fas fa-arrow-right ms-2"></i>
        </button>
    </div>

    <!-- Step 2: Password -->
    <div *ngIf="currentStep === 2" class="step-
content">
        <h4 class="step-title mb-4">Create
Password</h4>

        <!-- Password -->
        <div class="mb-3">
            <label class="form-label">{{
'forms.registration.password' | translate }}</label>
            <div class="input-group">
                <input [type]="showPassword ? 'text' :
'password'" class="form-control"
                    formControlName="password"
                    [class.is-
invalid]="registrationForm.get('password')?.invalid &&
registrationForm.get('password')?.touched">
                    <button type="button" class="btn btn-
outline-secondary"
                        (click)="togglePasswordVisibility(
'password')">
                        <i class="fas" [class.fa-
eye]="!showPassword"
                            [class.fa-eye-
slash]="showPassword"></i>
                    </button>
                </div>
                <div class="invalid-feedback"
                    *ngIf="registrationForm.get('password'
)?.invalid && registrationForm.get('password')?.touched">

```

```

                <small
*ngIf="registrationForm.get('password')?.errors?.['required']">Password
                is required</small>
                <small
*ngIf="registrationForm.get('password')?.errors?.['minlength']">Password
                must be at least 6
characters</small>
            </div>
        </div>

        <!-- Confirm Password -->
        <div class="mb-3">
            <label class="form-label">{{
'forms.registration.confirm_password' | translate
            }}</label>
            <div class="input-group">
                <input [type]="showConfirmPassword ?
'text' : 'password'" class="form-control"
                        FormControlName="confirmPassword"
                        [class.is-
invalid]="(registrationForm.get('confirmPassword')?.invalid &&
registrationForm.get('confirmPassword')?.touched) ||
registrationForm.errors?.['passwordMismatch']">
                <button type="button" class="btn btn-
outline-secondary"
                        (click)="togglePasswordVisibility(
'confirmPassword')">
                <i class="fas" [class.fa-
eye]="!showConfirmPassword"
                        [class.fa-eye-
slash]="showConfirmPassword"></i>
                </button>
            </div>
            <div class="invalid-feedback"
                *ngIf="(registrationForm.get('confirmP
assword')?.invalid && registrationForm.get('confirmPassword')?.touched) ||
registrationForm.errors?.['passwordMismatch']">
                <small
                    *ngIf="registrationForm.get('confi
rmPassword')?.errors?.['required']">Confirm
                    password is required</small>
                <small
*ngIf="registrationForm.errors?.['passwordMismatch']">Passwords do not
                    match</small>
            </div>
        </div>

        <!-- Terms Acceptance -->
        <div class="mb-4">

```

```

        <div class="form-check">
            <input class="form-check-input"
type="checkbox" formControlName="acceptedTerms"
                id="acceptTerms"
                [class.is-
invalid]="registrationForm.get('acceptedTerms')?.invalid &&
registrationForm.get('acceptedTerms')?.touched">
                <label class="form-check-label"
for="acceptTerms">
                    {{
'forms.registration.terms_acceptance' | translate }}
                </label>
                <div class="invalid-feedback"
                    *ngIf="registrationForm.get('accep
tedTerms')?.invalid && registrationForm.get('acceptedTerms')?.touched">
                    You must accept the terms and
conditions
                </div>
            </div>
        </div>

<!-- Buttons -->
<div class="row">
    <div class="col-6">
        <button type="button" class="btn btn-
outline-secondary w-100"
            (click)="previousStep()">
            <i class="fas fa-arrow-left me-
2"></i>
            {{ 'buttons.back' | translate }}
        </button>
    </div>
    <div class="col-6">
        <button type="submit" class="btn btn-
primary w-100" [disabled]="isLoading">
            <span *ngIf="isLoading"
                class="spinner-border spinner-
border-sm me-2"></span>
            {{
'forms.registration.register_button' | translate }}
        </button>
    </div>
</div>
</div>
</form>

<!-- Login Link -->
<div class="text-center mt-4" *ngIf="!successMessage">

```

```

        <span class="text-muted">{{
'forms.registration.already_have_account' | translate }}</span>
        <a routerLink="/home" class="text-primary text-
decoration-none ms-1">
            {{ 'forms.registration.login_link' | translate
}}
        </a>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>

```

- registration.component.scss :src/app/pages/registration/registration.component.scss:

```

.registration-page {
  background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
  min-height: 100vh;

  .registration-card {
    background: white;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    overflow: hidden;

    .card-header {
      background: linear-gradient(135deg, var(--primary-color), var(--
secondary-color));
      color: white;
      padding: 2rem;

      .card-title {
        font-weight: 700;
        margin-bottom: 1rem;
      }
    }

    .card-body {
      padding: 2rem;
    }
  }

  .progress-steps {
    display: flex;
    align-items: center;
    justify-content: center;
  }
}

```

```
margin: 2rem 0;

.step {
  display: flex;
  flex-direction: column;
  align-items: center;
  opacity: 0.5;
  transition: all 0.3s ease;

  &.active {
    opacity: 1;
  }

  &.completed {
    opacity: 1;

    .step-circle {
      background-color: var(--success-color);
    }
  }

  .step-circle {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    background-color: rgba(255, 255, 255, 0.3);
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
    margin-bottom: 0.5rem;
    transition: all 0.3s ease;
  }

  .step-label {
    font-size: 0.8rem;
    text-align: center;
  }
}

.step-line {
  width: 60px;
  height: 2px;
  background-color: rgba(255, 255, 255, 0.3);
  margin: 0 1rem;
  transition: all 0.3s ease;

  &.completed {
    background-color: var(--success-color);
  }
}
```

```

    }
  }
}

.step-content {
  .step-title {
    color: var(--primary-color);
    font-weight: 600;
    margin-bottom: 1.5rem;
  }
}

.account-type-selection {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;

  .form-check-card {
    margin: 0;

    .form-check-input {
      display: none;

      &:checked+.form-check-label {
        border-color: var(--primary-color);
        background-color: rgba(26, 115, 232, 0.05);

        .card-content {
          color: var(--primary-color);
        }
      }
    }
  }

  .form-check-label {
    border: 2px solid #dee2e6;
    border-radius: 12px;
    padding: 1.5rem;
    cursor: pointer;
    transition: all 0.3s ease;
    display: block;
    text-align: center;

    &:hover {
      border-color: var(--primary-color);
      transform: translateY(-2px);
    }

    .card-content {
      h6 {

```

```

        margin-bottom: 0.5rem;
        font-weight: 600;
    }
}

.form-control {
    border-radius: 8px;
    padding: 0.75rem 1rem;
    border: 1px solid #dee2e6;
    transition: all 0.3s ease;

    &:focus {
        border-color: var(--primary-color);
        box-shadow: 0 0 0 0.2rem rgba(26, 115, 232, 0.25);
    }
}

.btn {
    border-radius: 8px;
    padding: 0.75rem 1.5rem;
    font-weight: 500;
    transition: all 0.3s ease;

    &:hover {
        transform: translateY(-1px);
    }
}

}

@media (max-width: 768px) {
    .registration-page {
        .account-type-selection {
            grid-template-columns: 1fr;
        }

        .progress-steps {
            .step-label {
                font-size: 0.7rem;
            }

            .step-line {
                width: 40px;
            }
        }
    }
}
}
```



- search-files.component.ts :src/app/pages/search-files/search-files.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { TranslateService } from '@ngx-translate/core';
import { ApplicationService } from '../../services/application.service';
import { Application } from '../../models/application.model';

@Component({
  selector: 'app-search-files',
  templateUrl: './search-files.component.html',
  styleUrls: ['./search-files.component.scss']
})
export class SearchFilesComponent implements OnInit {
  searchForm: FormGroup;
  isLoading = false;
  searchResult: Application | null = null;
  errorMessage = '';
  hasSearched = false;

  constructor(
    private fb: FormBuilder,
    private applicationService: ApplicationService,
    private translate: TranslateService
  ) {
    this.searchForm = this.fb.group({
      trackingNumber: ['', [Validators.required, Validators.pattern(/^[MKG[0-9A-Z]{12}$/)]]
    });
  }

  ngOnInit(): void { }

  onSearch(): void {
    if (this.searchForm.valid) {
      this.isLoading = true;
      this.errorMessage = '';
      this.searchResult = null;
      this.hasSearched = true;

      const trackingNumber = this.searchForm.get('trackingNumber')?.value;

      this.applicationService.searchApplicationByTrackingNumber(trackingNumber)
        .subscribe({
          next: (application) => {
            this.isLoading = false;
            this.searchResult = application || null;
            if (!application) {

```

```

        this.errorMessage = 'No application found with this tracking
number.';
    }
},
error: (error) => {
    this.isLoading = false;
    this.errorMessage = 'Search failed. Please try again.';
    console.error('Search error:', error);
}
});
} else {
    this.searchForm.get('trackingNumber')?.markAsTouched();
}
}

clearSearch(): void {
    this.searchForm.reset();
    this.searchResult = null;
    this.errorMessage = '';
    this.hasSearched = false;
}

getStatusColor(status: string): string {
    switch (status) {
        case 'completed': return 'success';
        case 'processing': return 'primary';
        case 'payment_pending': return 'warning';
        case 'rejected': return 'danger';
        default: return 'secondary';
    }
}

getStatusIcon(status: string): string {
    switch (status) {
        case 'completed': return 'fas fa-check-circle';
        case 'processing': return 'fas fa-spinner fa-spin';
        case 'payment_pending': return 'fas fa-credit-card';
        case 'rejected': return 'fas fa-times-circle';
        default: return 'fas fa-clock';
    }
}

formatDate(date: Date | string): string {
    const dateObj = typeof date === 'string' ? new Date(date) : date;
    return dateObj.toLocaleDateString('en-US', {
        year: 'numeric',
        month: 'long',
        day: 'numeric'
    });
});

```

```
}  
}
```

- search-files.component.html :src/app/pages/search-files/search-files.component.html:

```
<div class="search-files-page">  
  <div class="container py-5">  
  
    <!-- Page Header -->  
    <div class="page-header text-center mb-5">  
      <div class="search-icon mb-3">  
        <i class="fas fa-search fa-3x text-primary"></i>  
      </div>  
      <h1 class="page-title">{{ 'navigation.search_files' | translate  
}}</h1>  
      <p class="page-subtitle">Track your application status using your  
tracking number</p>  
    </div>  
  
    <!-- Search Form -->  
    <div class="row justify-content-center mb-5">  
      <div class="col-lg-6">  
        <div class="search-card">  
          <form [formGroup]="searchForm" (ngSubmit)="onSearch()">  
            <div class="mb-3">  
              <label class="form-label">Application Tracking  
Number</label>  
              <div class="input-group input-group-lg">  
                <span class="input-group-text">  
                  <i class="fas fa-hashtag"></i>  
                </span>  
                <input type="text" class="form-control"  
formControlName="trackingNumber"  
placeholder="MKG123456789ABC"  
[class.is-  
invalid]="searchForm.get('trackingNumber')?.invalid &&  
searchForm.get('trackingNumber')?.touched">  
                <button type="submit" class="btn btn-primary"  
[disabled]="isLoading ||  
searchForm.invalid">  
                  <span *ngIf="isLoading" class="spinner-  
border spinner-border-sm me-2"></span>  
                  <i class="fas fa-search me-2"  
*ngIf="!isLoading"></i>  
                  {{ 'buttons.search' | translate }}  
                </button>  
              </div>  
              <div class="invalid-feedback"
```

```

                *ngIf="searchForm.get('trackingNumber')?.invalid
id && searchForm.get('trackingNumber')?.touched">
                <small
*ngIf="searchForm.get('trackingNumber')?.errors?.['required']">
                    Tracking number is required
                </small>
                <small
*ngIf="searchForm.get('trackingNumber')?.errors?.['pattern']">
                    Invalid tracking number format (e.g.,
MKG123456789ABC)
                </small>
            </div>
        </div>
        <div class="text-center">
            <button type="button" class="btn btn-outline-
secondary" *ngIf="hasSearched"
                (click)="clearSearch()">
                <i class="fas fa-times me-2"></i>
                Clear Search
            </button>
        </div>
    </form>
</div>
</div>
</div>

<!-- Error Message -->
<div class="row justify-content-center" *ngIf="errorMessage &&
!isLoading">
    <div class="col-lg-8">
        <div class="alert alert-warning text-center" role="alert">
            <i class="fas fa-exclamation-triangle me-2"></i>
            {{ errorMessage }}
        </div>
    </div>
</div>

<!-- Search Results -->
<div class="row justify-content-center" *ngIf="searchResult &&
!isLoading">
    <div class="col-lg-8">
        <div class="result-card">

            <!-- Application Header -->
            <div class="application-header">
                <div class="row align-items-center">
                    <div class="col-md-8">

```

```

        <h3 class="application-title">{{
searchResult.serviceName | translate }}</h3>
        <p class="application-meta">
            <strong>Tracking Number:</strong> {{
searchResult.trackingNumber }}
        </p>
        <p class="application-meta">
            <strong>Submitted:</strong> {{
formatDate(searchResult.submissionDate) }}
        </p>
    </div>
    <div class="col-md-4 text-md-end">
        <span class="badge badge-status" [class]='`bg-
' + getStatusColor(searchResult.status)`>
            <i
[class]="getStatusIcon(searchResult.status)" class="me-2"></i>
            {{ searchResult.status | titlecase }}
        </span>
    </div>
</div>
</div>

<!-- Application Timeline -->
<div class="application-timeline"
*ngIf="searchResult.timeline && searchResult.timeline.length > 0">
    <h5 class="timeline-title mb-4">
        <i class="fas fa-route me-2"></i>
        Application Progress
    </h5>

    <div class="timeline">
        <div class="timeline-item" *ngFor="let step of
searchResult.timeline"
            [class.completed]="step.isCompleted"
            [class.active]="step.status === 'in_progress'">
            <div class="timeline-marker"></div>
            <div class="timeline-content">
                <div class="timeline-header">
                    <h6 class="timeline-step">{{ step.step
}}</h6>
                    <span class="timeline-date">{{
formatDate(step.date) }}</span>
                </div>
                <p class="timeline-description">{{
step.description }}</p>
                <div class="timeline-duration"
*ngIf="step.estimatedDuration">
                    <i class="fas fa-clock me-1"></i>

```

```

                                {{ 'timeline.estimated_duration' |
translate }}: {{ step.estimatedDuration }}
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>

            <!-- Payment Information -->
            <div class="payment-info" *ngIf="searchResult.payments &&
searchResult.payments.length > 0">
                <h5 class="payment-title mb-3">
                    <i class="fas fa-credit-card me-2"></i>
                    Payment Information
                </h5>

                <div class="payment-item" *ngFor="let payment of
searchResult.payments">
                    <div class="row align-items-center">
                        <div class="col-md-6">
                            <strong>{{ payment.description }}</strong>
                            <br>
                            <small class="text-muted">Ref: {{
payment.referenceNumber }}</small>
                        </div>
                        <div class="col-md-3 text-center">
                            <span class="payment-amount">
                                {{ payment.amount |
currency:payment.currency:'symbol':'1.0-0' }}
                            </span>
                        </div>
                        <div class="col-md-3 text-end">
                            <span class="badge" [class]='bg-' +
getStatusColor(payment.status)">
                                {{ payment.status | titlecase }}
                            </span>
                            <br>
                            <small class="text-muted">{{
formatDate(payment.date) }}</small>
                        </div>
                    </div>
                </div>
            </div>

            <!-- Additional Information -->
            <div class="additional-info" *ngIf="searchResult.notes">
                <h5 class="info-title mb-3">
                    <i class="fas fa-info-circle me-2"></i>
                    Additional Information

```

```

        </h5>
        <p class="info-text">{{ searchResult.notes }}</p>
    </div>

</div>
</div>
</div>

<!-- Help Section -->
<div class="row justify-content-center mt-5" *ngIf="!searchResult ||
errorMessage">
    <div class="col-lg-8">
        <div class="help-section text-center">
            <h4 class="help-title">Need Help?</h4>
            <p class="help-text">
                If you can't find your application or need assistance,
please contact our support team.
            </p>
            <div class="help-actions">
                <button class="btn btn-outline-primary me-3">
                    <i class="fas fa-phone me-2"></i>
                    Contact Support
                </button>
                <button class="btn btn-outline-secondary">
                    <i class="fas fa-question-circle me-2"></i>
                    View FAQ
                </button>
            </div>
        </div>
    </div>
</div>
</div>
</div>

```

- search-files.component.scss :src/app/pages/search-files/search-files.component.scss:

```

.search-files-page {
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
    min-height: 100vh;

    .page-header {
        .search-icon {
            width: 100px;
            height: 100px;
            margin: 0 auto;
            background: linear-gradient(135deg, var(--primary-color), var(--
secondary-color));

```

```
        border-radius: 50%;
        display: flex;
        align-items: center;
        justify-content: center;
        color: white;
    }

    .page-title {
        font-size: 2.5rem;
        font-weight: 700;
        color: var(--dark-color);
        margin-bottom: 1rem;
    }

    .page-subtitle {
        font-size: 1.1rem;
        color: #666;
    }
}

.search-card {
    background: white;
    border-radius: 15px;
    padding: 2rem;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);

    .input-group-text {
        background-color: #f8f9fa;
        border-right: none;
    }

    .form-control {
        border-left: none;
        border-right: none;

        &:focus {
            box-shadow: none;
            border-color: var(--primary-color);
        }
    }

    .btn-primary {
        border-radius: 0 8px 8px 0;
    }
}

.result-card {
    background: white;
    border-radius: 15px;
```



```
box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
overflow: hidden;

.application-header {
  background: linear-gradient(135deg, #f8f9fa, #e9ecef);
  padding: 2rem;
  border-bottom: 1px solid #dee2e6;

  .application-title {
    color: var(--primary-color);
    font-weight: 700;
    margin-bottom: 1rem;
  }

  .application-meta {
    margin-bottom: 0.5rem;
    color: #666;
  }

  .badge-status {
    font-size: 1rem;
    padding: 0.5rem 1rem;
    border-radius: 20px;
  }
}

.application-timeline {
  padding: 2rem;
  border-bottom: 1px solid #dee2e6;

  .timeline-title {
    color: var(--dark-color);
    font-weight: 600;
  }
}

.payment-info {
  padding: 2rem;
  border-bottom: 1px solid #dee2e6;

  .payment-title {
    color: var(--dark-color);
    font-weight: 600;
  }

  .payment-item {
    background: #f8f9fa;
    padding: 1rem;
    border-radius: 8px;
  }
}
```

```

        margin-bottom: 1rem;

        .payment-amount {
            font-size: 1.2rem;
            font-weight: 600;
            color: var(--success-color);
        }
    }
}

.additional-info {
    padding: 2rem;

    .info-title {
        color: var(--dark-color);
        font-weight: 600;
    }

    .info-text {
        color: #666;
        line-height: 1.6;
    }
}

.help-section {
    .help-title {
        color: var(--dark-color);
        font-weight: 600;
        margin-bottom: 1rem;
    }

    .help-text {
        color: #666;
        margin-bottom: 2rem;
    }

    .help-actions {
        .btn {
            border-radius: 25px;
            padding: 0.75rem 1.5rem;
            font-weight: 500;
        }
    }
}

}

@media (max-width: 768px) {
    .search-files-page {

```

```

        .page-header .page-title {
            font-size: 2rem;
        }

        .result-card .application-header {
            text-align: center;

            .badge-status {
                margin-top: 1rem;
            }
        }

        .help-actions .btn {
            display: block;
            width: 100%;
            margin: 0.5rem 0;
        }
    }
}

```

- service-detail.component.ts :src/app/pages/service-detail/service-detail.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { NgbModal } from '@ng-bootstrap/ng-bootstrap';
import { TranslateService } from '@ngx-translate/core';
import { ServicesDataService } from '../../services/services-data.service';
import { AuthService } from '../../services/auth.service';
import { Service } from '../../models/service.model';
import { ServiceModalComponent } from '../../components/modals/service-modal/service-modal.component';
import { LoginModalComponent } from '../../components/modals/login-modal/login-modal.component';

@Component({
  selector: 'app-service-detail',
  templateUrl: './service-detail.component.html',
  styleUrls: ['./service-detail.component.scss']
})
export class ServiceDetailComponent implements OnInit {
  service: Service | null = null;
  isLoading = true;
  errorMessage = '';

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private modalService: NgbModal,

```

```

    private servicesDataService: ServicesDataService,
    private authService: AuthService,
    private translate: TranslateService
  ) { }

  ngOnInit(): void {
    this.route.params.subscribe(params => {
      const serviceId = params['serviceId'];
      if (serviceId) {
        this.loadService(serviceId);
      }
    });
  }

  private loadService(serviceId: string): void {
    this.servicesDataService.getServiceById(serviceId).subscribe({
      next: (service) => {
        this.service = service || null;
        this.isLoading = false;
        if (!service) {
          this.errorMessage = 'Service not found.';
        }
      },
      error: (error) => {
        console.error('Error loading service:', error);
        this.errorMessage = 'Failed to load service details.';
        this.isLoading = false;
      }
    });
  }

  onApplyForService(): void {
    if (!this.service) return;

    if (!this.authService.isAuthenticated()) {
      const loginModal = this.modalService.open(LoginModalComponent, {
        centered: true,
        backdrop: 'static'
      });
    }

    loginModal.result.then((result) => {
      if (result === 'login_success') {
        this.openServiceModal();
      }
    });
  } else {
    this.openServiceModal();
  }
}

```

```

private openServiceModal(): void {
  if (!this.service) return;

  const modalRef = this.modalService.open(ServiceModalComponent, {
    centered: true,
    size: 'lg'
  });

  modalRef.componentInstance.service = this.service;
}

formatCurrency(amount: number, currency: string): string {
  return new Intl.NumberFormat('en-US', {
    style: 'currency',
    currency: currency === 'XAF' ? 'XAF' : currency,
    minimumFractionDigits: 0
  }).format(amount);
}

goBack(): void {
  window.history.back();
}
}

```

- service-detail.component.html :src/app/pages/service-detail/service-detail.component.html:

```

<div class="service-detail-page">
  <div class="container py-4">

    <!-- Loading State -->
    <div class="text-center py-5" *ngIf="isLoading">
      <div class="spinner-border text-primary" role="status">
        <span class="visually-hidden">Loading...</span>
      </div>
    </div>

    <!-- Error State -->
    <div class="text-center py-5" *ngIf="errorMessage && !isLoading">
      <div class="error-icon mb-3">
        <i class="fas fa-exclamation-triangle fa-3x text-warning"></i>
      </div>
      <h3>{{ errorMessage }}</h3>
      <button class="btn btn-primary mt-3" (click)="goBack()">
        <i class="fas fa-arrow-left me-2"></i>
        Go Back
      </button>
    </div>
  </div>

```

```

<!-- Service Detail Content -->
<div *ngIf="service && !isLoading">

    <!-- Page Header -->
    <div class="page-header mb-4">
        <nav aria-label="breadcrumb">
            <ol class="breadcrumb">
                <li class="breadcrumb-item">
                    <a routerLink="/procedures">Procedures</a>
                </li>
                <li class="breadcrumb-item active">
                    {{ service.nameKey | translate }}
                </li>
            </ol>
        </nav>

        <div class="row align-items-center">
            <div class="col-md-8">
                <h1 class="service-title">{{ service.nameKey |
translate }}</h1>
                <p class="service-subtitle">{{ service.descriptionKey
| translate }}</p>
            </div>
            <div class="col-md-4 text-md-end">
                <div class="service-status">
                    <span class="badge badge-lg"
[ngClass]="service.isImplemented ? 'bg-
success' : 'bg-secondary'">
                    <i [ngClass]="service.isImplemented ? 'fas fa-
check-circle' : 'fas fa-clock'"
class="me-2"></i>
                    {{ service.isImplemented ? 'Available Online'
: 'Coming Soon' }}
                </span>
            </div>
        </div>
    </div>

    <!-- Service Overview -->
    <div class="service-overview mb-5">
        <div class="row g-4">
            <div class="col-lg-8">
                <div class="service-description-card">
                    <h3 class="card-title">
                        <i class="fas fa-info-circle me-2"></i>
                        Service Description
                    </h3>

```

```

        <div class="card-content">
            <p class="description-text">{{
service.detailedDescriptionKey | translate }}</p>

            <!-- Legal Texts -->
            <div class="legal-texts"
*ngIf="service.legalTexts && service.legalTexts.length > 0">
                <h5 class="mt-4 mb-3">Legal Framework</h5>
                <div class="legal-document" *ngFor="let
legalText of service.legalTexts">
                    <div class="d-flex justify-content-
between align-items-center">
                        <div>
                            <h6 class="legal-title">{{
legalText.title }}</h6>
                            <p class="legal-excerpt">{{
legalText.content | slice:0:150 }}...</p>
                        </div>
                        <button class="btn btn-outline-
primary btn-sm" *ngIf="legalText.pdfUrl">
                            <i class="fas fa-download me-
2"></i>
                            Download PDF
                        </button>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

<div class="col-lg-4">
    <div class="service-details-card">
        <h4 class="card-title">Service Details</h4>

        <div class="detail-item">
            <div class="detail-icon">
                <i class="fas fa-clock"></i>
            </div>
            <div class="detail-content">
                <strong>Processing Time</strong>
                <span>{{ service.processingTime }}</span>
            </div>
        </div>

        <div class="detail-item">
            <div class="detail-icon">
                <i class="fas fa-money-bill-wave"></i>
            </div>

```

```

        <div class="detail-content">
            <strong>Cost</strong>
            <span>{{ formatCurrency(service.cost,
service.currency) }}</span>
        </div>
    </div>

    <div class="detail-item">
        <div class="detail-icon">
            <i class="fas fa-building"></i>
        </div>
        <div class="detail-content">
            <strong>Provided By</strong>
            <span>{{ service.providedBy }}</span>
        </div>
    </div>

    <!-- Apply Button -->
    <button class="btn btn-primary btn-lg w-100 mt-4"
[disabled]="!service.isImplemented"
        (click)="onApplyForService()">
        <i class="fas fa-paper-plane me-2"></i>
        {{ service.isImplemented ? 'Apply Now' :
'Coming Soon' }}
    </button>

    <div class="alert alert-info mt-3"
*ngIf="!service.isImplemented">
        <small>
            <i class="fas fa-info-circle me-2"></i>
            This service will be available online
soon. Please visit the relevant office for
            now.
        </small>
    </div>
</div>
</div>
</div>
</div>
</div>

    <!-- Required Documents -->
    <div class="required-documents mb-5" *ngIf="service.requirements
&& service.requirements.length > 0">
        <h3 class="section-title">
            <i class="fas fa-file-alt me-2"></i>
            Required Documents
        </h3>

        <div class="row g-3">

```



```

        <div class="col-md-6" *ngFor="let requirement of
service.requirements">
            <div class="document-card">
                <div class="document-header">
                    <div class="document-icon">
                        <i class="fas fa-file-alt"></i>
                    </div>
                    <div class="document-info">
                        <h6 class="document-name">{{
requirement.nameKey | translate }}</h6>
                        <span class="badge" [class.bg-
danger]="requirement.isMandatory"
                                [class.bg-
secondary]="!requirement.isMandatory">
                            {{ requirement.isMandatory ?
'Required' : 'Optional' }}
                        </span>
                    </div>
                </div>
                <div class="document-description">{{
requirement.descriptionKey | translate }}</div>
            </div>
        </div>

<!-- Help Section -->
<div class="help-section">
    <div class="row g-4">
        <div class="col-md-6">
            <div class="help-card">
                <div class="help-icon">
                    <i class="fas fa-question-circle"></i>
                </div>
                <h5>Need Help?</h5>
                <p>Contact our support team for assistance with
your application.</p>
                <button class="btn btn-outline-primary">
                    <i class="fas fa-phone me-2"></i>
                    Contact Support
                </button>
            </div>
        </div>
        <div class="col-md-6">
            <div class="help-card">
                <div class="help-icon">
                    <i class="fas fa-search"></i>
                </div>
            </div>
        </div>
    </div>

```

```

        <h5>Track Your Application</h5>
        <p>Already submitted? Track the status of your
application.</p>
        <button class="btn btn-outline-primary"
routerLink="/search-files">
            <i class="fas fa-search me-2"></i>
            Track Application
        </button>
    </div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>

```

- service-detail.component.scss :src/app/pages/service-detail/service-detail.component.scss:

```

.service-detail-page {
  background: #f8f9fa;
  min-height: 100vh;

  .page-header {
    .breadcrumb {
      background: none;
      padding: 0;

      .breadcrumb-item a {
        color: var(--primary-color);
        text-decoration: none;

        &:hover {
          text-decoration: underline;
        }
      }
    }
  }

  .service-title {
    font-size: 2.5rem;
    font-weight: 700;
    color: var(--dark-color);
    margin-bottom: 0.5rem;
  }

  .service-subtitle {
    font-size: 1.1rem;
    color: #666;
    margin-bottom: 0;
  }
}

```

```
}

.service-status {
  .badge-lg {
    font-size: 1rem;
    padding: 0.75rem 1.5rem;
    border-radius: 25px;
  }
}

}

.service-overview {

  .service-description-card,
  .service-details-card {
    background: white;
    border-radius: 15px;
    padding: 2rem;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    height: 100%;

    .card-title {
      color: var(--primary-color);
      font-weight: 600;
      margin-bottom: 1.5rem;
    }

    .description-text {
      font-size: 1.1rem;
      line-height: 1.7;
      color: #333;
    }

    .legal-texts {
      .legal-document {
        background: #f8f9fa;
        padding: 1rem;
        border-radius: 8px;
        margin-bottom: 1rem;

        .legal-title {
          font-weight: 600;
          margin-bottom: 0.5rem;
        }

        .legal-excerpt {
          color: #666;
          font-size: 0.9rem;
          margin-bottom: 0;
        }
      }
    }
  }
}
```

```

    }
  }
}

.service-details-card {
  .detail-item {
    display: flex;
    align-items: flex-start;
    margin-bottom: 1.5rem;

    .detail-icon {
      width: 40px;
      height: 40px;
      background: var(--primary-color);
      color: white;
      border-radius: 50%;
      display: flex;
      align-items: center;
      justify-content: center;
      margin-right: 1rem;
      flex-shrink: 0;
    }

    .detail-content {
      flex: 1;

      strong {
        display: block;
        margin-bottom: 0.25rem;
        color: var(--dark-color);
      }

      span {
        color: #666;
      }
    }
  }
}

.required-documents {
  .section-title {
    font-size: 1.8rem;
    font-weight: 600;
    color: var(--dark-color);
    margin-bottom: 2rem;
  }
}

```

```
.document-card {
  background: white;
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  height: 100%;
  transition: all 0.3s ease;

  &:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
  }

  .document-header {
    display: flex;
    align-items: flex-start;
    margin-bottom: 1rem;

    .document-icon {
      width: 50px;
      height: 50px;
      background: var(--secondary-color);
      color: white;
      border-radius: 12px;
      display: flex;
      align-items: center;
      justify-content: center;
      margin-right: 1rem;
      font-size: 1.5rem;
    }

    .document-info {
      flex: 1;

      .document-name {
        font-weight: 600;
        margin-bottom: 0.5rem;
        color: var(--dark-color);
      }
    }
  }

  .document-description {
    color: #666;
    line-height: 1.5;
    margin-bottom: 0;
  }
}
```

```

    .help-section {
        .help-card {
            background: white;
            border-radius: 15px;
            padding: 2rem;
            text-align: center;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
            height: 100%;

            .help-icon {
                width: 80px;
                height: 80px;
                background: linear-gradient(135deg, var(--primary-color),
var(--secondary-color));
                color: white;
                border-radius: 50%;
                display: flex;
                align-items: center;
                justify-content: center;
                margin: 0 auto 1.5rem;
                font-size: 2rem;
            }

            h5 {
                font-weight: 600;
                margin-bottom: 1rem;
                color: var(--dark-color);
            }

            p {
                color: #666;
                margin-bottom: 1.5rem;
            }
        }
    }
}

@media (max-width: 768px) {
    .service-detail-page {
        .page-header {
            .service-title {
                font-size: 2rem;
            }

            .service-status {
                margin-top: 1rem;
                text-align: center;
            }
        }
    }
}

```

```

    }

    .service-overview {
      .service-details-card {
        margin-top: 2rem;
      }
    }
  }
}

```

- application.service.ts :src/app/services/application.service.ts:

```

import { Injectable } from '@angular/core';
import { Observable, of, BehaviorSubject } from 'rxjs';
import { delay, map } from 'rxjs/operators';
import {
  Application,
  ApplicationStatus,
  Payment,
  PaymentMethod,
  PaymentStatus,
  ApplicationTimeline,
  TimelineStatus
} from '../models/application.model';
import { ApplicantType } from '../models/service.model';

@Injectable({
  providedIn: 'root'
})
export class ApplicationService {
  private applicationsSubject = new BehaviorSubject<Application[]>([]);
  public applications$ = this.applicationsSubject.asObservable();

  constructor() {
    this.loadApplications();
  }

  private loadApplications(): void {
    const savedApplications = localStorage.getItem('mk-gov-applications');
    if (savedApplications) {
      const applications = JSON.parse(savedApplications);
      this.applicationsSubject.next(applications);
    }
  }

  private saveApplications(applications: Application[]): void {
    localStorage.setItem('mk-gov-applications', JSON.stringify(applications));
    this.applicationsSubject.next(applications);
  }
}

```

```

}

createApplication(serviceId: string, serviceName: string, applicantType:
ApplicantType, userId: string): Observable<Application> {
  const newApplication: Application = {
    id: 'app-' + Date.now(),
    userId,
    serviceId,
    serviceName,
    applicantType,
    status: ApplicationStatus.DRAFT,
    submissionDate: new Date(),
    documents: [],
    timeline: this.createInitialTimeline(serviceId),
    payments: [],
    trackingNumber: this.generateTrackingNumber()
  };

  const currentApplications = this.applicationsSubject.value;
  const updatedApplications = [...currentApplications, newApplication];
  this.saveApplications(updatedApplications);

  return of(newApplication).pipe(delay(500));
}

private createInitialTimeline(serviceId: string): ApplicationTimeline[] {
  if (serviceId === 'passport') {
    return [
      {
        id: 'step-1',
        step: 'Pre-enrollment',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Complete online pre-enrollment form on PASSCAM
platform',
        isCompleted: false,
        estimatedDuration: '10-15 minutes'
      },
      {
        id: 'step-2',
        step: 'Payment',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Pay passport fees (110,000 XAF) via MTN Money or
Orange Money',
        isCompleted: false,
        estimatedDuration: '5 minutes'
      },
    ];
  }
}

```



```

        id: 'step-3',
        step: 'Appointment Booking',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Schedule biometric enrollment appointment at DGSN
center',
        isCompleted: false,
        estimatedDuration: '5 minutes'
    },
    {
        id: 'step-4',
        step: 'Document Verification',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Submit required documents for verification',
        isCompleted: false,
        estimatedDuration: '15-20 minutes'
    },
    {
        id: 'step-5',
        step: 'Biometric Enrollment',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Biometric data capture (fingerprints and photo) at
DGSN center',
        isCompleted: false,
        estimatedDuration: '30 minutes'
    },
    {
        id: 'step-6',
        step: 'Production',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Passport production with advanced security features',
        isCompleted: false,
        estimatedDuration: '48 hours'
    },
    {
        id: 'step-7',
        step: 'Ready for Collection',
        status: TimelineStatus.PENDING,
        date: new Date(),
        description: 'Passport ready for collection at designated center',
        isCompleted: false,
        estimatedDuration: 'Immediate'
    }
];
} else if (serviceId === 'birth-certificate-copy') {
    return [

```

```

    {
      id: 'step-1',
      step: 'Form Submission',
      status: TimelineStatus.PENDING,
      date: new Date(),
      description: 'Submit request form with required information',
      isCompleted: false,
      estimatedDuration: '5-10 minutes'
    },
    {
      id: 'step-2',
      step: 'Payment',
      status: TimelineStatus.PENDING,
      date: new Date(),
      description: 'Pay certification fees (300 XAF total)',
      isCompleted: false,
      estimatedDuration: '2 minutes'
    },
    {
      id: 'step-3',
      step: 'Record Verification',
      status: TimelineStatus.PENDING,
      date: new Date(),
      description: 'Verification of birth record in civil registry',
      isCompleted: false,
      estimatedDuration: '30 minutes - 2 hours'
    },
    {
      id: 'step-4',
      step: 'Document Preparation',
      status: TimelineStatus.PENDING,
      date: new Date(),
      description: 'Preparation of certified copy with official seal',
      isCompleted: false,
      estimatedDuration: '15-30 minutes'
    },
    {
      id: 'step-5',
      step: 'Ready for Collection',
      status: TimelineStatus.PENDING,
      date: new Date(),
      description: 'Certified copy ready for collection',
      isCompleted: false,
      estimatedDuration: 'Immediate'
    }
  ];
}
return [];
}

```

```

private generateTrackingNumber(): string {
    const prefix = 'MKG';
    const timestamp = Date.now().toString().slice(-8);
    const random = Math.random().toString(36).substr(2, 4).toUpperCase();
    return `${prefix}${timestamp}${random}`;
}

simulatePayment(applicationId: string, amount: number, method:
PaymentMethod): Observable<Payment> {
    const payment: Payment = {
        id: 'pay-' + Date.now(),
        amount,
        currency: 'XAF',
        description: this.getPaymentDescription(applicationId),
        method,
        status: PaymentStatus.PROCESSING,
        date: new Date(),
        referenceNumber: this.generatePaymentReference(),
        isSimulated: true
    };

    return of(payment).pipe(
        delay(this.getPaymentProcessingDelay(method)),
        map(() => {
            // Simulate successful payment with 95% success rate
            const isSuccessfull = Math.random() > 0.05;
            payment.status = isSuccessfull ? PaymentStatus.COMPLETED :
PaymentStatus.FAILED;

            if (isSuccessfull) {
                this.updateApplicationPayment(applicationId, payment);
                this.updateApplicationTimeline(applicationId, 'payment');
            }

            return payment;
        })
    );
}

private getPaymentDescription(applicationId: string): string {
    const application = this.applicationsSubject.value.find(app => app.id ===
applicationId);
    if (!application) return 'Service application fee';

    switch (application.serviceId) {
        case 'passport':
            return 'Biometric passport application fee';
        case 'birth-certificate-copy':

```

```

        return 'Birth certificate copy certification fee';
    default:
        return 'Government service application fee';
    }
}

private getPaymentProcessingDelay(method: PaymentMethod): number {
    switch (method) {
        case PaymentMethod.MTN_MONEY:
        case PaymentMethod.ORANGE_MONEY:
            return 2000; // 2 seconds for mobile money
        case PaymentMethod.CREDIT_CARD:
            return 3000; // 3 seconds for credit card
        case PaymentMethod.BANK_TRANSFER:
            return 5000; // 5 seconds for bank transfer
        default:
            return 1000;
    }
}

private generatePaymentReference(): string {
    const prefix = 'PAY';
    const timestamp = Date.now().toString();
    const random = Math.random().toString(36).substr(2, 6).toUpperCase();
    return `${prefix}${timestamp}${random}`;
}

private updateApplicationPayment(applicationId: string, payment: Payment):
void {
    const applications = this.applicationsSubject.value;
    const application = applications.find(app => app.id === applicationId);
    if (application) {
        application.payments.push(payment);
        if (payment.status === PaymentStatus.COMPLETED) {
            application.status = ApplicationStatus.PROCESSING;
        }
        this.saveApplications(applications);
    }
}

private updateApplicationTimeline(applicationId: string, completedStep:
string): void {
    const applications = this.applicationsSubject.value;
    const application = applications.find(app => app.id === applicationId);
    if (application) {
        const timeline = application.timeline;
        let stepFound = false;

        for (let i = 0; i < timeline.length; i++) {

```

```

        const step = timeline[i];
        if (step.step.toLowerCase().includes(completedStep.toLowerCase()) ||
            (completedStep === 'payment' && step.id === 'step-2')) {
            step.status = TimelineStatus.COMPLETED;
            step.isCompleted = true;
            step.date = new Date();
            stepFound = true;

            // Mark next step as in progress
            if (i + 1 < timeline.length) {
                timeline[i + 1].status = TimelineStatus.IN_PROGRESS;
            }
            break;
        }
    }

    if (stepFound) {
        this.saveApplications(applications);
    }
}

updateApplicationStatus(applicationId: string, status: ApplicationStatus):
Observable<boolean> {
    const applications = this.applicationsSubject.value;
    const application = applications.find(app => app.id === applicationId);

    if (application) {
        application.status = status;

        // Update completion date if completed
        if (status === ApplicationStatus.COMPLETED) {
            application.completionDate = new Date();
            // Mark all timeline steps as completed
            application.timeline.forEach(step => {
                step.status = TimelineStatus.COMPLETED;
                step.isCompleted = true;
            });
        }

        this.saveApplications(applications);
        return of(true).pipe(delay(500));
    }

    return of(false);
}

getUserApplications(userId: string): Observable<Application[]> {
    return this.applications$.pipe(

```

```

        map(applications => applications.filter(app => app.userId === userId))
    );
}

getApplicationById(applicationId: string): Observable<Application |
undefined> {
    return this.applications$.pipe(
        map(applications => applications.find(app => app.id === applicationId))
    );
}

searchApplicationByTrackingNumber(trackingNumber: string):
Observable<Application | undefined> {
    return this.applications$.pipe(
        delay(1000), // Simulate API call
        map(applications => applications.find(app => app.trackingNumber ===
trackingNumber))
    );
}

getApplicationStatistics(): Observable<any> {
    return this.applications$.pipe(
        map(applications => {
            const total = applications.length;
            const completed = applications.filter(app => app.status ===
ApplicationStatus.COMPLETED).length;
            const processing = applications.filter(app => app.status ===
ApplicationStatus.PROCESSING).length;
            const pending = applications.filter(app => app.status ===
ApplicationStatus.DRAFT ||
app.status === ApplicationStatus.SUBMITTED).length;

            const serviceBreakdown = applications.reduce((acc, app) => {
                acc[app.serviceId] = (acc[app.serviceId] || 0) + 1;
                return acc;
            }, {} as any);

            return {
                total,
                completed,
                processing,
                pending,
                completionRate: total > 0 ? Math.round((completed / total) * 100) :
0,
                serviceBreakdown
            };
        })
    );
}

```

```

    simulateApplicationProgress(applicationId: string):
Observable<ApplicationTimeline[]> {
    return new Observable(observer => {
        const interval = setInterval(() => {
            const applications = this.applicationsSubject.value;
            const application = applications.find(app => app.id ===
applicationId);

            if (application) {
                const timeline = application.timeline;
                const nextPendingStep = timeline.find(step => step.status ===
TimelineStatus.PENDING);

                if (nextPendingStep) {
                    nextPendingStep.status = TimelineStatus.IN_PROGRESS;

                    setTimeout(() => {
                        nextPendingStep.status = TimelineStatus.COMPLETED;
                        nextPendingStep.isCompleted = true;
                        nextPendingStep.date = new Date();

                        // Check if all steps are completed
                        const allCompleted = timeline.every(step => step.isCompleted);
                        if (allCompleted) {
                            application.status = ApplicationStatus.COMPLETED;
                            application.completionDate = new Date();
                            clearInterval(interval);
                            observer.complete();
                        }

                        this.saveApplications(applications);
                        observer.next(timeline);
                    }, 2000);
                }
            }
        }, 5000); // Progress every 5 seconds

        // Stop simulation after 2 minutes
        setTimeout(() => {
            clearInterval(interval);
            observer.complete();
        }, 120000);
    });
}
}

```

- auth.service.ts :src/app/services/auth.service.ts:

```

import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable, of } from 'rxjs';
import { delay, map } from 'rxjs/operators';
import { User, LoginData, RegistrationData } from '../models/user.model';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private currentUserSubject = new BehaviorSubject<User | null>(null);
  public currentUser$ = this.currentUserSubject.asObservable();

  constructor() {
    this.initializeUser();
  }

  private initializeUser(): void {
    const savedUser = localStorage.getItem('mk-gov-user');
    if (savedUser) {
      const user = JSON.parse(savedUser);
      user.isAuthenticated = true;
      this.currentUserSubject.next(user);
    }
  }

  login(loginData: LoginData): Observable<{ success: boolean; message: string;
user?: User }> {
  // Simulate API call
  return of(null).pipe(
    delay(1000),
    map(() => {
      // Mock authentication logic
      const mockUser: User = {
        id: 'user-123',
        phoneNumber: loginData.identifier,
        email: loginData.identifier.includes('@') ? loginData.identifier :
'user@example.com',
        firstName: 'John',
        lastName: 'Doe',
        isDiaspora: false,
        isAuthenticated: true,
        registrationDate: new Date(),
        lastLoginDate: new Date()
      };

      this.currentUserSubject.next(mockUser);
      localStorage.setItem('mk-gov-user', JSON.stringify(mockUser));

      return {

```



```

        success: true,
        message: 'Login successful',
        user: mockUser
    });
    })
    );
}

register(registrationData: RegistrationData): Observable<{ success: boolean;
message: string; user?: User }> {
    // Simulate API call
    return of(null).pipe(
        delay(1500),
        map(() => {
            const newUser: User = {
                id: 'user-' + Date.now(),
                cniNumber: registrationData.cniNumber,
                consularCardNumber: registrationData.consularCardNumber,
                phoneNumber: registrationData.phoneNumber,
                email: registrationData.email,
                isDiaspora: registrationData.accountType === 'diaspora',
                isAuthenticated: true,
                registrationDate: new Date(),
                lastLoginDate: new Date()
            };

            this.currentUserSubject.next(newUser);
            localStorage.setItem('mk-gov-user', JSON.stringify(newUser));

            return {
                success: true,
                message: 'Registration successful',
                user: newUser
            };
        })
    );
}

logout(): void {
    this.currentUserSubject.next(null);
    localStorage.removeItem('mk-gov-user');
}

isAuthenticated(): boolean {
    return this.currentUserSubject.value?.isAuthenticated || false;
}

getCurrentUser(): User | null {
    return this.currentUserSubject.value;
}

```

```

    }

    sendPasswordResetCode(identifier: string): Observable<{ success: boolean;
message: string }> {
        return of(null).pipe(
            delay(1000),
            map(() => ({
                success: true,
                message: 'Password reset code sent successfully'
            })))
    });
}
}

```

- services-data.service.ts :src/app/services/services-data.service.ts:

```

import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { ServiceCategory, Service, DocumentType, ApplicantType } from
'../models/service.model';

@Injectable({
    providedIn: 'root'
})
export class ServicesDataService {
    private serviceCategories: ServiceCategory[] = [
        {
            id: 'police-justice',
            nameKey: 'categories.police_justice.name',
            descriptionKey: 'categories.police_justice.description',
            icon: 'fas fa-balance-scale',
            color: '#1a73e8',
            services: [
                {
                    id: 'passport',
                    categoryId: 'police-justice',
                    nameKey: 'services.passport.name',
                    descriptionKey: 'services.passport.description',
                    detailedDescriptionKey: 'services.passport.detailed_description',
                    requirements: [
                        {
                            id: 'passport-form',
                            nameKey: 'requirements.passport_form',
                            descriptionKey: 'requirements.passport_form_desc',
                            isMandatory: true,
                            documentType: DocumentType.FORM
                        },
                    ],
                },
            ],
        },
    ],
}

```

```
    id: 'birth-cert',
    nameKey: 'requirements.birth_certificate',
    descriptionKey: 'requirements.birth_cert_desc',
    isMandatory: true,
    documentType: DocumentType.BIRTH_CERTIFICATE
  },
  {
    id: 'national-id',
    nameKey: 'requirements.national_id',
    descriptionKey: 'requirements.national_id_desc',
    isMandatory: true,
    documentType: DocumentType.IDENTITY
  },
  {
    id: 'photos',
    nameKey: 'requirements.passport_photos',
    descriptionKey: 'requirements.passport_photos_desc',
    isMandatory: true,
    documentType: DocumentType.PHOTO
  },
  {
    id: 'residence-proof',
    nameKey: 'requirements.residence_proof',
    descriptionKey: 'requirements.residence_proof_desc',
    isMandatory: true,
    documentType: DocumentType.RESIDENCE_PROOF
  }
],
processingTime: '48 hours to 3 months',
cost: 110000,
currency: 'XAF',
providedBy: 'DGSN (Direction Générale de la Sûreté Nationale)',
legalTexts: [
  {
    id: 'passport-law-1',
    title: 'Loi N° 2016/007 du 12 juillet 2016 portant Code de la
nationalité camerounaise',
    content: 'Cette loi définit les conditions d\'acquisition, de
perte et de recouvrement de la nationalité camerounaise, ainsi que les droits
et devoirs qui en découlent...',
    pdfUrl: '/assets/legal/passport-law-2016.pdf'
  },
  {
    id: 'passport-decree',
    title: 'Décret N° 2019/286 du 29 mai 2019 fixant les modalités
d\'établissement du passeport biométrique',
    content: 'Ce décret fixe les conditions et modalités
d\'établissement, de délivrance et de gestion du passeport biométrique
camerounais...',
  }
]
```

```

        pdfUrl: '/assets/legal/passport-decree-2019.pdf'
    }
],
isImplemented: true,
canApplyFor: [ApplicantType.SELF, ApplicantType.CHILD]
},
{
    id: 'national-id',
    categoryId: 'police-justice',
    nameKey: 'services.national_id.name',
    descriptionKey: 'services.national_id.description',
    detailedDescriptionKey: 'services.national_id.detailed_description',
    requirements: [
        {
            id: 'birth-cert-ni',
            nameKey: 'requirements.birth_certificate',
            descriptionKey: 'requirements.birth_cert_desc',
            isMandatory: true,
            documentType: DocumentType.BIRTH_CERTIFICATE
        },
        {
            id: 'photos-ni',
            nameKey: 'requirements.passport_photos',
            descriptionKey: 'requirements.passport_photos_desc',
            isMandatory: true,
            documentType: DocumentType.PHOTO
        }
    ],
    processingTime: '2-4 weeks',
    cost: 5000,
    currency: 'XAF',
    providedBy: 'Ministry of Interior',
    isImplemented: false,
    canApplyFor: [ApplicantType.SELF]
},
{
    id: 'criminal-record',
    categoryId: 'police-justice',
    nameKey: 'services.criminal_record.name',
    descriptionKey: 'services.criminal_record.description',
    detailedDescriptionKey:
'services.criminal_record.detailed_description',
    requirements: [
        {
            id: 'id-card-cr',
            nameKey: 'requirements.national_id',
            descriptionKey: 'requirements.national_id_desc',
            isMandatory: true,
            documentType: DocumentType.IDENTITY
        }
    ]
}

```

```

    }
  ],
  processingTime: '1-3 days',
  cost: 1000,
  currency: 'XAF',
  providedBy: 'Ministry of Justice',
  isImplemented: false,
  canApplyFor: [ApplicantType.SELF]
}
]
},
{
  id: 'family',
  nameKey: 'categories.family.name',
  descriptionKey: 'categories.family.description',
  icon: 'fas fa-users',
  color: '#34a853',
  services: [
    {
      id: 'birth-certificate-copy',
      categoryId: 'family',
      nameKey: 'services.birth_certificate_copy.name',
      descriptionKey: 'services.birth_certificate_copy.description',
      detailedDescriptionKey:
'services.birth_certificate_copy.detailed_description',
      requirements: [
        {
          id: 'request-form',
          nameKey: 'requirements.request_form',
          descriptionKey: 'requirements.request_form_desc',
          isMandatory: true,
          documentType: DocumentType.FORM
        },
        {
          id: 'requester-id',
          nameKey: 'requirements.requester_id',
          descriptionKey: 'requirements.requester_id_desc',
          isMandatory: true,
          documentType: DocumentType.IDENTITY
        }
      ],
      processingTime: 'Immediate to 48 hours',
      cost: 300,
      currency: 'XAF',
      providedBy: 'Civil Registry Office',
      legalTexts: [
        {
          id: 'civil-status-law',

```

```

        title: 'Ordonnance N° 81-02 du 29 juin 1981 portant organisation
de l\'état civil',
        content: 'Cette ordonnance organise la tenue des registres
d\'état civil et définit les procédures de délivrance des actes...',
        pdfUrl: '/assets/legal/civil-status-law-1981.pdf'
    }
],
    isImplemented: true,
    canApplyFor: [ApplicantType.SELF, ApplicantType.OTHER]
},
{
    id: 'marriage-certificate',
    categoryId: 'family',
    nameKey: 'services.marriage_certificate.name',
    descriptionKey: 'services.marriage_certificate.description',
    detailedDescriptionKey:
'services.marriage_certificate.detailed_description',
    requirements: [
        {
            id: 'marriage-form',
            nameKey: 'requirements.request_form',
            descriptionKey: 'requirements.request_form_desc',
            isMandatory: true,
            documentType: DocumentType.FORM
        }
    ],
    processingTime: 'Immediate to 24 hours',
    cost: 300,
    currency: 'XAF',
    providedBy: 'Civil Registry Office',
    isImplemented: false,
    canApplyFor: [ApplicantType.SELF, ApplicantType.OTHER]
},
{
    id: 'death-certificate',
    categoryId: 'family',
    nameKey: 'services.death_certificate.name',
    descriptionKey: 'services.death_certificate.description',
    detailedDescriptionKey:
'services.death_certificate.detailed_description',
    requirements: [
        {
            id: 'death-form',
            nameKey: 'requirements.request_form',
            descriptionKey: 'requirements.request_form_desc',
            isMandatory: true,
            documentType: DocumentType.FORM
        }
    ]
},

```

```

        processingTime: 'Immediate to 24 hours',
        cost: 300,
        currency: 'XAF',
        providedBy: 'Civil Registry Office',
        isImplemented: false,
        canApplyFor: [ApplicantType.OTHER]
    }
]
},
{
    id: 'transport',
    nameKey: 'categories.transport.name',
    descriptionKey: 'categories.transport.description',
    icon: 'fas fa-car',
    color: '#ff6d01',
    services: [
        {
            id: 'driving-license',
            categoryId: 'transport',
            nameKey: 'services.driving_license.name',
            descriptionKey: 'services.driving_license.description',
            detailedDescriptionKey:
'services.driving_license.detailed_description',
            requirements: [
                {
                    id: 'driving-form',
                    nameKey: 'requirements.driving_form',
                    descriptionKey: 'requirements.driving_form_desc',
                    isMandatory: true,
                    documentType: DocumentType.FORM
                },
                {
                    id: 'medical-cert',
                    nameKey: 'requirements.medical_certificate',
                    descriptionKey: 'requirements.medical_cert_desc',
                    isMandatory: true,
                    documentType: DocumentType.AUTHORIZATION
                }
            ],
            processingTime: '2-4 weeks',
            cost: 25000,
            currency: 'XAF',
            providedBy: 'Ministry of Transport',
            isImplemented: false,
            canApplyFor: [ApplicantType.SELF]
        }
    ]
},
{

```

```
id: 'education',
nameKey: 'categories.education.name',
descriptionKey: 'categories.education.description',
icon: 'fas fa-graduation-cap',
color: '#9c27b0',
services: [
  {
    id: 'diploma-certification',
    categoryId: 'education',
    nameKey: 'services.diploma_certification.name',
    descriptionKey: 'services.diploma_certification.description',
    detailedDescriptionKey:
'services.diploma_certification.detailed_description',
    requirements: [
      {
        id: 'diploma-original',
        nameKey: 'requirements.diploma_original',
        descriptionKey: 'requirements.diploma_original_desc',
        isMandatory: true,
        documentType: DocumentType.AUTHORIZATION
      }
    ],
    processingTime: '1-2 weeks',
    cost: 5000,
    currency: 'XAF',
    providedBy: 'Ministry of Higher Education',
    isImplemented: false,
    canApplyFor: [ApplicantType.SELF]
  }
],
},
{
  id: 'business',
  nameKey: 'categories.business.name',
  descriptionKey: 'categories.business.description',
  icon: 'fas fa-briefcase',
  color: '#607d8b',
  services: [
    {
      id: 'business-registration',
      categoryId: 'business',
      nameKey: 'services.business_registration.name',
      descriptionKey: 'services.business_registration.description',
      detailedDescriptionKey:
'services.business_registration.detailed_description',
      requirements: [
        {
          id: 'business-form',
          nameKey: 'requirements.business_form',
```



```

        descriptionKey: 'requirements.business_form_desc',
        isMandatory: true,
        documentType: DocumentType.FORM
    }
],
processingTime: '1-2 weeks',
cost: 50000,
currency: 'XAF',
providedBy: 'CFCE (Centre de Formalités de Création
d\'Entreprises)',
isImplemented: false,
canApplyFor: [ApplicantType.SELF]
}
]
},
{
    id: 'public-service',
    nameKey: 'categories.public_service.name',
    descriptionKey: 'categories.public_service.description',
    icon: 'fas fa-user-tie',
    color: '#795548',
    services: [
        {
            id: 'civil-service-application',
            categoryId: 'public-service',
            nameKey: 'services.civil_service_application.name',
            descriptionKey: 'services.civil_service_application.description',
            detailedDescriptionKey:
'services.civil_service_application.detailed_description',
            requirements: [],
            processingTime: '1-3 months',
            cost: 0,
            currency: 'XAF',
            providedBy: 'Ministry of Public Service',
            isImplemented: false,
            canApplyFor: [ApplicantType.SELF]
        }
    ]
},
{
    id: 'land-construction',
    nameKey: 'categories.land_construction.name',
    descriptionKey: 'categories.land_construction.description',
    icon: 'fas fa-home',
    color: '#4caf50',
    services: [
        {
            id: 'land-title',
            categoryId: 'land-construction',

```

```

        nameKey: 'services.land_title.name',
        descriptionKey: 'services.land_title.description',
        detailedDescriptionKey: 'services.land_title.detailed_description',
        requirements: [],
        processingTime: '3-6 months',
        cost: 100000,
        currency: 'XAF',
        providedBy: 'Ministry of State Property',
        isImplemented: false,
        canApplyFor: [ApplicantType.SELF]
    }
]
},
{
    id: 'consular',
    nameKey: 'categories.consular.name',
    descriptionKey: 'categories.consular.description',
    icon: 'fas fa-passport',
    color: '#2196f3',
    services: [
        {
            id: 'visa-application',
            categoryId: 'consular',
            nameKey: 'services.visa_application.name',
            descriptionKey: 'services.visa_application.description',
            detailedDescriptionKey:
'services.visa_application.detailed_description',
            requirements: [],
            processingTime: '1-4 weeks',
            cost: 50000,
            currency: 'XAF',
            providedBy: 'Ministry of External Relations',
            isImplemented: false,
            canApplyFor: [ApplicantType.SELF]
        }
    ]
},
{
    id: 'health',
    nameKey: 'categories.health.name',
    descriptionKey: 'categories.health.description',
    icon: 'fas fa-heartbeat',
    color: '#f44336',
    services: [
        {
            id: 'health-facility-authorization',
            categoryId: 'health',
            nameKey: 'services.health_facility_authorization.name',

```

```

        descriptionKey:
'services.health_facility_authorization.description',
        detailedDescriptionKey:
'services.health_facility_authorization.detailed_description',
        requirements: [],
        processingTime: '1-3 months',
        cost: 200000,
        currency: 'XAF',
        providedBy: 'Ministry of Public Health',
        isImplemented: false,
        canApplyFor: [ApplicantType.SELF]
    }
]
},
{
    id: 'civic-life',
    nameKey: 'categories.civic_life.name',
    descriptionKey: 'categories.civic_life.description',
    icon: 'fas fa-flag',
    color: '#ff9800',
    services: [
        {
            id: 'event-authorization',
            categoryId: 'civic-life',
            nameKey: 'services.event_authorization.name',
            descriptionKey: 'services.event_authorization.description',
            detailedDescriptionKey:
'services.event_authorization.detailed_description',
            requirements: [],
            processingTime: '1-2 weeks',
            cost: 10000,
            currency: 'XAF',
            providedBy: 'Prefecture',
            isImplemented: false,
            canApplyFor: [ApplicantType.SELF]
        }
    ]
}
];

getServiceCategories(): Observable<ServiceCategory[]> {
    return of(this.serviceCategories);
}

getServicesByCategory(categoryId: string): Observable<Service[]> {
    const category = this.serviceCategories.find(cat => cat.id ===
categoryId);
    return of(category?.services || []);
}

```

```

getServiceById(serviceId: string): Observable<Service | undefined> {
  for (const category of this.serviceCategories) {
    const service = category.services.find(s => s.id === serviceId);
    if (service) {
      return of(service);
    }
  }
  return of(undefined);
}

searchServices(query: string): Observable<Service[]> {
  const results: Service[] = [];
  const searchTerm = query.toLowerCase();

  for (const category of this.serviceCategories) {
    for (const service of category.services) {
      if (service.nameKey.toLowerCase().includes(searchTerm) ||
          service.descriptionKey.toLowerCase().includes(searchTerm) ||
          service.detailedDescriptionKey.toLowerCase().includes(searchTerm)) {
        results.push(service);
      }
    }
  }
  return of(results);
}
}

```

- translation.service.ts :src/app/services/translation.service.ts:

```

import { Injectable } from '@angular/core';
import { TranslateService } from '@ngx-translate/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class TranslationService {
  private currentLanguageSubject = new BehaviorSubject<string>('en');
  public currentLanguage$ = this.currentLanguageSubject.asObservable();

  constructor(private translate: TranslateService) {
    this.initializeLanguage();
  }

  private initializeLanguage(): void {
    const savedLanguage = localStorage.getItem('mk-gov-language') || 'en';
    this.setLanguage(savedLanguage);
  }
}

```

```

}

setLanguage(language: string): void {
  this.translate.use(language);
  this.currentLanguageSubject.next(language);
  localStorage.setItem('mk-gov-language', language);
}

getCurrentLanguage(): string {
  return this.currentLanguageSubject.value;
}

getAvailableLanguages(): { code: string; name: string; flag: string }[] {
  return [
    { code: 'en', name: 'English', flag: 'US' },
    { code: 'fr', name: 'Français', flag: 'FR' },
    { code: 'pt', name: 'Português', flag: 'PT' }
  ];
}
}

```

- styles.scss :src/styles.scss:

```

/* You can add global styles to this file, and also import other style files
*/
@import 'bootstrap/scss/bootstrap';

// Font Awesome Import (fallback if CDN fails)
@import '~@fortawesome/fontawesome-free/css/all.min.css';

// Custom CSS Variables
:root {
  --primary-color: #1a73e8;
  --primary-color-dark: #1557b0;
  --secondary-color: #34a853;
  --warning-color: #fbbc04;
  --warning-color-dark: #e6a600;
  --danger-color: #ea4335;
  --success-color: #34a853;
  --dark-color: #202124;
  --light-color: #f8f9fa;
  --cameroon-green: #009639;
  --cameroon-red: #ce1126;
  --cameroon-yellow: #fcdd09;

  --font-family-primary: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  --border-radius: 8px;
  --box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

```

```

    --transition: all 0.3s ease;
}

// Global Styles
* {
    box-sizing: border-box;
}

body {
    font-family: var(--font-family-primary);
    line-height: 1.6;
    color: var(--dark-color);
    position: relative;
    background-color: #f5f5f5;
    margin: 0;
    padding: 0;
}

#root,
app-root {
    display: flex;
    flex-direction: column;
    min-height: 100vh;
}

// Font Awesome Icons Global Fix
.fas,
.far,
.fab,
.fal,
.fad {
    font-family: "Font Awesome 6 Free", "Font Awesome 6 Pro", "Font Awesome 6 Brands" !important;
    font-weight: 900;
    display: inline-block !important;
    font-style: normal;
    font-variant: normal;
    text-rendering: auto;
    line-height: 1;
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
}

.fab {
    font-family: "Font Awesome 6 Brands" !important;
    font-weight: 400 !important;
}

.far {

```

```

    font-weight: 400 !important;
}

// Specific icon fixes
.fa-check-circle::before {
    content: "\f058";
}

.fa-clock::before {
    content: "\f017";
}

// Badge and Status Elements
.badge {
    i {
        margin-right: 0.25rem;
        font-size: 0.9em;
        vertical-align: baseline;
    }

    &.bg-success {
        background-color: var(--success-color) !important;
        color: white !important;

        i {
            color: white !important;
        }
    }

    &.bg-secondary {
        background-color: #6c757d !important;
        color: white !important;

        i {
            color: white !important;
        }
    }
}

// Ensure icons are visible
i[class*="fa-"] {
    min-width: 1em;
    text-align: center;
}

// Category Icons with proper sizing
.category-icon {
    i {
        font-size: inherit !important;
    }
}

```

```

        color: inherit !important;
        opacity: 1 !important;
        visibility: visible !important;
        display: inline-block !important;
        line-height: 1;
        vertical-align: middle;
    }
}

// Service Status Indicators
.service-badges {
    .badge {
        font-size: 0.75rem;
        padding: 0.25rem 0.5rem;
        border-radius: 12px;
        font-weight: 500;
        text-transform: uppercase;
        letter-spacing: 0.02em;

        &.bg-success {
            background-color: #28a745 !important;
            color: white !important;
        }

        &.bg-secondary {
            background-color: #6c757d !important;
            color: white !important;
        }
    }
}

// Ensure pages take full height to push footer down
.home-page,
.procedures-page,
.registration-page,
.search-files-page,
.passport-application-page,
.birth-certificate-page,
.service-detail-page {
    flex: 1;
    min-height: calc(100vh - 80px);
}

// Navigation and Links
.breadcrumb {
    background: none;
    padding: 0;
    margin-bottom: 1rem;
}

```



```

.breadcrumb-item {
  a {
    color: var(--primary-color);
    text-decoration: none;
    transition: all 0.3s ease;

    &:hover {
      text-decoration: underline;
      color: var(--primary-color-dark);
    }
  }

  &.active {
    color: #6c757d;
  }

  &:not(:last-child)::after {
    content: ">";
    color: #6c757d;
    margin: 0 0.5rem;
  }
}

}

.service-detail-page {
  .service-status {
    .badge-lg {
      font-size: 1rem;
      padding: 0.75rem 1.5rem;
      border-radius: 25px;
      font-weight: 500;
      display: inline-flex;
      align-items: center;
      gap: 0.5rem;

      i.fas {
        font-size: 1rem;
        display: inline-block !important;
        width: auto;
        height: auto;
        line-height: 1;

        &.fa-check-circle {
          color: white;
        }

        &.fa-clock {
          color: white;
        }
      }
    }
  }
}

```

```

    }
  }
}

// Service Icons
.service-item {
  .meta-item i {
    color: var(--primary-color) !important;
    margin-right: 0.5rem;
  }
}

// Header Icons
.navbar .nav-link i,
.btn i {
  margin-right: 0.5rem;
}

// Custom Bootstrap Overrides
.btn-primary {
  background-color: var(--primary-color);
  border-color: var(--primary-color);
  border-radius: var(--border-radius);
  font-weight: 500;
  padding: 12px 24px;
  transition: var(--transition);

  &:hover {
    background-color: var(--primary-color-dark);
    border-color: var(--primary-color-dark);
    transform: translateY(-1px);
  }
}

.btn-success {
  background-color: var(--cameroon-green);
  border-color: var(--cameroon-green);
}

.card {
  border: none;
  border-radius: var(--border-radius);
  box-shadow: var(--box-shadow);
  transition: var(--transition);

  &:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
  }
}

```

```

    }
}

.form-control {
  border-radius: var(--border-radius);
  border: 1px solid #dadce0;
  padding: 12px 16px;
  transition: var(--transition);

  &:focus {
    border-color: var(--primary-color);
    box-shadow: 0 0 0 0.2rem rgba(26, 115, 232, 0.25);
  }
}

// Loading Overlay
.loading-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 9999;

  .loading-spinner {
    background-color: white;
    padding: 2rem;
    border-radius: var(--border-radius);
    text-align: center;
  }
}

// Modal and overlay fixes
.modal {
  z-index: 1055;
}

.modal-backdrop {
  z-index: 1050;
}

// Category Cards with Proper Icons
.category-card {
  background: white;
  border-radius: 15px;

```

```
padding: 2rem;
box-shadow: var(--box-shadow);
cursor: pointer;
transition: var(--transition);
height: 100%;

&:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 25px rgba(0, 0, 0, 0.2);
}

.category-icon {
  width: 80px;
  height: 80px;
  margin: 0 auto 1.5rem;
  background: linear-gradient(135deg, var(--primary-color), var(--
secondary-color));
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  font-size: 2rem;

  i {
    font-size: 2rem !important;
    line-height: 1;
  }
}

.category-name {
  font-size: 1.25rem;
  font-weight: 600;
  margin-bottom: 1rem;
  color: var(--dark-color);
  text-align: center;
}

.category-description {
  color: #666;
  text-align: center;
  margin-bottom: 1.5rem;
}

.services-list {
  border-top: 1px solid #eee;
  padding-top: 1rem;

  .services-title {
```

```

        font-size: 0.9rem;
        font-weight: 600;
        color: var(--primary-color);
        margin-bottom: 0.5rem;
    }

    .service-link {
        padding: 0.25rem 0;
        cursor: pointer;
        transition: all 0.3s ease;
        border-radius: 4px;
        padding-left: 0.5rem;

        &:hover {
            background-color: #f8f9fa;
            color: var(--primary-color);
            padding-left: 1rem;
        }

        i {
            color: var(--primary-color);
            font-size: 0.8rem;
        }
    }
}

// Service List Items
.service-item {
    background: white;
    border-radius: var(--border-radius);
    padding: 1.5rem;
    margin-bottom: 1rem;
    box-shadow: var(--box-shadow);
    cursor: pointer;
    transition: var(--transition);

    &:hover {
        transform: translateX(5px);
        box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
    }

    .service-name {
        font-size: 1.1rem;
        font-weight: 600;
        color: var(--primary-color);
        margin-bottom: 0.5rem;
    }
}

```

```

.service-description {
  color: #666;
  font-size: 0.9rem;
}

.service-meta {
  margin-top: 1rem;
  padding-top: 1rem;
  border-top: 1px solid #eee;

  .meta-item {
    display: inline-block;
    margin-right: 1rem;
    font-size: 0.8rem;
    color: #888;

    i {
      margin-right: 0.25rem;
      color: var(--primary-color);
    }
  }
}

```

// Timeline Styles

```

.timeline {
  position: relative;

  &:before {
    content: '';
    position: absolute;
    left: 20px;
    top: 0;
    bottom: 0;
    width: 2px;
    background: var(--primary-color);
  }

  .timeline-item {
    position: relative;
    margin-bottom: 2rem;
    padding-left: 60px;

    .timeline-marker {
      position: absolute;
      left: 12px;
      top: 0;
      width: 16px;
      height: 16px;

```

```

        border-radius: 50%;
        background: var(--primary-color);
        border: 3px solid white;
        box-shadow: 0 0 0 3px var(--primary-color);

        &.completed {
            background: var(--success-color);
            box-shadow: 0 0 0 3px var(--success-color);
        }

        &.pending {
            background: #ccc;
            box-shadow: 0 0 0 3px #ccc;
        }
    }

    .timeline-content {
        background: white;
        padding: 1.5rem;
        border-radius: var(--border-radius);
        box-shadow: var(--box-shadow);

        .timeline-title {
            font-weight: 600;
            margin-bottom: 0.5rem;
        }

        .timeline-description {
            color: #666;
            margin-bottom: 0.5rem;
        }

        .timeline-duration {
            font-size: 0.8rem;
            color: #888;
        }
    }
}

.app-container {
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    position: relative;
}

.main-content {
    flex: 1;

```

```

padding-top: 80px; // Account for fixed header
min-height: calc(100vh - 80px);
display: flex;
flex-direction: column;
}

// Footer positioning and visibility
.footer {
margin-top: auto;
flex-shrink: 0;
width: 100%;
position: relative;
z-index: 10;
background-color: #212529 !important;

// Ensure footer appears above other elements
&::before {
content: '';
position: absolute;
top: 0;
left: 0;
right: 0;
bottom: 0;
background-color: #212529;
z-index: -1;
}
}

// Cameroon Flag Colors Helper Classes
.bg-cameroon-green {
background-color: var(--cameroon-green) !important;
}

.bg-cameroon-red {
background-color: var(--cameroon-red) !important;
}

.bg-cameroon-yellow {
background-color: var(--cameroon-yellow) !important;
}

.text-cameroon-green {
color: var(--cameroon-green) !important;
}

.text-cameroon-red {
color: var(--cameroon-red) !important;
}

```



```
.text-cameroon-yellow {  
  color: var(--cameroon-yellow) !important;  
}
```

```
// Responsive Design
```

```
@media (max-width: 768px) {  
  .container-fluid {  
    padding-left: 15px;  
    padding-right: 15px;  
  }  
}
```

```
.btn {  
  width: 100%;  
  margin-bottom: 10px;  
}
```

```
.category-card {  
  .category-icon {  
    width: 60px;  
    height: 60px;  
    font-size: 1.5rem;  
  
    i {  
      font-size: 1.5rem !important;  
    }  
  }  
}
```

```
.service-status {  
  margin-top: 1rem;  
  text-align: center;  
  
  .badge-lg {  
    font-size: 0.9rem;  
    padding: 0.5rem 1rem;  
  }  
}
```

```
.main-content {  
  padding-top: 70px;  
}
```

```
}
```